

# SIH

PS = 26174

## **AI Human Activity Recognition for On-board BAS Experiments**

# Overall Requirements of the System

1. Process Video from fixed payload cameras
2. Recognize the actions involved in a predefined experiments
3. Understand the sequence of those actions
4. Suggest the next step
5. Detect Skipped step
6. Detect out-of-order step
7. Give voice based alerts
8. Generate a timestamped lightweight experiment log
9. Store the video locally
10. Provide a monitoring GUI
11. Run as an offline standalone system

# What is BAS ?

**BAS = Bhartiya Antariks Station** ,India's planned space station

ISRO's current roadmap targets the first BAS Module around 2028 , with the complete station targeted around 2035 . The station is intended to support scientific research including life sciences , pharmaceuticals , materials science and manufacturing.

ISRO's 2026 Indian Microgravity Experiments programme specifically describes BAS as a future platform for conducting scientific experiments in microgravity.

The intended scenario is roughly :

**BAS → Scientific Payload → Fixed Cameras → EDGE AI System**

The important architectural idea is :

Camera → AI perceptions → Action Understanding → Procedure State

Machine → Astronaut Assistance

# This isn't Ordinary HAR

Traditional Human Activity Recognition might classify :

1. Walking
2. Running
3. Sitting
4. Standing
5. Jumping
6. Waving

Our problem is closer to :

Experiment :

1. Step 1 : Open Container
2. Step 2 : Remove the red object
3. Step 3 : Place the red object

.....

The AI needs to understand:

Observed:

OPEN\_CONTAINER

↓

REMOVE\_RED

↓

PLACE\_RED

↓

REMOVE\_YELLOW

↓

...

Then compare it against:

Expected:

$S1 \rightarrow S2 \rightarrow S3 \rightarrow S4 \rightarrow S5 \rightarrow S6 \rightarrow S7$

This makes the system fundamentally a **temporal procedure-understanding system**.

# The Dataset Problem

The statement explicitly tells teams to generate their own dataset for :

1. Object Detection
2. Pose Estimation
3. Hand - Object Interaction

Based on the experiment steps

This is actually good news for us.

We don't need to train some gigantic general-purpose astronaut model from scratch.

We can define our own controlled experiment.

For example:

Experiment

A box contains:

-  red component
-  yellow component

Procedure:

S0 Idle

S1 Reach toward container

S2 Open container

S3 Pick red object

S4 Place red object in target

S5 Pick yellow object

S6 Place yellow object in target

S7 Close container

S8 Experiment complete

Then deliberately record:

Correct:

$S1 \rightarrow S2 \rightarrow S3 \rightarrow S4 \rightarrow S5 \rightarrow S6 \rightarrow S7$

Skipped:

$S1 \rightarrow S2 \rightarrow S3 \rightarrow S5$

Out of order:

$S1 \rightarrow S2 \rightarrow S5 \rightarrow S4$

Repeated:

$S1 \rightarrow S2 \rightarrow S3 \rightarrow S3 \rightarrow S4$

Wrong object:

$S1 \rightarrow S2 \rightarrow \text{PICK\_YELLOW}$

Now we have a **procedure validation benchmark**, not just an action-recognition dataset.

# Research Paper we will use

This is one of the most useful discoveries from the research

A 2026 ICLR paper called **Go Beyond Earth : Understanding Human Actions and Scenes in Microgravity Environments** introduced **MicroG-4M**.

The dataset contains :

**4,759** video clips

**50** action categories

**13,261** action annotations

**1,238** human-written captions

**7,000+** question-answer pairs

footage from real space missions plus simulations.

This is extremely relevant because the researchers explicitly found that models designed around terrestrial video understanding degrade in microgravity.

So we have scientific evidence supporting the exact concern behind ISRO's optional HMR requirement.

The dataset is also publicly available through Hugging Face.

**We should absolutely study MicroG-4M.**

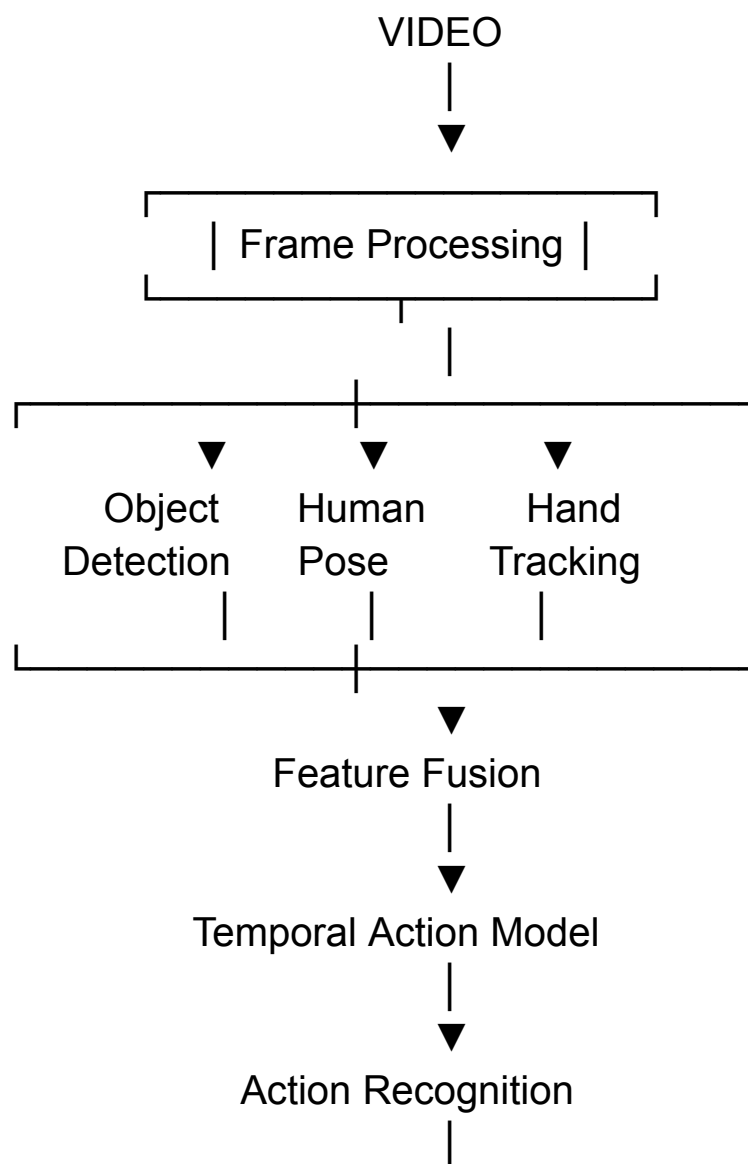
[MicroG-4M GitHub repository](#)

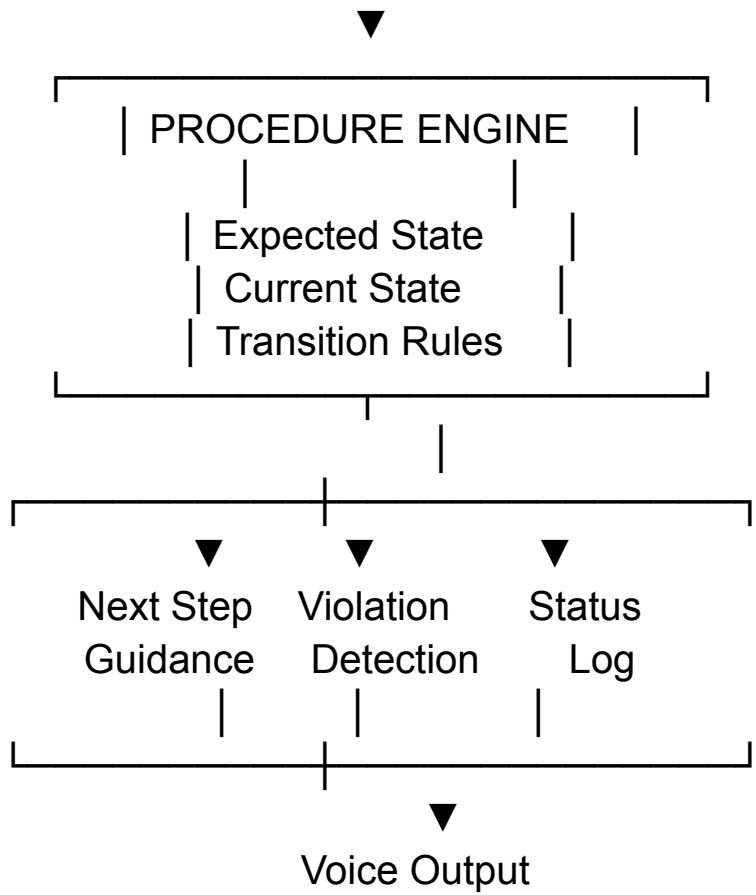
[MicroG-4M dataset on Hugging Face](#)

# What should OUR AI actually look like

I would not build a monolithic neural network

Instead :





9. Hand-object interaction is probably MORE important than full-body pose

This is something I think our team should emphasize.

Consider:

"Pick red object"

Full-body pose alone may not distinguish:

pick red  
pick yellow  
touch red  
touch yellow  
move hand near red

But:

hand trajectory  
+  
object identity  
+  
hand-object distance  
+  
object state

can.

For example:

Hand  
↓  
approaches RED  
↓

distance < threshold  
↓  
grasp detected  
↓  
RED moves with hand  
↓  
RED reaches target zone  
↓  
release

That's a much stronger representation of the action.

## 10. The real secret sauce: a Procedure State Machine

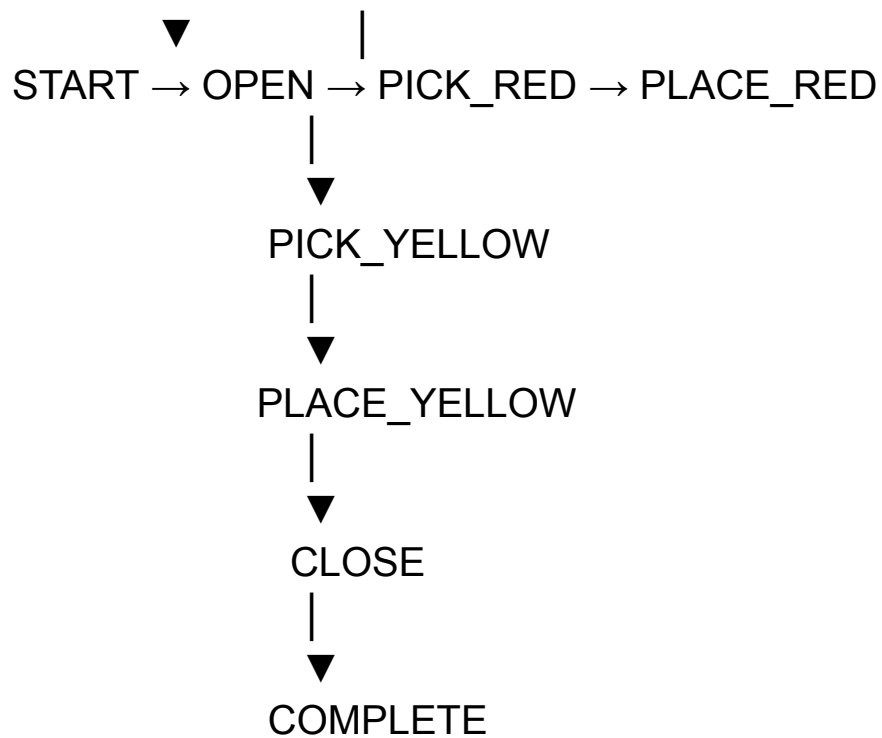
This is where I think we can make the solution substantially better than a basic HAR project.

Suppose the experiment is:

OPEN  
↓  
PICK\_RED  
↓  
PLACE\_RED  
↓  
PICK\_YELLOW  
↓  
PLACE\_YELLOW  
↓  
CLOSE

Represent it as a directed graph:





Now the AI doesn't have to magically learn “what is wrong.”

The neural model answers:

What action is happening?

The procedure engine answers:

Is that action valid at this point in the experiment?

That's a very clean separation.

11. This gives us deterministic error detection

Suppose current state is:

PLACE\_RED

Expected:

PICK\_YELLOW

But model predicts:

CLOSE

The procedure engine says:

INVALID TRANSITION

Then:

 "Please complete the yellow component placement before closing the container."

Or if the astronaut completely skips:

PICK\_YELLOW → CLOSE

we detect:

SKIPPED STEP: PLACE\_YELLOW

This is much more reliable than asking an LLM:

“Is the astronaut doing the experiment correctly?”

I would not make an LLM the core decision-maker.

12. We can model the procedure mathematically

This is where your ML/DSA background becomes useful.

Define:

$S_t = \text{current procedure state}$   $A_t = \text{recognized action}$

and a transition function:

$T(S_t, A_t) \rightarrow S_{t+1}$

For example:

$T(S_2, \text{PICK\_RED}) = S_3$

but:

$T(S_2, \text{CLOSE}) = \text{INVALID}$

We can also maintain confidence:

$P(A_t | V_{t-k:t})$

where  $(V_{t-k:t})$  is a temporal window of video.

Then:

Action:

PICK\_RED = 0.91

PICK\_YELLOW = 0.05

OPEN = 0.02

OTHER = 0.02

Procedure engine:

Expected = PICK\_RED

Confidence = 0.91

→ ACCEPT

This gives us an interpretable system.

### 13. Even better: temporal smoothing

A single frame shouldn't determine an action.

Instead:

Frame 100 → reach  
Frame 101 → reach  
Frame 102 → reach  
Frame 103 → grasp  
Frame 104 → grasp  
Frame 105 → move  
Frame 106 → move

...

The system recognizes:

REACH → GRASP → MOVE → RELEASE

We can use:

temporal CNN

LSTM/GRU

Transformer

TCN

ST-GCN

VideoMAE-derived features

depending on how far we want to push it.

VideoMAE is particularly interesting because it provides pretrained video representations and has demonstrated data-efficient video learning.

But for a hackathon, don't immediately jump to training a giant Video Transformer.

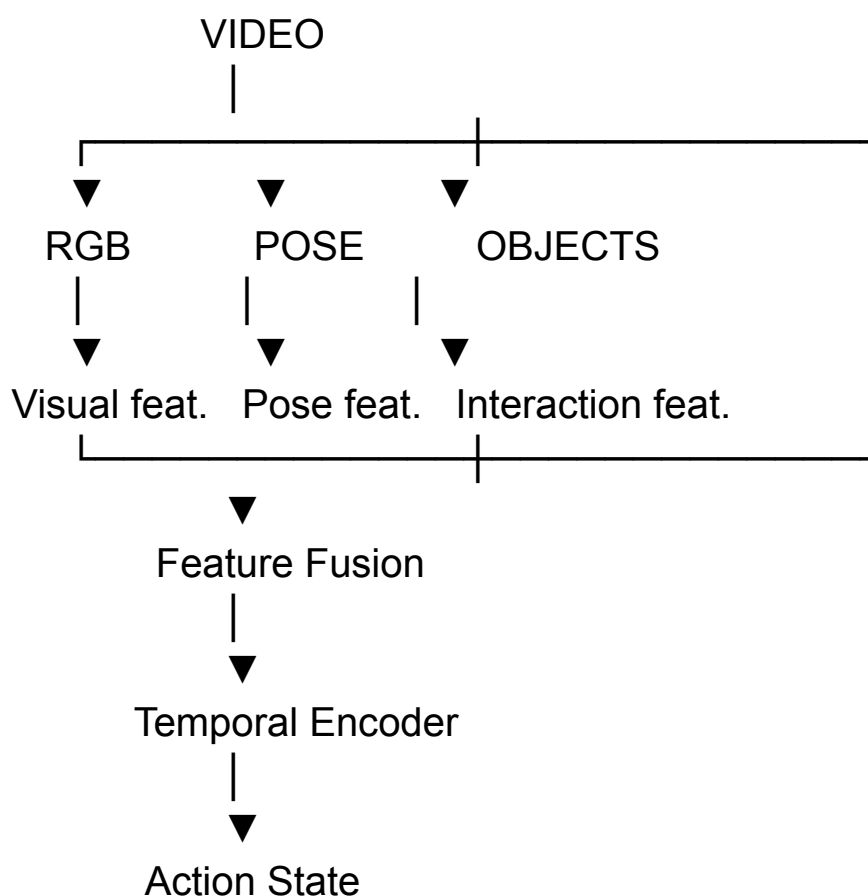
14. Our model could be multimodal

This is where the project could become seriously impressive.

Instead of:

RGB  $\rightarrow$  Action

we use:



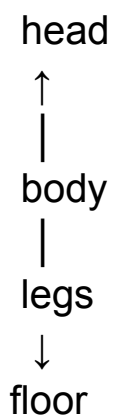
This is much closer to what the problem statement actually describes.

## 15. Now the crazy part: microgravity

The optional requirement is not just marketing.

Imagine:

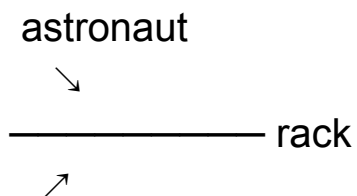
Earth



We naturally encode:

Y-axis = vertical

But in microgravity:



The astronaut might be:

upright

sideways

upside-down

floating  
rotated 90°

So an Earth-centric representation can become brittle.

ISRO explicitly suggests estimating the astronaut relative to the payload rack rather than the floor.

That gives us a beautiful coordinate-frame formulation:

$$P_{\text{body}}^{\text{rack}} = T_{\text{world}}^{\text{rack}^{-1}} P_{\text{body}}^{\text{world}}$$

Conceptually:

Don't ask “where is the astronaut relative to gravity?”

Ask:

“Where is the astronaut relative to the experiment?”

That's a far better invariant.

16. We don't necessarily need full HMR for MVP

This is important.

The statement calls HMR optional.

I would divide our implementation into:

Level 1 — Must have

Object detection

+

2D pose

+

Hand tracking  
+  
Temporal action recognition  
+  
Procedure state machine  
Level 2 — Strong  
3D pose  
+  
rack-relative coordinates  
+  
orientation normalization  
Level 3 — Research-grade  
3D Human Mesh Recovery  
+  
orientation-agnostic representation  
+  
microgravity augmentation

This gives us a working system even if Level 3 becomes too expensive.

17. There's another extremely useful dataset

MicroG-4M should become our external research/validation dataset.

It contains real microgravity footage from ISS/Tiangong/EVA contexts and specifically targets action understanding in microgravity.

But:

We should NOT use it as our primary experiment dataset.

Why?

Because the SIH statement specifically wants us to create a dataset around the predefined experiment.

Instead:

MicroG-4M



Pretraining / research / robustness experiments

Our custom dataset



Actual experiment recognition

That's much stronger.

18. Dataset strategy I'd recommend

We can create something like:

8 experiment actions

| ID | Action |
|----|--------|
|----|--------|

|    |      |
|----|------|
| A0 | Idle |
|----|------|

|    |                 |
|----|-----------------|
| A1 | Reach container |
|----|-----------------|

|    |                |
|----|----------------|
| A2 | Open container |
|----|----------------|

|    |                 |
|----|-----------------|
| A3 | Pick red object |
|----|-----------------|

|    |                  |
|----|------------------|
| A4 | Place red object |
|----|------------------|

|    |                    |
|----|--------------------|
| A5 | Pick yellow object |
|----|--------------------|

|    |                     |
|----|---------------------|
| A6 | Place yellow object |
|----|---------------------|

|    |                 |
|----|-----------------|
| A7 | Close container |
|----|-----------------|

Then record:

10–20 participants

Different:

heights  
clothing  
movement speed  
starting positions  
body orientations

And importantly:

different camera conditions  
Camera 1  
Camera 2  
Camera 3

with fixed camera positions.

Then generate:

Correct sequences  
Skipped sequences  
Out-of-order sequences  
Repeated sequences  
Wrong-object sequences  
Partial occlusion  
Fast execution  
Slow execution

This becomes an actual research dataset.

19. We can simulate microgravity

We can't reproduce true microgravity in a normal lab.

But we can introduce orientation/domain augmentation.

For example:

normal  
90° rotation  
180° rotation  
270° rotation  
horizontal body  
inverted body

And perhaps synthetic compositing.

This tests whether:

$$f(x) \approx f(Rx)$$

for rotations  $\{R\}$ .

In other words:

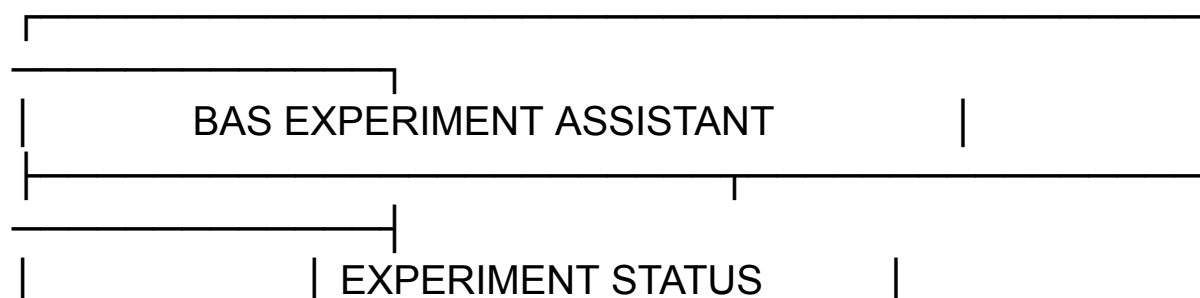
Does the representation recognize the same action even when the astronaut is rotated?

That would make a fantastic experiment in our presentation.

20. The system's output should look like this

Imagine the astronaut starts.

GUI



|                                                                                   |                      |
|-----------------------------------------------------------------------------------|----------------------|
| LIVE VIDEO                                                                        | Step: 3 / 7          |
|  | ✓ Open container     |
|                                                                                   | ✓ Pick red object    |
| [Payload Rack]                                                                    | ✓ Place red object   |
|                                                                                   | → Pick yellow object |
|                                                                                   | Confidence: 94.2%    |
| <hr/>                                                                             |                      |
| AI PERCEPTION                                                                     | SYSTEM               |
| Person: 0.97                                                                      | Next: PICK YELLOW    |
| Red object: 0.99                                                                  | Status: NORMAL       |
| Yellow object: 0.98                                                               |                      |
| Hand interaction: 0.93                                                            | Recording: ●         |
| <hr/>                                                                             |                      |

Then astronaut accidentally skips.

⚠ PROCEDURE VIOLATION

Expected:

PLACE\_YELLOW

Observed:

CLOSE\_CONTAINER

Voice:

“Warning. Yellow component placement has not been completed.”

And log:

```
{  
  "timestamp": "00:03:42",  
  "expected_step": "PLACE_YELLOW",  
  "observed_step": "CLOSE_CONTAINER",  
  "event": "OUT_OF_SEQUENCE",  
  "confidence": 0.91  
}
```

That is exactly the kind of demo a judge can understand in 10 seconds.

## 21. Edge deployment matters

The statement specifically says:

process locally at the edge

and requires an offline standalone trained model.

Therefore:

Don't build

Camera



Internet



Cloud API



AI

Build:

Camera



Local inference machine



AI



GUI

Potential deployment:

Python

PyTorch

OpenCV

ONNX Runtime / TensorRT

FastAPI

WebSocket

SQLite

GUI:

React



WebSocket



FastAPI

or, if we want a simpler hackathon deployment:

Python

PySide6

OpenCV

PyTorch

22. We should treat bandwidth as a design constraint

The point isn't just "offline because SIH says so."

There's a legitimate systems argument:

Raw video:

1920 × 1080 × 30 FPS

≈ enormous bandwidth/storage

Instead:

Local AI

↓

structured events

↓

few KB/minute

For example:

```
{
  "experiment": "EXP001",
  "steps": [
    {
      "id": "S1",
      "action": "OPEN_CONTAINER",
      "timestamp": 171.42,
      "status": "SUCCESS"
    },
    {
      "id": "S2",
      "action": "PICK_RED",
      "timestamp": 184.91,
      "status": "SUCCESS"
    }
  ]
}
```

Mission control doesn't necessarily need the entire raw video stream to understand experiment status.

That's a strong systems-level justification.

## 23. Voice system

This part is actually easy compared to the vision.

We need:

Procedure Engine



Event



Message Generator



Offline TTS



Speaker

Examples:

"Next step: pick the yellow component."

"Warning. Step three has been skipped."

"Warning. The observed action is out of sequence."

"Experiment completed successfully."

Because the system must be standalone, I'd keep TTS local/offline, not an online API.

## 24. One thing I DON'T recommend

Don't build:

“ChatGPT for astronauts.”

That's not the problem.

And don't make:

Video → LLM → everything

The judge can immediately ask:

Why do you need an LLM?

And the answer becomes weak.

Instead:

Deterministic AI pipeline

Computer Vision

+

Temporal ML

+

Procedure Graph

+

Deterministic Validation

+

Offline Voice

Then an LLM, if we use one at all, can sit outside the safety-critical loop for things like:

natural-language experiment summary

report generation

operator queries

25. Where the actual innovation can be

The basic version:

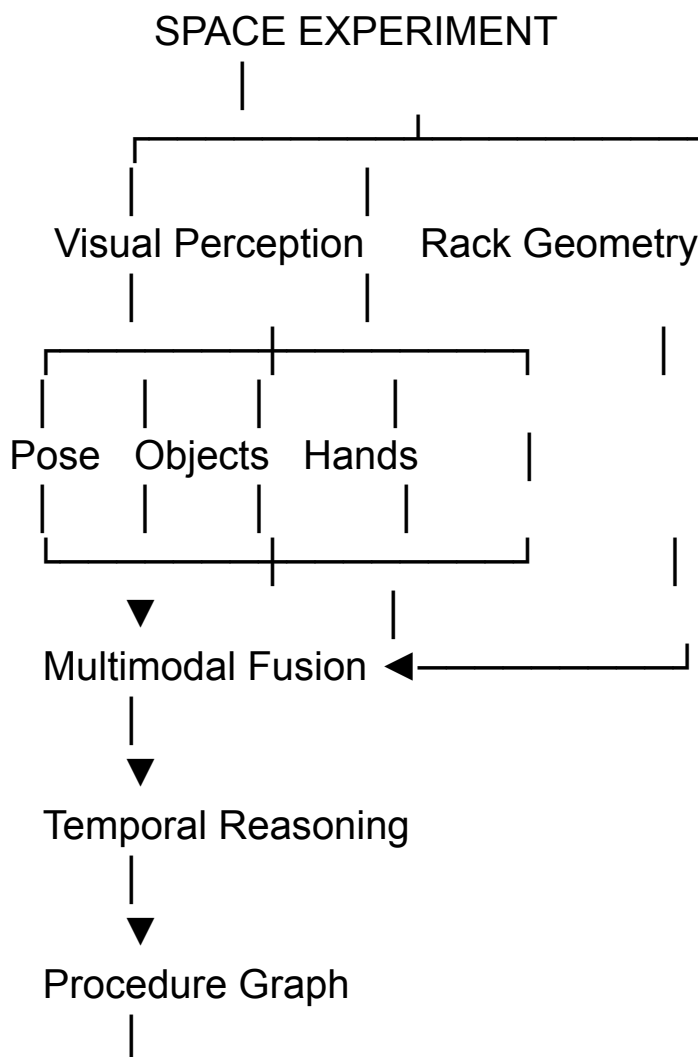
YOLO + pose + LSTM

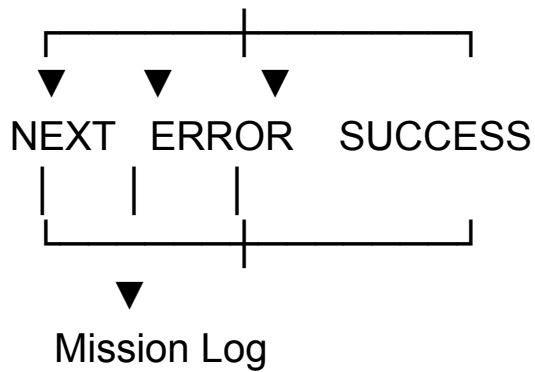
is not particularly novel.

Our stronger concept should be:

Space-Aware Multimodal Procedural Intelligence

Something like:





That is a much better story.

## 26. Our research questions

Before writing code, I would make the team answer these six questions experimentally:

RQ1

Can a model recognize our experiment steps reliably from fixed-camera video?

RQ2

Does adding pose improve RGB-only action recognition?

RQ3

Does adding hand-object interaction improve recognition of visually similar steps?

RQ4

Can a deterministic procedure graph reliably detect skipped/out-of-order actions?

RQ5

How robust is the system under astronaut orientation changes?

RQ6

Can the entire pipeline run in real time on edge hardware?

These become our evaluation framework.

## 27. Metrics

Don't just report:

“Our model gives 95% accuracy.”

That's weak.

We should report:

Perception

Precision

Recall

mAP

Action recognition

Accuracy

Macro F1

confusion matrix

Temporal detection

action onset delay

action completion detection accuracy

Procedure validation

skipped-step detection rate

out-of-order detection rate

false alarm rate

System

FPS  
inference latency  
RAM  
GPU/CPU utilization  
model size  
Edge  
FPS  $\geq$  target  
Latency  $\leq$  target  
Offline = 100%

This makes the project look like an engineering system rather than a college ML demo.

## 28. One particularly strong experiment

We should create three versions:

Model A  
RGB only  
Model B  
RGB + Pose  
Model C  
RGB + Pose + Hand/Object Interaction

Then compare:

| Model            | Step Accuracy |   |   | F1 | Skip Detection | Out-of-order |
|------------------|---------------|---|---|----|----------------|--------------|
| RGB ?            | ?             | ? | ? |    |                |              |
| RGB + Pose       | ?             | ? | ? | ?  | ?              |              |
| RGB + Pose + HOI | ?             | ? | ? | ?  | ?              | ?            |

This demonstrates why our architecture exists.

## 29. And then the killer experiment

Take the exact same experiment.

Run:

Orientation 0°

Orientation 90°

Orientation 180°

Orientation 270°

Compare:

Earth-normal model

vs

orientation-aware model

If we can demonstrate:

Normal:

94%

Rotated:

61%

Our orientation-aware:

88%

even on our synthetic dataset, that becomes a very compelling technical story.

We don't need to claim:

“This solves astronaut activity recognition in actual BAS.”

Instead:

“We demonstrate improved orientation robustness under simulated microgravity viewpoints and identify the remaining domain gap to real flight data.”

That is scientifically honest and much harder for a judge to attack.

### 30. Our proposed tech stack

I'd currently shortlist:

Vision

PyTorch

OpenCV

YOLO

MMPose / MediaPipe

Temporal recognition

ST-GCN

PoseC3D

TCN

small Transformer

Optional advanced

VideoMAE

3D HMR

SMPL

Backend

FastAPI

WebSocket

SQLite

Frontend

React

TypeScript

Deployment

ONNX

TensorRT

CUDA

Voice

Offline TTS

31. The most important strategic conclusion

After researching this, I would keep this problem.

It's a good SIH choice because the requirements align unusually well:

Clear input

Fixed camera video

Clear AI task

Human activity + object + pose + interaction

Clear reasoning task

Validate predefined sequence

Clear failure modes

Skip / out-of-order

Clear user interaction

Voice guidance

Clear output

Timestamped experiment log

Clear systems requirement

Offline edge inference

Optional research depth

Orientation-agnostic 3D HMR

And BAS itself is an active part of India's human-spaceflight roadmap, not merely a hypothetical setting.

# Problem Statement

## AI-Based Human Activity and Experiment Procedure Recognition System for On-Board BAS Experiments

### 1. Background

Future Indian human-spaceflight missions and the **Bharatiya Antariksh Station (BAS)** are expected to support scientific experiments in a microgravity environment. During such missions, communication between astronauts and ground control may experience significant latency and limited bandwidth, making continuous real-time supervision from Earth impractical.

Scientific experiments conducted onboard therefore require a degree of **autonomous monitoring and procedural assistance**.

A fixed-camera-based AI system can locally observe an astronaut performing an experiment, identify relevant human activities and interactions with experimental objects, determine the current stage of the experiment, and verify whether the prescribed sequence is being followed.

---

### 2. Problem

The proposed system addresses the problem of **automatically monitoring and validating astronaut execution of predefined scientific experiment procedures using onboard camera feeds**.

Given video captured by one or more fixed cameras observing an experimental payload, the system shall:

- identify the astronaut and relevant experimental objects;
- recognize predefined human activities and human–object interactions;

- determine the current experiment step from temporal video observations;
- compare the observed activity sequence against the prescribed experiment procedure;
- detect **skipped, repeated, incorrect, or out-of-order steps**;
- provide the astronaut with appropriate next-step guidance and warnings;
- maintain a lightweight, timestamped record of experiment execution; and
- operate locally without requiring continuous communication with ground control.

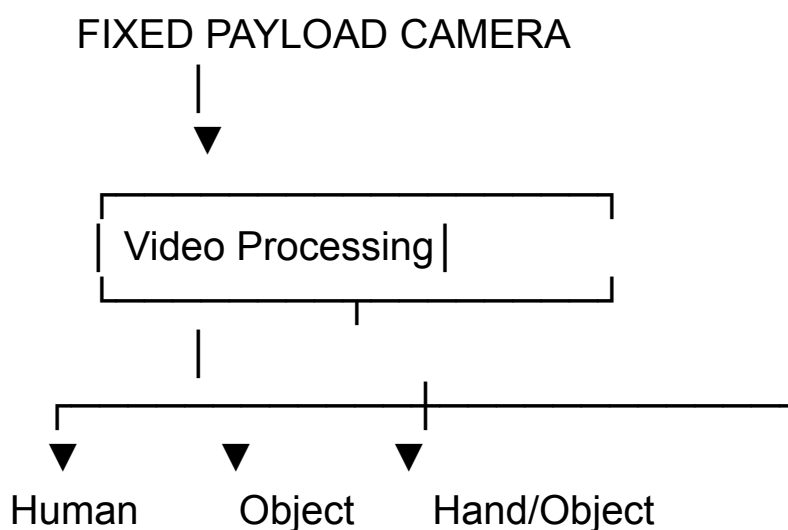
The system should additionally be designed to handle the distinctive visual and spatial characteristics of **microgravity**, where conventional gravity-dependent assumptions about human orientation and posture may not hold.

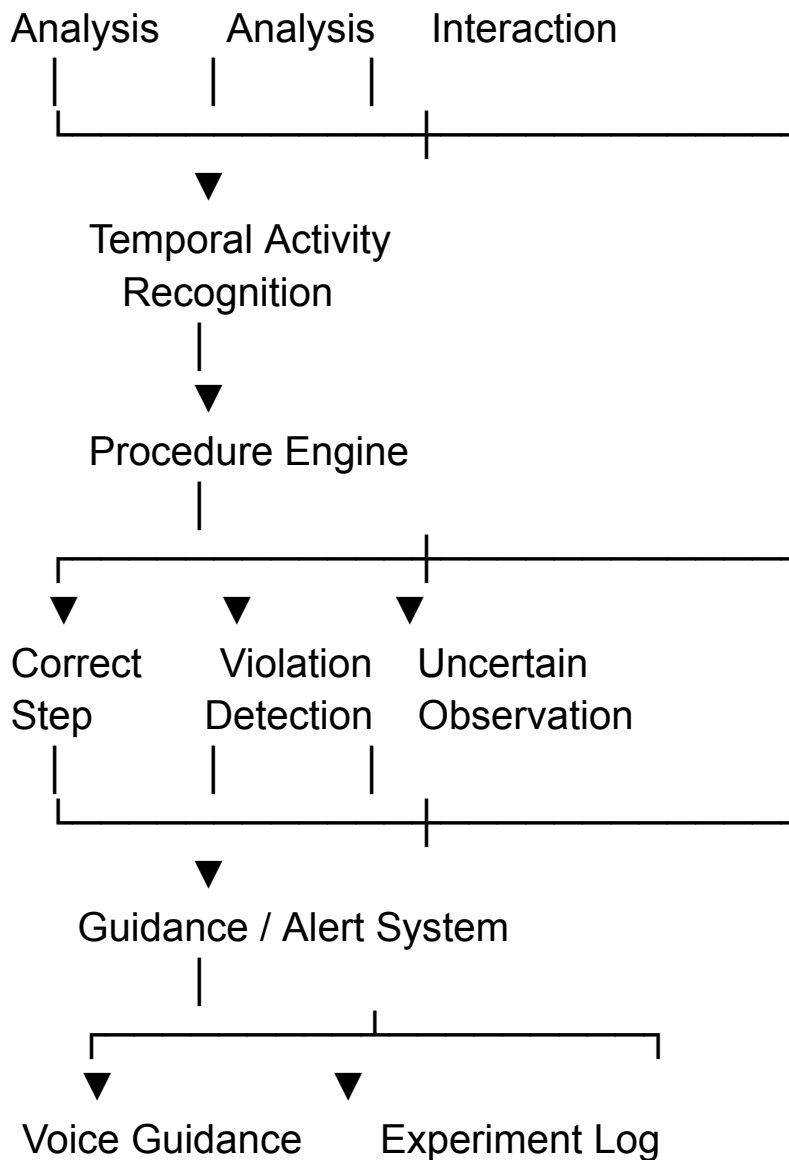
---

### 3. Proposed System

We propose an **AI-driven onboard Experiment Activity Recognition and Procedure Validation System**.

At a high level:





The system will separate **perception** from **procedure reasoning**. AI models will determine *what is happening*, while a procedure model/state machine will determine *whether it should be happening at that point in the experiment*.

---

## 4. Core Objectives

The system shall achieve the following objectives:

### O1 — Human Activity Recognition

Recognize predefined astronaut activities relevant to the experiment from video.

## **O2 — Object Recognition**

Detect and track experimental objects, tools, containers, and relevant target regions.

## **O3 — Human–Object Interaction Recognition**

Determine interactions such as:

- reaching;
- grasping;
- picking;
- moving;
- placing;
- releasing;
- manipulating experimental equipment.

## **O4 — Temporal Understanding**

Use a sequence of observations rather than individual frames to determine the current experiment action.

## **O5 — Procedure Validation**

Represent the experiment as an ordered sequence of states/actions and validate observed execution against it.

## **O6 — Error Detection**

Detect events including:

Correct execution

- |
- | — Skipped step
- | — Out-of-order step
- | — Repeated step
- | — Incorrect action

- └─ Incorrect object
- └─ Uncertain/ambiguous action

## **O7 — Astronaut Assistance**

Provide contextual next-step instructions and warnings through an onboard interface and voice output.

## **O8 — Experiment Logging**

Generate a lightweight timestamped record of experiment execution, including detected actions, procedure state, confidence and violations.

## **O9 — Edge/Offline Operation**

Perform the core AI inference and procedure validation locally without depending on continuous Internet or ground communication.

## **O10 — Microgravity Robustness**

Investigate and support orientation-independent activity recognition, preferably using the experimental payload/rack as the spatial reference frame rather than assuming a fixed gravitational "up."

---

# **5. System Scope**

## **In Scope**

The system will cover:

- fixed-camera video acquisition;
- video preprocessing;
- astronaut detection/tracking;
- relevant object detection;
- human pose estimation;

- hand/object interaction analysis;
- temporal activity recognition;
- experiment-state estimation;
- procedure validation;
- anomaly/violation detection;
- next-step recommendation;
- voice alerts;
- experiment logging;
- local video storage;
- local monitoring dashboard;
- optional video streaming over a local/network interface;
- offline inference;
- performance monitoring.

## Out of Scope

The system will **not** attempt to:

- autonomously control spacecraft systems;
- physically manipulate experimental equipment;
- replace astronaut decision-making;
- replace ground mission control;
- perform unrestricted/general-purpose understanding of every possible astronaut activity;
- make safety-critical spacecraft decisions without human authorization.

This distinction is important: **our system is an experiment-assistance and monitoring system, not an autonomous spacecraft controller.**

---

## 6. Operating Assumptions

The initial system will assume:

1. The experiment procedure is predefined.
2. The relevant actions and objects are known.
3. Cameras are installed at fixed locations.
4. The camera view sufficiently covers the experimental workspace.
5. A representative dataset can be generated for the target experiment.
6. The system has access to local compute hardware capable of running the trained models.
7. The experiment can be represented as a finite sequence/state graph.

For example:

START



OPEN\_CONTAINER



PICK\_RED\_OBJECT



PLACE\_RED\_OBJECT



PICK\_YELLOW\_OBJECT



PLACE\_YELLOW\_OBJECT



CLOSE\_CONTAINER



COMPLETE

---

## 7. Key System Principle

The fundamental design principle is:

**The AI should not merely recognize what the astronaut is doing; it should understand where that action occurs within the prescribed experiment procedure.**

For example:

AI perception:

"Close container"      confidence = 0.94

is insufficient by itself.

The procedure engine additionally evaluates:

Current experiment state:

PLACE\_YELLOW

Expected:

PLACE\_YELLOW

Observed:

CLOSE\_CONTAINER

Result:

OUT-OF-SEQUENCE

Therefore:

System Decision=Activity Recognition+Temporal Context+Procedure State  
$$\text{System Decision} = \text{Activity Recognition} + \text{Temporal Context} + \text{Procedure State}$$

This is the core differentiator of our system.

---

## 8. Functional Requirements — High Level

The system shall provide the following major functions:

| <b>ID</b> | <b>Function</b>                    |
|-----------|------------------------------------|
| FR-01     | Acquire video from fixed cameras   |
| FR-02     | Process video locally              |
| FR-03     | Detect and track astronaut         |
| FR-04     | Detect relevant experiment objects |
| FR-05     | Estimate human pose                |
| FR-06     | Analyze hand-object interactions   |
| FR-07     | Recognize predefined activities    |
| FR-08     | Maintain temporal activity context |
| FR-09     | Maintain current experiment state  |
| FR-10     | Validate action sequence           |
| FR-11     | Detect skipped steps               |

- FR- Detect out-of-order steps  
12
- FR- Detect repeated/incorrect actions  
13
- FR- Determine next expected action  
14
- FR- Generate astronaut alerts  
15
- FR- Provide voice guidance  
16
- FR- Generate timestamped experiment  
17 logs
- FR- Store video locally  
18
- FR- Provide monitoring GUI  
19
- FR- Operate without continuous  
20 Internet connectivity

We'll expand these considerably in the SRS.

---

## 9. Non-Functional Requirements

### Performance

The system should support near-real-time inference with an explicitly measured:

- FPS;

- end-to-end latency;
- action recognition latency;
- alert latency.

### **Reliability**

The system should avoid generating alerts from a single uncertain observation.

For example:

Frame 1 → PICK\_RED 0.51

Frame 2 → PICK\_RED 0.53

Frame 3 → PICK\_RED 0.48

should **not immediately trigger a procedure transition**.

Temporal confidence and state confirmation should be used.

### **Offline Capability**

Core functionality must remain operational without Internet connectivity.

### **Resource Efficiency**

The system should be optimized for constrained onboard/edge computing environments.

### **Explainability**

Every procedure decision should ideally be traceable to:

Observed action

+

Confidence

+

Current state

+

Expected action

+

Decision

For example:

Current State : STEP\_04

Expected : PICK\_YELLOW

Observed : CLOSE\_CONTAINER

Confidence : 91.4%

Decision : OUT\_OF\_SEQUENCE

### **Extensibility**

The procedure engine should allow a new experiment to be introduced through a new experiment definition rather than requiring the entire AI pipeline to be redesigned.

---

## **10. Experiment-Agnostic Architecture**

This is something I'd explicitly put into our SRS.

We shouldn't hard-code:

RED → YELLOW → CLOSE

into the AI.

Instead, define an experiment configuration:

experiment:

id: EXP\_001

name: Sample BAS Experiment

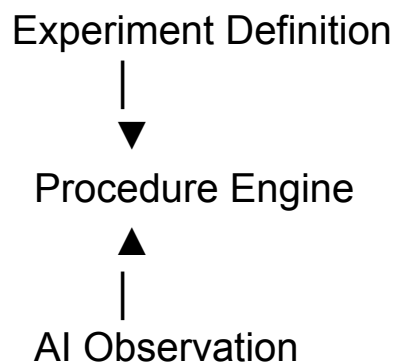
steps:

- id: S01

action: OPEN\_CONTAINER

- id: S02  
action: PICK\_OBJECT  
object: RED
- id: S03  
action: PLACE\_OBJECT  
object: RED  
target: TARGET\_A
- id: S04  
action: PICK\_OBJECT  
object: YELLOW
- id: S05  
action: PLACE\_OBJECT  
object: YELLOW  
target: TARGET\_B
- id: S06  
action: CLOSE\_CONTAINER

Then:



This transforms the project from a **single SIH demo** into a potentially reusable **experiment monitoring platform**.

---

# 11. Microgravity Extension

The system should optionally support a rack-relative spatial representation.

Instead of assuming:

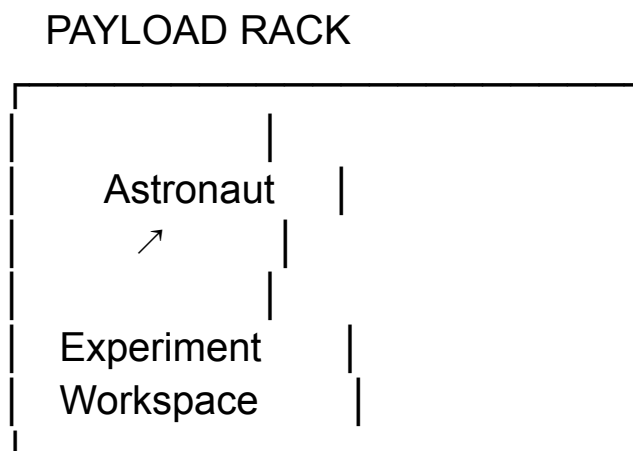
$Y = \text{gravity-up}$

we define:

$\text{Coordinate Frame} = \text{Payload Rack}$   
 $\text{Coordinate} \setminus \text{Frame} = \text{Payload} \setminus \text{Rack}$

Then human pose and object locations can be interpreted relative to the experiment.

Conceptually:



This makes the system more robust to:

- body rotation;
- inverted posture;
- floating;
- different astronaut orientations.

This should be treated as an **advanced capability**, not a dependency for the MVP.

---

## 12. Success Criteria

The system will be considered successful when it can demonstrate that it:

### **Recognition**

Reliably recognizes the predefined experiment actions from camera footage.

### **Procedure Understanding**

Correctly determines the current stage of the experiment.

### **Violation Detection**

Detects deliberately introduced skipped and out-of-order steps.

### **Assistance**

Provides the correct next-step guidance and appropriate warnings.

### **Logging**

Produces an accurate timestamped experiment execution log.

### **Edge Operation**

Performs the above pipeline locally without requiring cloud inference.

### **Robustness**

Maintains acceptable recognition performance under variations in astronaut appearance, execution speed, camera conditions and orientation.

---

## 13. One-line Problem Statement

For the **SRS cover/abstract**, I'd condense the entire thing to:

**To develop an offline, edge-deployable AI system that uses fixed-camera video to recognize astronaut activities and human–object interactions during predefined BAS scientific experiments, understand their temporal sequence, validate execution against the prescribed procedure, detect procedural deviations, and provide real-time guidance, alerts, and lightweight experiment logs.**

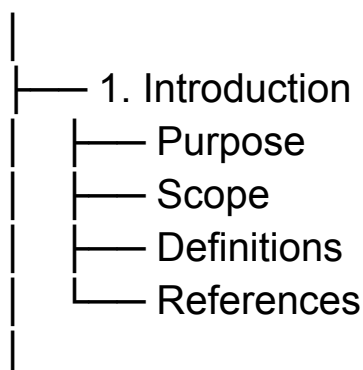
That's the version I'd use as our **official internal problem statement**.

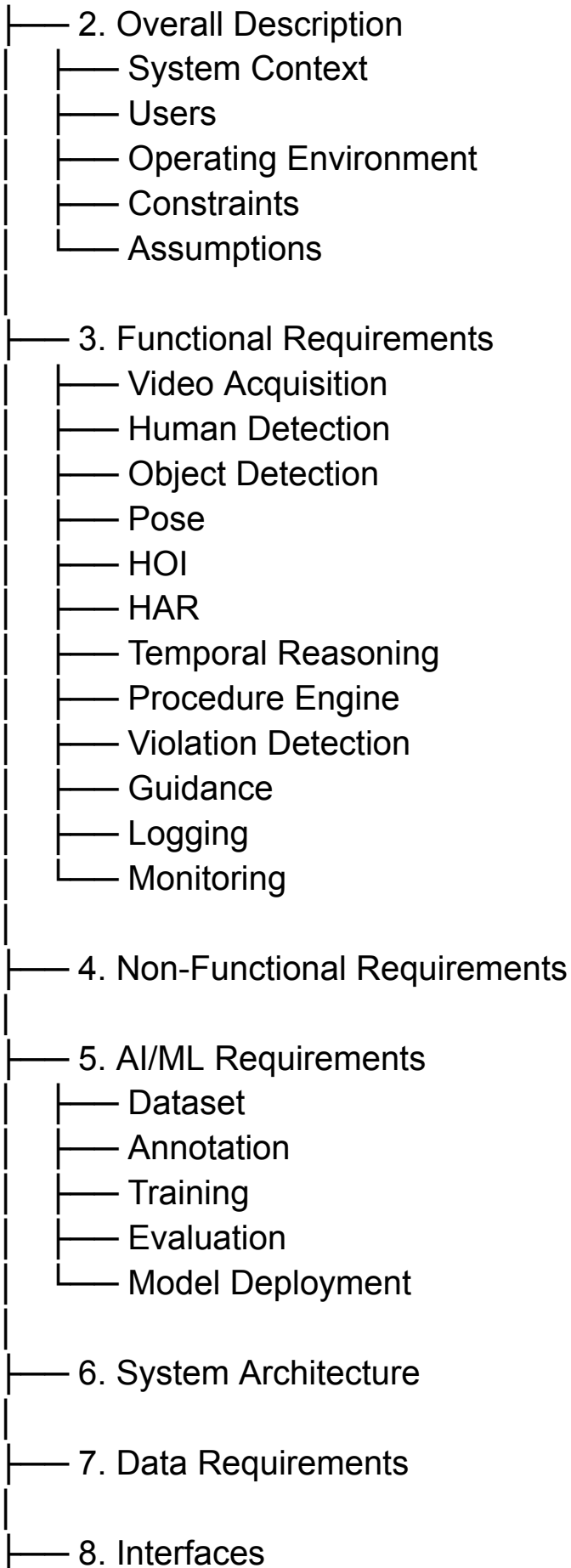
---

### And now we can move to the SRS

I recommend we **freeze this problem statement first**, then structure the SRS as:

SRS





- 9. Security & Reliability
- 10. Performance Requirements
- 11. Acceptance Criteria

# ASTRA-E

## Autonomous Space Task Recognition & Assistance for Experiments

Software Requirements Specification — v1.0

**Project:** SIH 2026 — Problem Statement 26174

**System:** ASTRA-E

**Domain:** Space Technology / Computer Vision / AI / Edge Computing

**Primary Function:** On-board astronaut activity and experiment-procedure monitoring

---

## 1. Introduction

### 1.1 Purpose

ASTRA-E is an AI-powered onboard monitoring and assistance system designed to recognize, interpret, and validate astronaut activities during predefined scientific experiments.

The system will process video acquired from fixed cameras positioned around an experimental payload. It will identify relevant humans, objects, poses, and interactions, infer the current activity, and compare the observed activity sequence against the predefined experimental procedure.

When a deviation occurs, ASTRA-E will notify the astronaut through appropriate visual and/or voice feedback and record the event in an experiment log.

---

## 1.2 Problem Definition

During future human-spaceflight missions and BAS scientific operations, continuous supervision from ground control may be limited by communication latency, bandwidth constraints, and operational requirements.

A scientific experiment may contain a sequence such as:

START



Open container



Pick red object



Place red object



Pick yellow object



Place yellow object



Close container



COMPLETE

ASTRA-E shall determine whether the astronaut's observed actions conform to this prescribed procedure.

The system must distinguish between:

Correct execution

Skipped step

Out-of-order step

Repeated step

Incorrect action

Incorrect object interaction

Uncertain observation

---

## 2. Scope

### 2.1 In Scope

ASTRA-E shall provide:

- Fixed-camera video acquisition
- Video preprocessing
- Astronaut detection/tracking
- Relevant object detection/tracking
- Human pose estimation
- Hand tracking
- Human-object interaction analysis
- Human activity recognition
- Temporal activity understanding
- Experiment state tracking
- Procedure validation
- Procedural deviation detection
- Next-step identification
- Visual alerts
- Voice alerts/guidance
- Timestamped experiment logging
- Local video storage
- Monitoring interface
- Local/network video streaming
- Offline AI inference
- Performance monitoring

---

### 2.2 Out of Scope

ASTRA-E shall **not**:

- directly control spacecraft systems;
- physically manipulate experiment hardware;
- autonomously modify an experiment procedure;
- replace astronaut judgment;
- replace mission control;
- make safety-critical spacecraft decisions;
- require unrestricted understanding of arbitrary human activities;
- depend on an Internet/cloud service for core functionality.

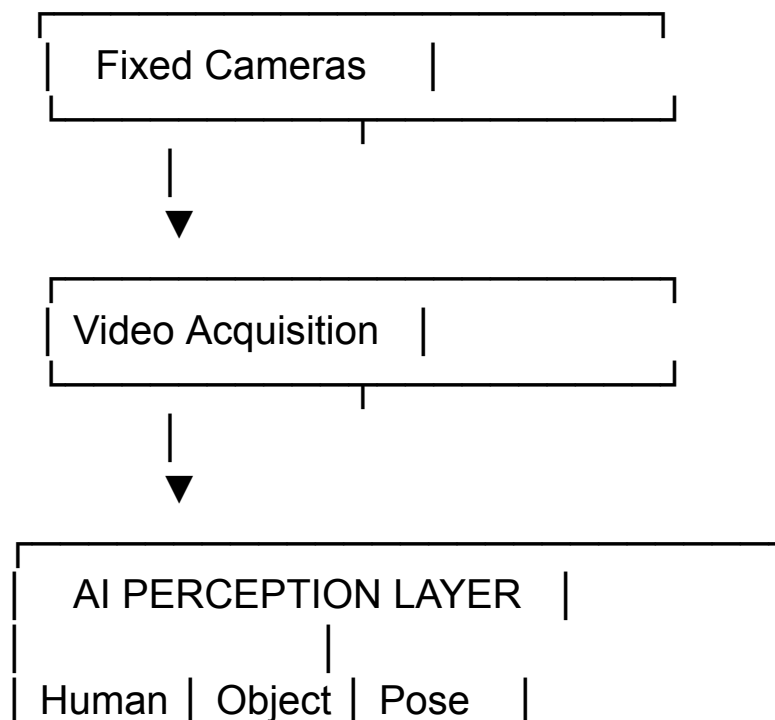
This distinction is important:

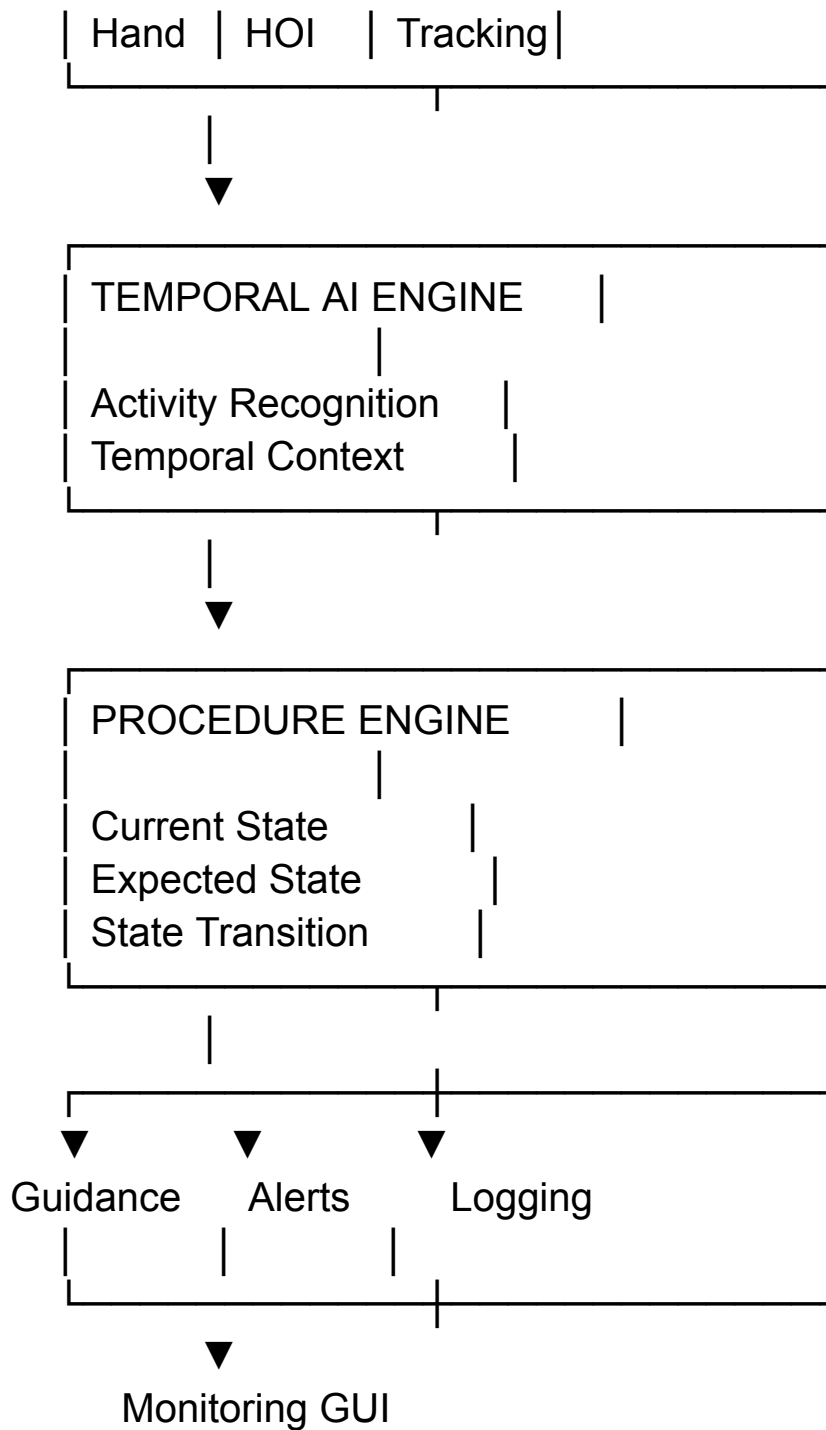
**ASTRA-E is an experiment-monitoring and assistance system, not an autonomous spacecraft-control system.**

---

### 3. System Context

The overall system can be represented as:





---

## 4. Users and Actors

ASTRA-E has four primary actors.

### 4.1 Astronaut

The primary operational user.

The astronaut receives:

- current experiment status;
- next-step guidance;
- warnings;
- procedure completion notifications;
- system-status information.

#### **4.2 Experiment Operator**

Responsible for:

- configuring experiments;
- loading experiment procedures;
- reviewing experiment logs;
- reviewing detected deviations.

#### **4.3 Ground/Mission Control**

May receive:

- experiment status;
- lightweight telemetry;
- event logs;
- selected video streams.

However, **core system functionality shall remain operational without ground communication.**

#### **4.4 System Administrator**

Responsible for:

- system configuration;
- camera configuration;
- model deployment;
- storage management;
- diagnostics.

---

## 5. System Assumptions

ASTRA-E shall initially assume:

1. The experiment procedure is known beforehand.
  2. The experiment consists of a finite set of predefined steps.
  3. Relevant objects are known.
  4. Cameras are fixed relative to the experimental payload.
  5. The experimental workspace is visible to at least one camera.
  6. Appropriate training data can be generated for the target experiment.
  7. Local compute resources are available.
  8. The AI model has been trained before deployment.
  9. The procedure definition can be represented digitally.
- 

## 6. Functional Requirements

This is the heart of our SRS.

---

### FR-001 — Video Acquisition

ASTRA-E shall acquire video from one or more fixed cameras.

The system shall support configurable:

- resolution;
- frame rate;
- camera identifier;
- camera orientation;
- video format.

---

## **FR-002 — Video Preprocessing**

The system shall preprocess incoming video for AI inference.

Processing may include:

- resizing;
- normalization;
- frame sampling;
- noise reduction;
- region-of-interest extraction.

---

## **FR-003 — Human Detection**

The system shall detect the astronaut within the camera frame.

The system shall maintain a unique tracking identity for the astronaut during the experiment where possible.

---

## **FR-004 — Object Detection**

The system shall detect relevant experiment objects.

Each relevant object shall have:

Object ID

Object Type

Position

Bounding Region

Confidence

Tracking State

Example:

```
{  
  "object": "RED_COMPONENT",  
  "confidence": 0.97,  
  "position": [421, 238]  
}
```

---

## 7. Human Pose and Interaction

### FR-005 — Pose Estimation

ASTRA-E shall estimate relevant human body landmarks.

The system should support:

- 2D pose;
  - temporal pose tracking;
  - optional 3D pose.
- 

### FR-006 — Hand Tracking

The system shall track the astronaut's hands sufficiently to identify relevant interactions with experiment objects.

---

### FR-007 — Human-Object Interaction

The system shall determine interactions between the astronaut and relevant experiment objects.

Examples:

APPROACH

TOUCH

GRASP

PICK  
MOVE  
PLACE  
RELEASE  
MANIPULATE

---

## 8. Activity Recognition

### FR-008 — Activity Classification

The system shall classify predefined experiment activities.

For example:

OPEN\_CONTAINER  
PICK\_RED  
PLACE\_RED  
PICK\_YELLOW  
PLACE\_YELLOW  
CLOSE\_CONTAINER

Each recognition result shall contain:

Activity  
Confidence  
Timestamp  
Duration

---

### FR-009 — Temporal Activity Recognition

The system shall use a temporal sequence of observations when determining an activity rather than relying solely on an individual video frame.

This requirement exists because:

Frame → ambiguous  
Sequence → meaningful action

For example:

Reach  
↓  
Hand approaches object  
↓  
Grasp  
↓  
Object moves with hand  
↓  
Object enters target region  
↓  
Release

should be interpreted as a complete **Pick** → **Move** → **Place** interaction.

---

## 9. Experiment Procedure Engine

This is the **core reasoning component** of ASTRA-E.

### FR-010 — Experiment Definition

The system shall support loading a predefined experiment procedure.

An experiment definition shall contain:

Experiment ID  
Experiment Name  
Version  
Steps

Expected Actions  
Relevant Objects  
Allowed Transitions  
Completion Conditions

---

## **FR-011 — Procedure State Tracking**

The system shall maintain the current state of the experiment.

Example:

Current:  
STEP\_04 — PLACE\_RED

Expected:  
STEP\_05 — PICK\_YELLOW

---

## **FR-012 — State Transition Validation**

Every recognized action shall be evaluated against the current procedure state.

Conceptually:

$$S_{t+1} = T(S_t, A_t) \quad S_{\{t+1\}} = T(S_t, A_t)$$

where:

- $S_t$  = current procedure state
  - $A_t$  = recognized activity
  - $T$  = permitted transition function
-

# 10. Procedure Deviation Detection

## FR-013 — Skipped Step Detection

The system shall detect when one or more required steps have not been completed before a subsequent step is attempted.

Example:

Expected:

```
PICK_YELLOW
  ↓
PLACE_YELLOW
  ↓
CLOSE
```

Observed:

```
PICK_YELLOW
  ↓
CLOSE
```

Result:

VIOLATION  
Skipped: PLACE\_YELLOW

---

## FR-014 — Out-of-Order Detection

The system shall detect when a valid experiment action occurs at an invalid point in the procedure.

Example:

Current:

PLACE\_RED

Observed:

CLOSE\_CONTAINER

Result:

OUT\_OF\_SEQUENCE

---

### **FR-015 — Repeated-Step Detection**

The system shall detect inappropriate repetition of a completed step where repetition is not permitted by the procedure.

---

### **FR-016 — Incorrect Object Detection**

The system shall detect when an action is performed on an incorrect object.

Example:

Expected:

PICK\_RED

Observed:

PICK\_YELLOW

---

### **FR-017 — Uncertain Observation**

The system shall be capable of representing uncertainty.

If:

$P(A|V) < \tau P(A|V) < \tau$

the system shall avoid making an irreversible procedure transition and may classify the observation as:

UNCERTAIN

This is critical.

**We don't want one bad frame causing an astronaut warning.**

---

## 11. Guidance and Alerting

### FR-018 — Next-Step Identification

The system shall determine the next expected experiment action.

Example:

Completed:

PLACE\_RED

Next:

PICK\_YELLOW

---

### FR-019 — Visual Guidance

The monitoring interface shall display:

- current step;
  - completed steps;
  - next step;
  - procedure progress;
  - system status.
-

## FR-020 — Voice Guidance

The system shall provide optional voice instructions.

Example:

"Next step: pick the yellow component."

---

## FR-021 — Violation Alerts

The system shall provide an alert when a sufficiently confident procedural violation is detected.

Examples:

"Warning. The current action is out of sequence."

"Warning. The yellow component placement has not been completed."

---

## FR-022 — Alert Suppression

The system shall avoid repeatedly issuing the same alert for a persistent single event.

This prevents:

WARNING!

WARNING!

WARNING!

WARNING!

every frame. 😅

Instead, the system should maintain an event state:

VIOLATION\_DETECTED

↓  
ALERT\_ISSUED  
↓  
WAIT\_FOR\_RESOLUTION

---

## 12. Experiment Logging

### FR-023 — Event Logging

The system shall record significant experiment events.

Each event shall contain at minimum:

Timestamp  
Experiment ID  
Step ID  
Expected Action  
Observed Action  
Confidence  
Event Type  
System Status

Example:

```
{  
  "timestamp": "00:03:42.521",  
  "step": "S05",  
  "expected": "PLACE_YELLOW",  
  "observed": "CLOSE_CONTAINER",  
  "confidence": 0.914,  
  "event": "OUT_OF_SEQUENCE"  
}
```

---

### FR-024 — Experiment Summary

At experiment completion, ASTRA-E shall generate a summary containing:

Experiment status  
Total duration  
Completed steps  
Skipped steps  
Out-of-order events  
Uncertain events  
Warnings  
Completion timestamp

---

## 13. Video Management

### FR-025 — Local Recording

The system shall support local recording of camera footage.

### FR-026 — Event Association

Recorded footage should be associated with relevant timestamps/events where feasible.

For example:

00:03:42

OUT\_OF\_SEQUENCE



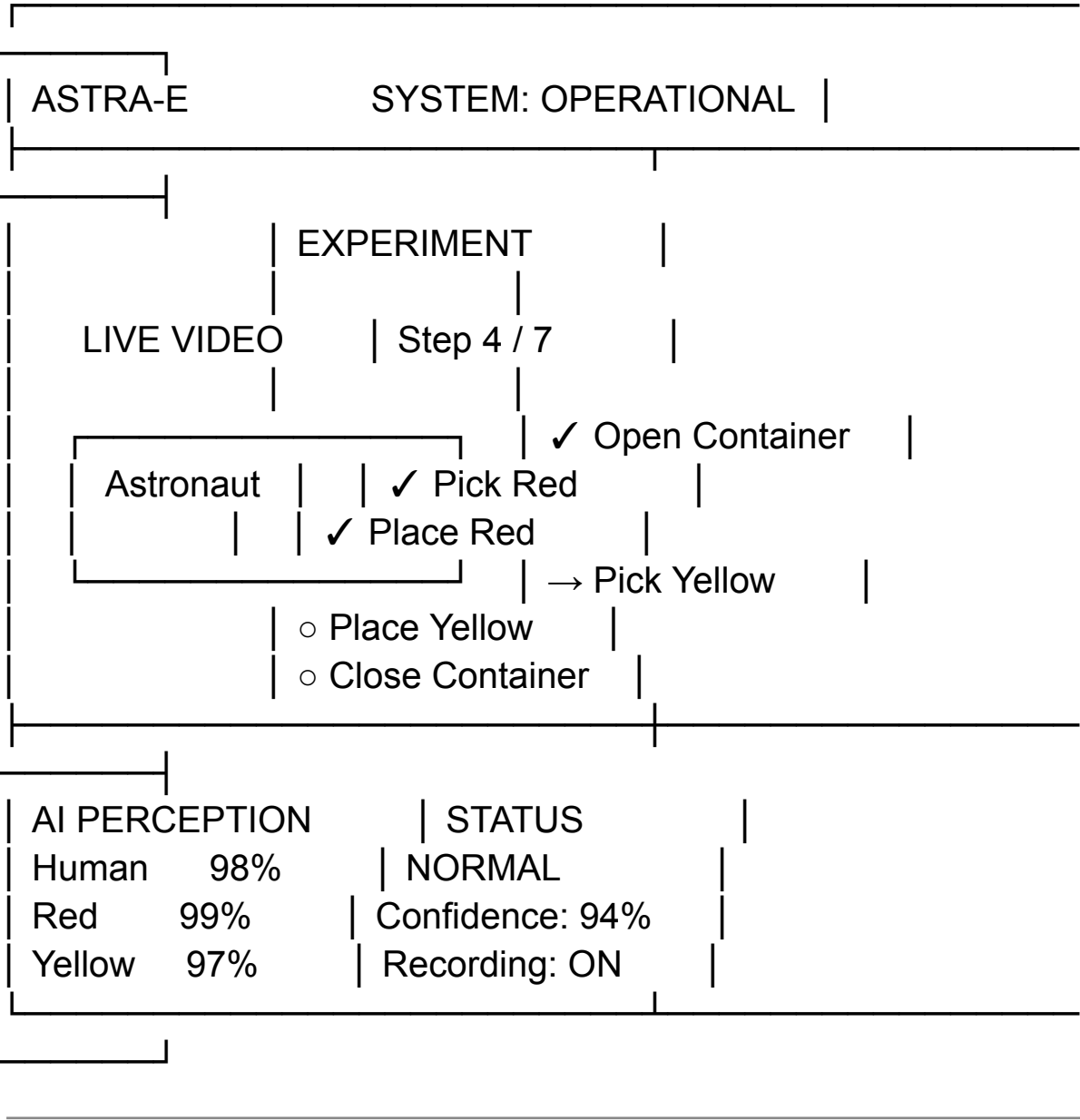
Video segment 00:03:35–00:03:50

This would be **extremely useful during post-experiment analysis.**

---

# 14. Monitoring Interface

The GUI shall provide at minimum:



# 15. Offline Operation

## FR-027 — Local Inference

Core perception, activity recognition, procedure validation and alerting shall execute locally.

The system shall not require Internet access for normal operation.

---

## FR-028 — Network Independence

Temporary loss of communication with ground control shall not prevent:

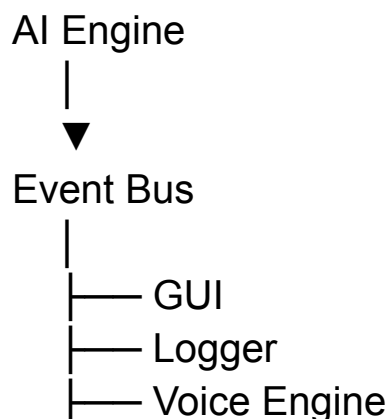
- AI inference;
  - procedure tracking;
  - violation detection;
  - local alerts;
  - local logging.
- 

## 16. Communication

ASTRA-E may expose a local/network API for:

- experiment status;
- logs;
- system health;
- video stream;
- AI events.

A possible architecture is:



## 17. Non-Functional Requirements

Now we define **how well** the system must perform.

### **NFR-001 — Real-Time Performance**

The system shall support near-real-time processing.

Our final SRS should eventually specify a measurable target such as:

Target inference rate:  $\geq$  XX FPS

End-to-end event latency:  $\leq$  XXX ms

**We'll benchmark hardware before freezing these numbers rather than inventing them.**

---

### **NFR-002 — Accuracy**

The system shall be evaluated using:

- activity accuracy;
- precision;
- recall;
- macro F1;
- confusion matrix.

Procedure validation shall separately report:

- skipped-step detection;
- out-of-order detection;
- false alerts.

---

### **NFR-003 — Reliability**

The system shall avoid changing procedure state based solely on low-confidence observations.

---

### **NFR-004 — Availability**

ASTRA-E shall continue core operation during temporary network/ground communication loss.

---

### **NFR-005 — Extensibility**

New experiments should be deployable through a new experiment definition and appropriate trained models without redesigning the complete application architecture.

---

### **NFR-006 — Maintainability**

The system shall maintain separation between:

Perception  
Temporal AI  
Procedure Logic  
Guidance  
Logging  
UI

---

### **NFR-007 — Resource Efficiency**

The deployed model should be optimized for available edge hardware.

We will measure:

- RAM usage;
  - VRAM usage;
  - CPU utilization;
  - GPU utilization;
  - model size;
  - inference latency;
  - power consumption where measurable.
- 

## 18. AI/ML Requirements

This deserves its own section because this is fundamentally an AI system.

### **ML-001 — Custom Dataset**

The system shall support a dataset representing the target experiment.

Dataset samples shall contain:

Video

Action Label

Object Label

Temporal Boundaries

Human/Hand Information

Procedure Step

---

### **ML-002 — Dataset Variability**

The dataset should include variation in:

- astronaut/operator;
  - clothing;
  - body proportions;
  - execution speed;
  - camera conditions;
  - lighting;
  - object positions;
  - action trajectories.
- 

### **ML-003 — Procedure Variations**

The dataset shall include both valid and invalid sequences:

Correct

Skipped

Out-of-order

Repeated

Incorrect object

Incomplete

This is important because **procedure validation itself needs testing data**.

---

## **19. Microgravity / Orientation Requirements**

This will be an advanced requirement.

### **ML-004 — Orientation Robustness**

The system should evaluate activity recognition under changes in astronaut orientation.

Test cases should include:

0°  
90°  
180°  
270°

and other simulated microgravity orientations where practical.

---

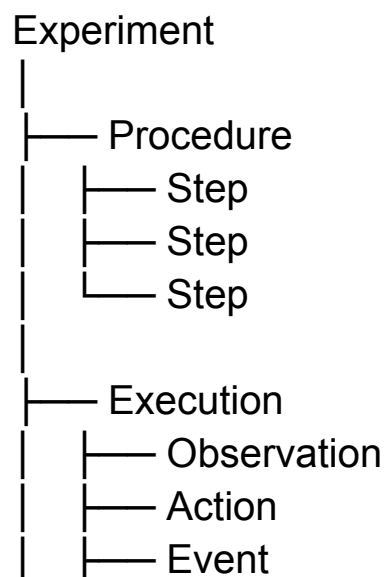
## **ML-005 — Rack-Relative Representation**

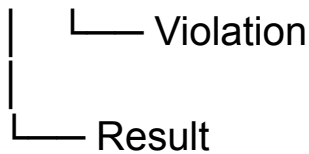
The advanced system should support representing astronaut pose relative to the experimental payload/rack coordinate frame rather than relying exclusively on gravity-oriented coordinates.

---

## **20. Data Model**

At the system level, we should have something approximately like:





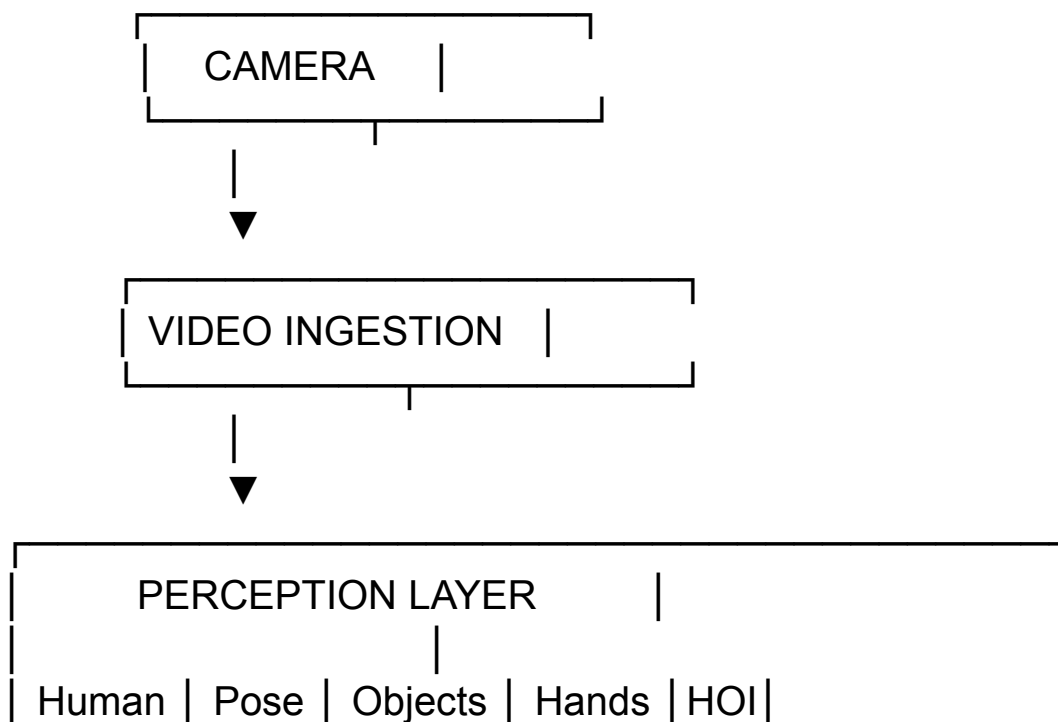
An event could be:

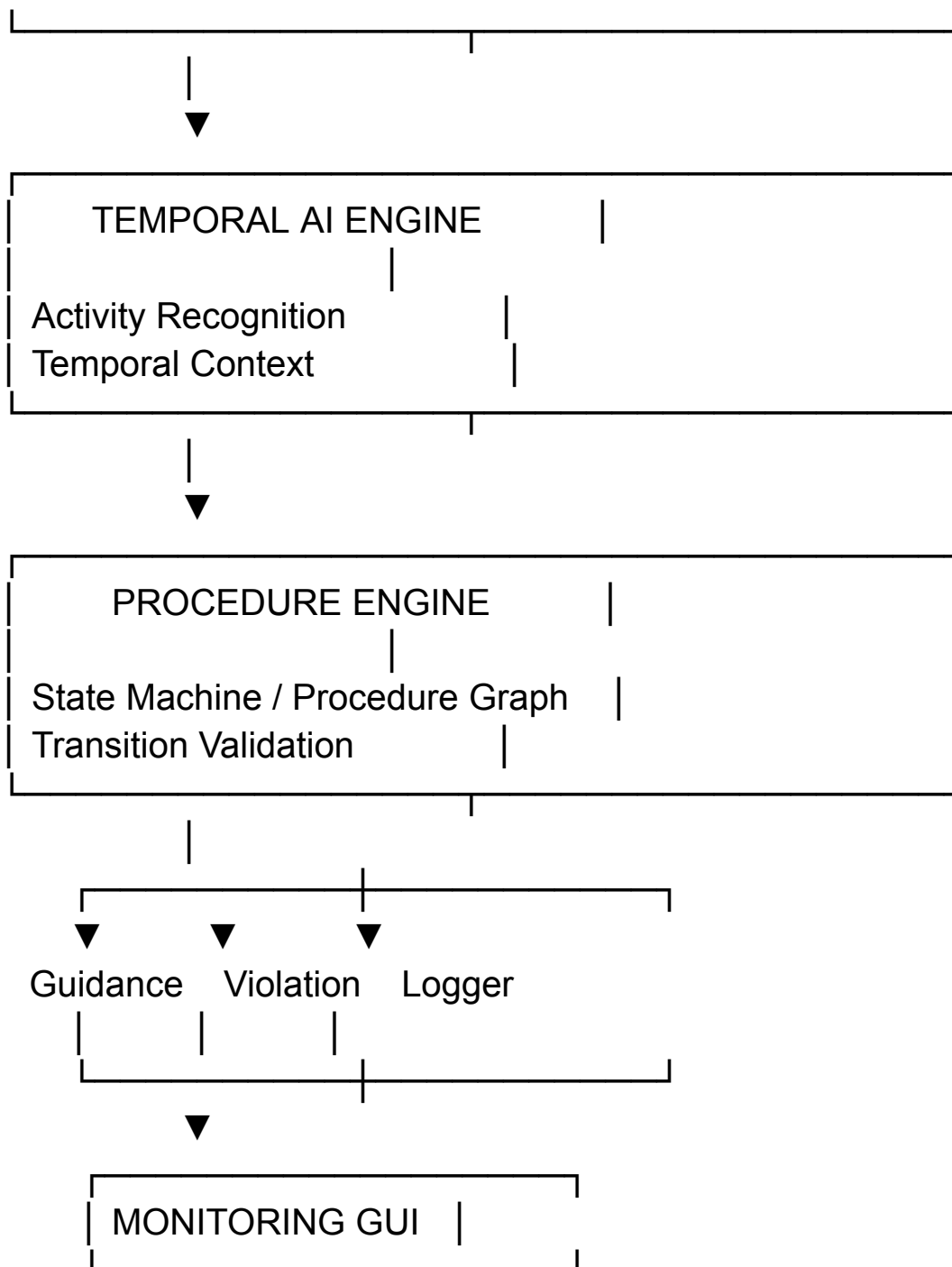
```
{
  "event_id": "EVT-00125",
  "experiment_id": "EXP-001",
  "timestamp": 172.521,
  "current_step": "S04",
  "expected_action": "PICK_YELLOW",
  "observed_action": "PICK_RED",
  "confidence": 0.93,
  "event_type": "INCORRECT_OBJECT"
}
```

---

## 21. High-Level Architecture

I would freeze the architecture at this level:





---

## 22. Critical Architectural Principle

There is one requirement I would make **non-negotiable**:

**AI perception ≠ procedure reasoning**

We don't want:

Video → Neural Network → "Everything is okay"

Instead:

VIDEO



AI



"I observe PLACE\_YELLOW with 92% confidence."



PROCEDURE ENGINE



"PLACE\_YELLOW is the expected transition."



VALID

or:

AI



"I observe CLOSE\_CONTAINER with 94% confidence."



PROCEDURE ENGINE



"Expected PLACE\_YELLOW."



VIOLATION

This gives ASTRA-E **deterministic procedural reasoning on top of probabilistic perception.**

That's an excellent architecture for this problem.

---

## 23. Acceptance Criteria

The first prototype will be considered successful if it can demonstrate the following:

### **AC-01 — Correct Procedure**

Given a valid experiment sequence:

$S1 \rightarrow S2 \rightarrow S3 \rightarrow S4 \rightarrow S5 \rightarrow S6$

ASTRA-E shall recognize the steps and reach:

EXPERIMENT COMPLETE

without false violation alerts.

### **AC-02 — Skipped Step**

Given:

$S1 \rightarrow S2 \rightarrow S4$

ASTRA-E shall identify:

S3 SKIPPED

### **AC-03 — Out-of-Order**

Given:

$S1 \rightarrow S2 \rightarrow S4 \rightarrow S3$

ASTRA-E shall identify the invalid transition.

### **AC-04 — Wrong Object**

Given:

Expected: PICK\_RED

Observed: PICK\_YELLOW

the system shall identify the incorrect interaction.

#### **AC-05 — Guidance**

For an incomplete procedure, the system shall identify the next expected step.

#### **AC-06 — Logging**

Every validated action and detected violation shall generate a timestamped event.

#### **AC-07 — Offline**

The complete core pipeline shall function without Internet connectivity.

#### **AC-08 — Orientation**

The advanced prototype shall evaluate recognition under simulated changes in astronaut orientation.

---

## **24. Requirements Traceability**

Eventually we'll maintain a matrix like:

| <b>Requirement</b>     | <b>Module</b>      | <b>Test</b> |
|------------------------|--------------------|-------------|
| FR-003 Human Detection | Perception         | T-HUMAN-001 |
| FR-007 HOI             | Interaction Engine | T-HOI-001   |
| FR-008 HAR             | Temporal AI        | T-HAR-001   |
| FR-013 Skip Detection  | Procedure Engine   | T-PROC-001  |

|                        |                     |                  |
|------------------------|---------------------|------------------|
| FR-014<br>Out-of-order | Procedure<br>Engine | T-PROC-00<br>2   |
| FR-018 Next Step       | Guidance            | T-GUIDE-0<br>01  |
| FR-023 Logging         | Logger              | T-LOG-001        |
| FR-027 Offline         | Runtime             | T-EDGE-00<br>1   |
| ML-004<br>Orientation  | Spatial AI          | T-MICROG-<br>001 |

This will later let us prove:

**Every requirement has an implementation and a test.**

That's exactly how we should approach the project if we want ASTRA-E to look like a serious engineering project rather than an SIH prototype.

---

## 25. SRS v1 — What is now frozen

I'd consider these **architectural decisions** frozen:

### Core

#### ASTRA-E

Autonomous Space Task Recognition & Assistance for  
Experiments

### Input

Fixed-camera video

### Perception

Human + pose + objects + hands + interactions

## **AI**

Temporal activity recognition

## **Reasoning**

Deterministic experiment procedure graph/state machine

## **Output**

Current state + next step + violation + confidence

## **Assistance**

GUI + offline voice

## **Storage**

Local video + structured experiment logs

## **Deployment**

Offline edge inference

## **Advanced capability**

Microgravity/orientation-aware spatial representation

# ASTRA-E — Solution & Architecture Plan

Architecture Version 1.0

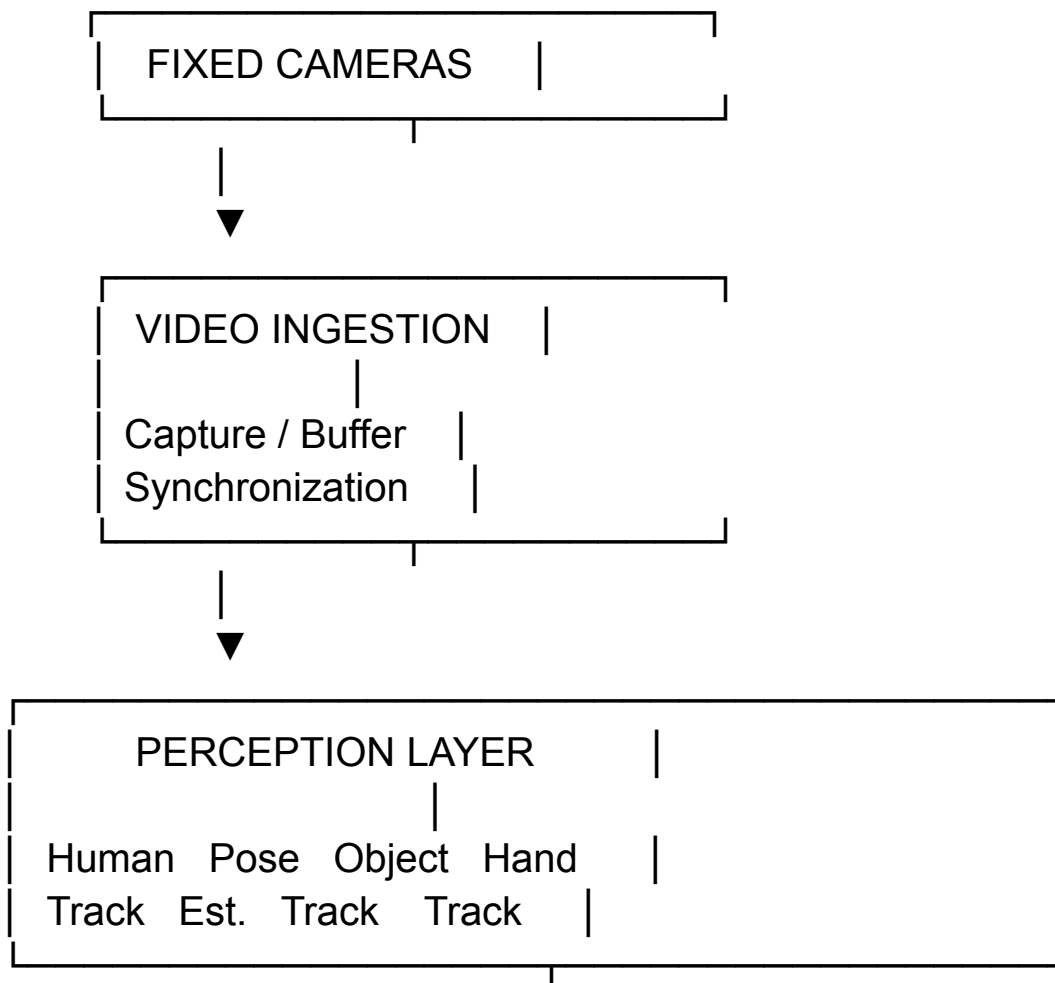
---

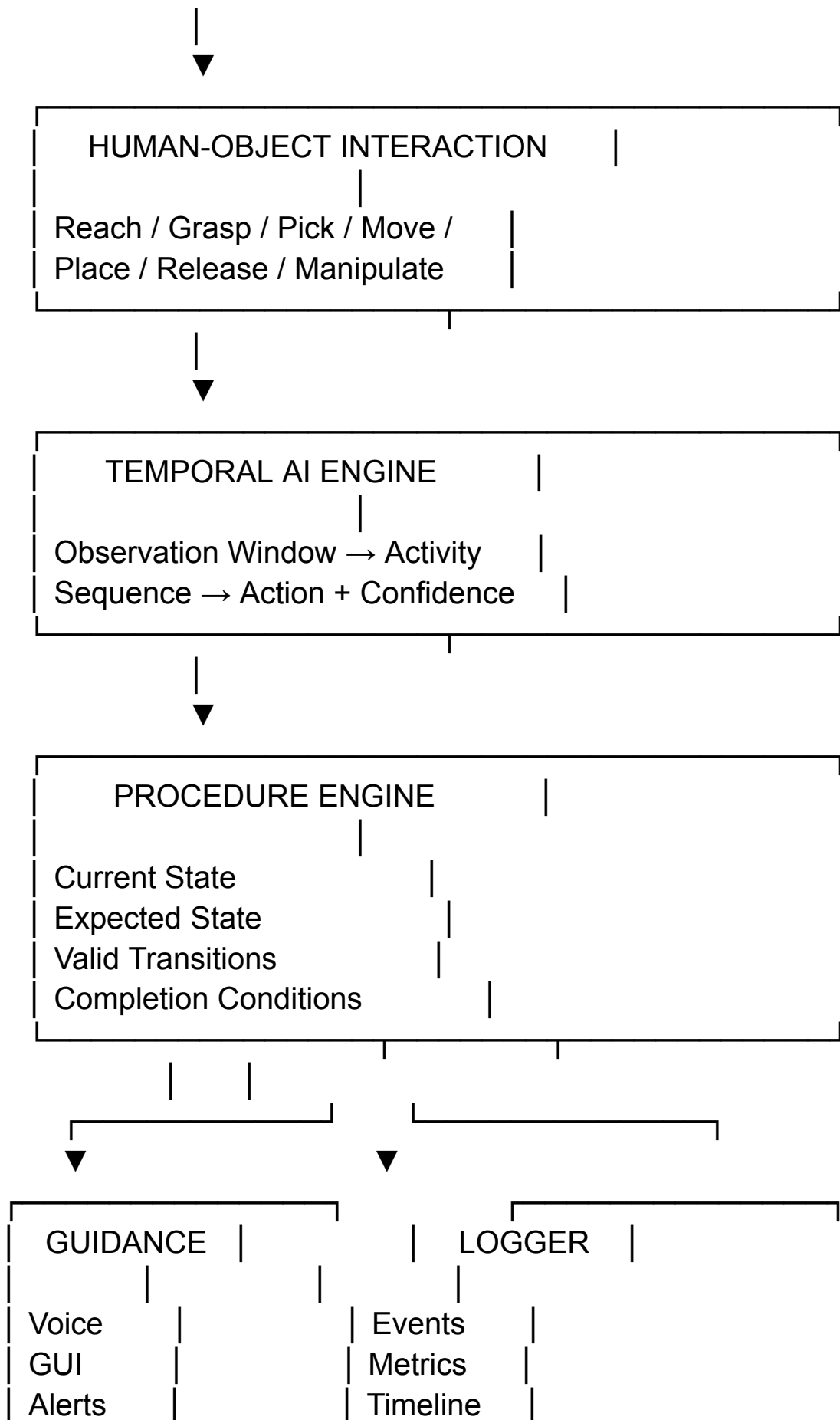
## 1. Solution Concept

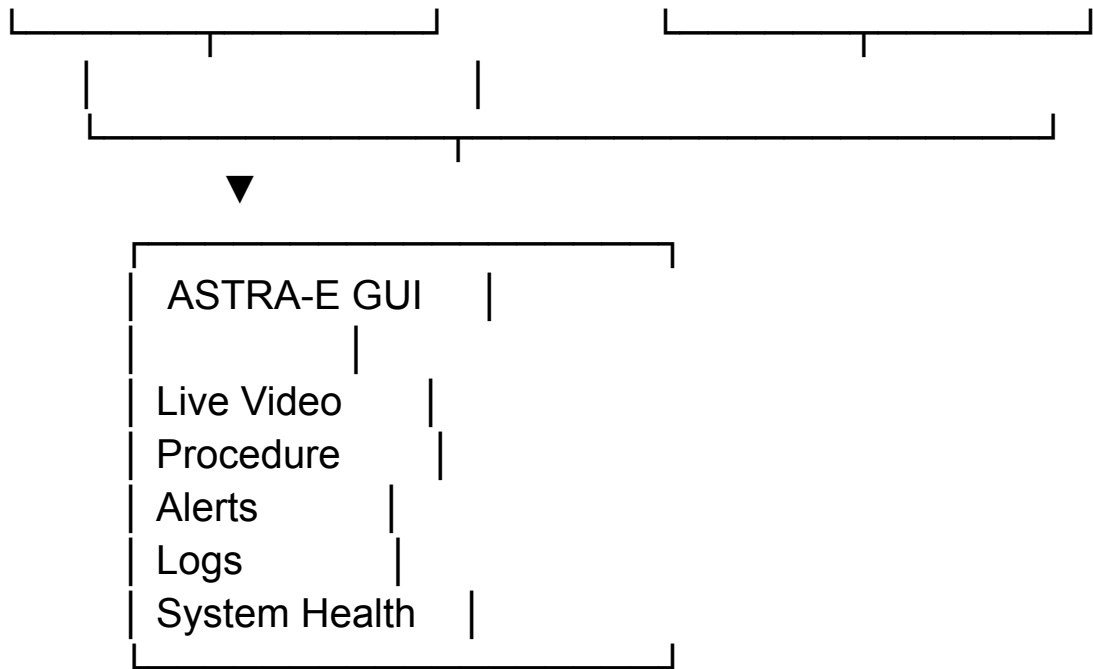
The core idea is:

**ASTRA-E observes the astronaut, converts video into structured observations, converts observations into actions, and validates those actions against a formal experiment procedure.**

The complete pipeline is:

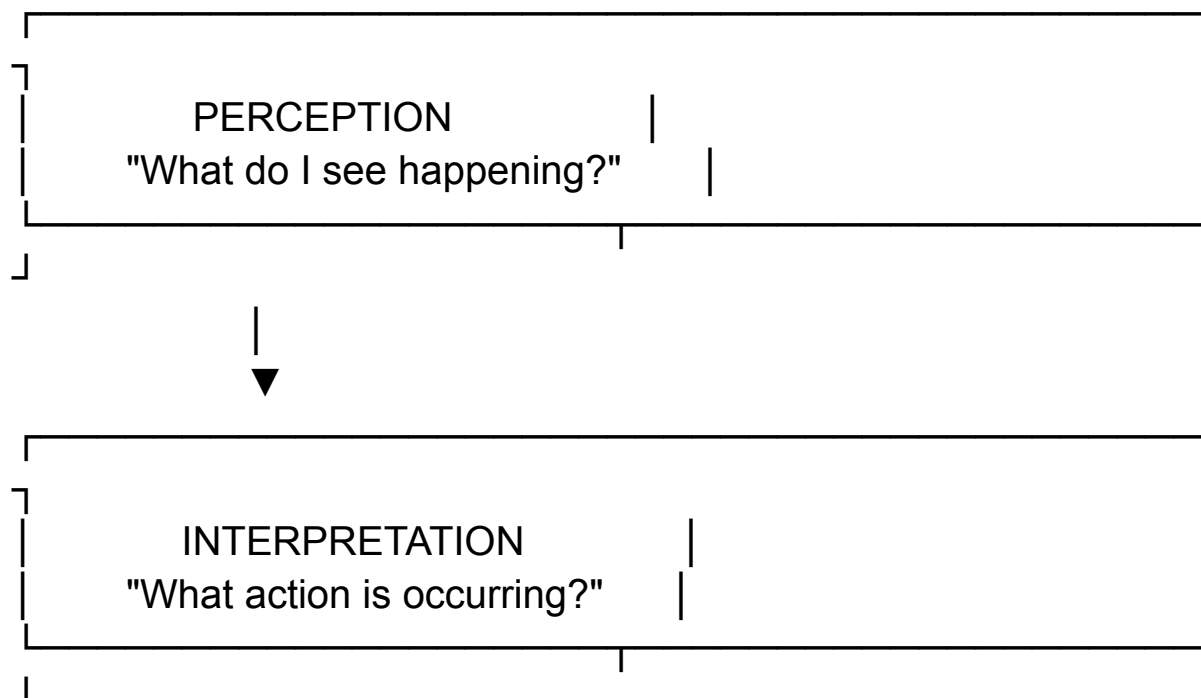


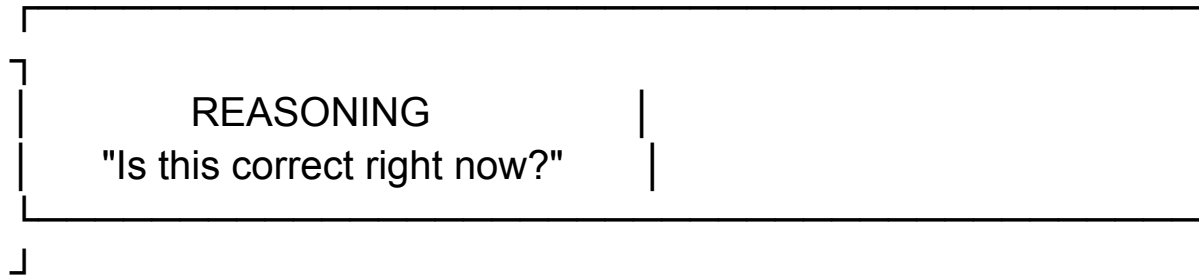




## 2. The Most Important Architectural Decision

We will have **three fundamentally different layers**:





This separation is **critical**.

AI should say:

PLACE\_YELLOW – confidence 0.93

It should **not** say:

Experiment is correct.

The Procedure Engine makes that decision.

That gives us:

**Probabilistic perception + deterministic procedural reasoning.**

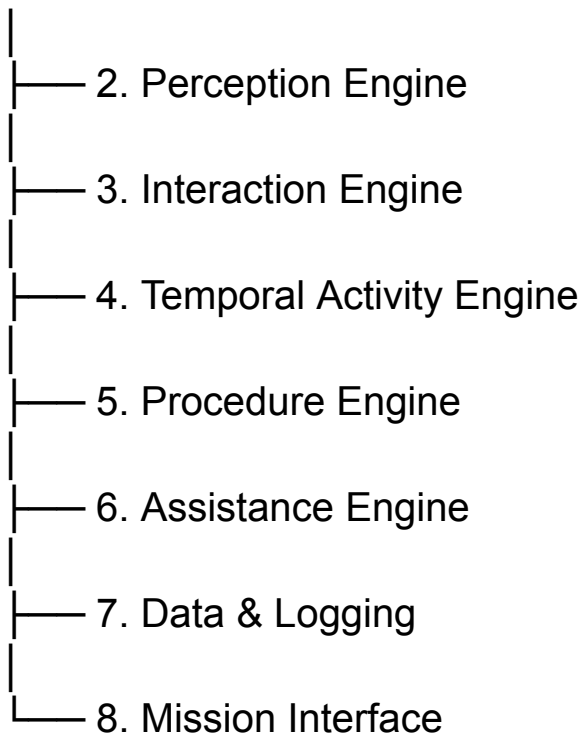
This is one of the strongest architectural decisions we can make.

### 3. Overall System Architecture

I'd divide ASTRA-E into **8 major subsystems**.

ASTRA-E





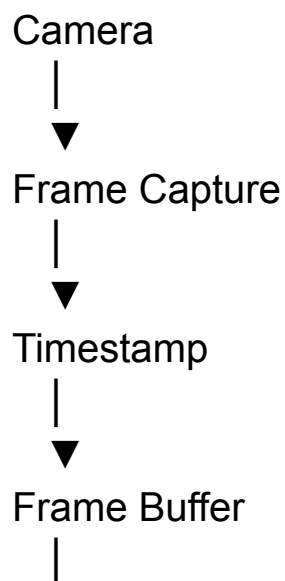
Let's define each.

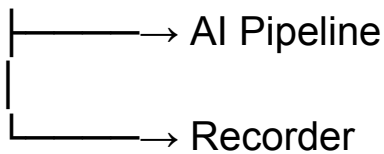
---

## 4. Subsystem 1 — Video Acquisition

### Responsibility

Get reliable frames from fixed payload cameras.





### Components

CameraManager

FrameBuffer

TimestampManager

StreamManager

VideoRecorder

### Important design

We should timestamp frames **at ingestion**, not later.

Why?

Because our entire system depends on temporal reasoning:

Frame



Action



Procedure transition



Event

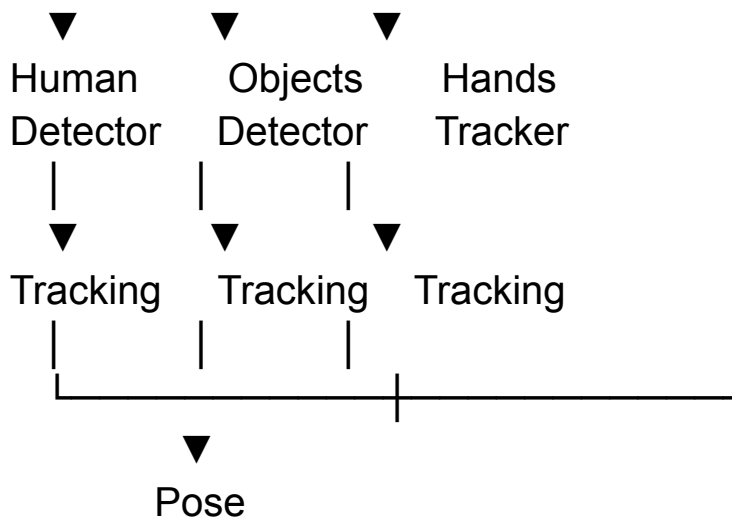
Everything needs a consistent timeline.

---

## 5. Subsystem 2 — Perception Engine

This converts pixels into structured entities.





## Output

Instead of passing images everywhere, we produce a structured observation:

```
{
  "timestamp": 12.42,

  "human": {
    "id": 1,
    "bbox": [120, 80, 450, 720]
  },

  "objects": [
    {
      "id": "red_box",
      "type": "RED_COMPONENT",
      "bbox": [500, 300, 560, 350],
      "confidence": 0.98
    }
  ],

  "hands": {
    "left": [420, 300],
    "right": [520, 340]
```

```
},  
  
"pose": {  
  "keypoints": []  
}  
}
```

This is an important architectural boundary.

The rest of ASTRA-E doesn't need to care whether we used YOLO, RT-DETR, MediaPipe, MMPose, etc.

---

## 6. Subsystem 3 — Interaction Engine

This is where perception becomes **interaction**.

Suppose:

Hand  
↓  
approaches  
↓  
RED OBJECT  
↓  
distance decreases  
↓  
grasp  
↓  
object follows hand

The engine can infer:

HAND\_APPROACH  
↓  
GRASP

↓  
OBJECT\_PICKED

Similarly:

Object  
↓  
moves  
↓  
TARGET REGION  
↓  
release

becomes:

OBJECT\_PLACED

### **Why this layer exists**

We don't want the temporal model to learn every microscopic interaction directly.

Instead:

Vision  
↓  
Interaction primitives  
↓  
Temporal Activity

This gives us better interpretability.

---

## **7. Subsystem 4 — Temporal Activity Engine**

Now we answer:

## What action is actually occurring?

Input:

Observation[t-30:t]

rather than a single frame.

For example:

t0 hand far from object  
t1 hand approaching  
t2 hand near object  
t3 grasp  
t4 object follows hand  
t5 target reached  
t6 release

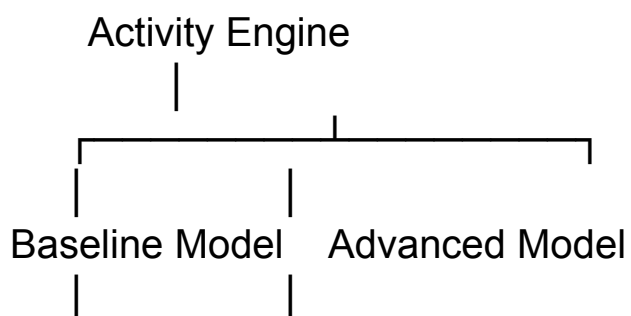
Output:

```
{  
  "action": "PLACE_RED",  
  "confidence": 0.94,  
  "start_time": 15.2,  
  "end_time": 17.8  
}
```

---

## 8. Model Architecture

We should make this **model-pluggable**.



|          |             |
|----------|-------------|
| LSTM/GRU | TCN         |
| ST-GCN   | Transformer |
| PoseC3D  | Video Model |

The architecture should not depend on a particular model.

**Our development strategy:**

### **Baseline**

Pose + object features



LSTM/GRU

Then benchmark:

ST-GCN

TCN

Small Transformer

Only after establishing the baseline should we consider heavier video models.

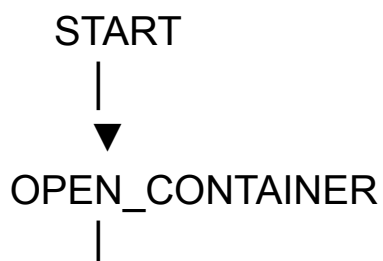
---

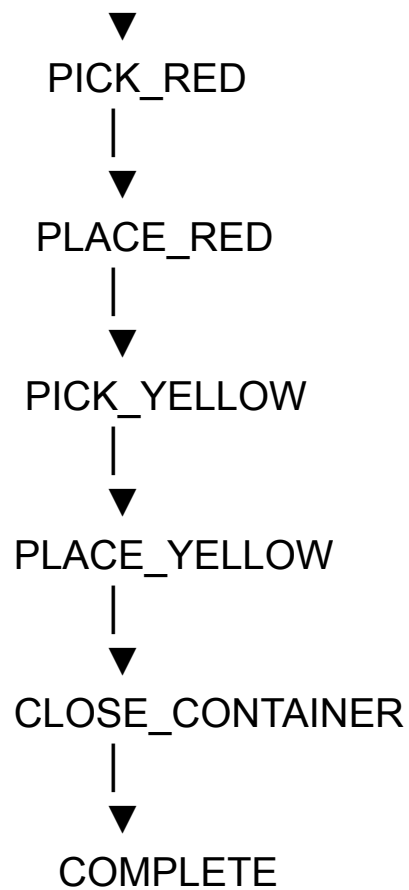
## **9. Subsystem 5 — Procedure Engine**

This is ASTRA-E's **brain for experiment logic**.

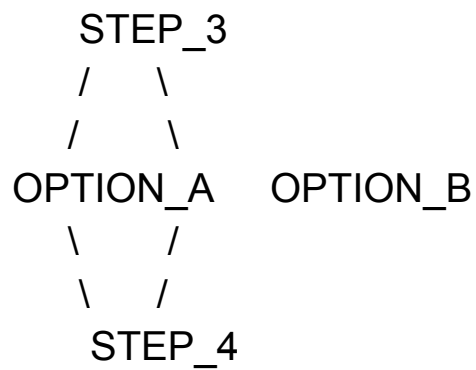
The procedure should be represented as a graph rather than hardcoded Python `if/else`.

For example:





But the graph can also support branches:



That makes the engine usable for future experiments.

---

## 10. Procedure Engine State

Internally:

ProcedureState

could conceptually contain:

experiment\_id  
procedure\_version  
current\_step  
completed\_steps  
pending\_steps  
allowed\_transitions  
last\_action  
last\_transition  
violations  
status

Example:

```
{  
  "experiment": "EXP_001",  
  "current_step": "S04",  
  "current_action": "PLACE_RED",  
  "next_expected": "PICK_YELLOW",  
  "status": "RUNNING"  
}
```

---

## 11. Action → Procedure Validation

Suppose AI outputs:

Observed:

PICK\_YELLOW

Confidence:

0.94

Procedure Engine receives:

Current State:  
S04 — PLACE\_RED

Expected:  
S05 — PICK\_YELLOW

It performs:  
T(S04, PICK\_YELLOW)

If valid:  
VALID TRANSITION  
↓  
S05

If invalid:  
INVALID TRANSITION  
↓  
Violation Detector

---

## 12. Confidence-Aware State Transitions

This is another major architectural component.

We should **never transition on raw model output**.

Instead:

AI prediction  
↓  
Confidence Filter  
↓  
Temporal Stabilization  
↓  
Action Confirmation

↓  
Procedure Engine

For example:

Frame predictions:

PICK\_RED 0.51

PICK\_RED 0.62

PICK\_RED 0.81

PICK\_RED 0.92

PICK\_RED 0.94

Only after sufficient temporal evidence:

CONFIRMED ACTION

↓  
PICK\_RED

This significantly reduces false transitions.

---

## 13. Uncertainty State

The procedure engine should explicitly support:

NORMAL

UNCERTAIN

VIOLATION

COMPLETED

PAUSED

ERROR

For example:

AI:

PLACE\_YELLOW = 0.47

Procedure:

Expected PLACE\_YELLOW

Decision:

UNCERTAIN

Rather than:

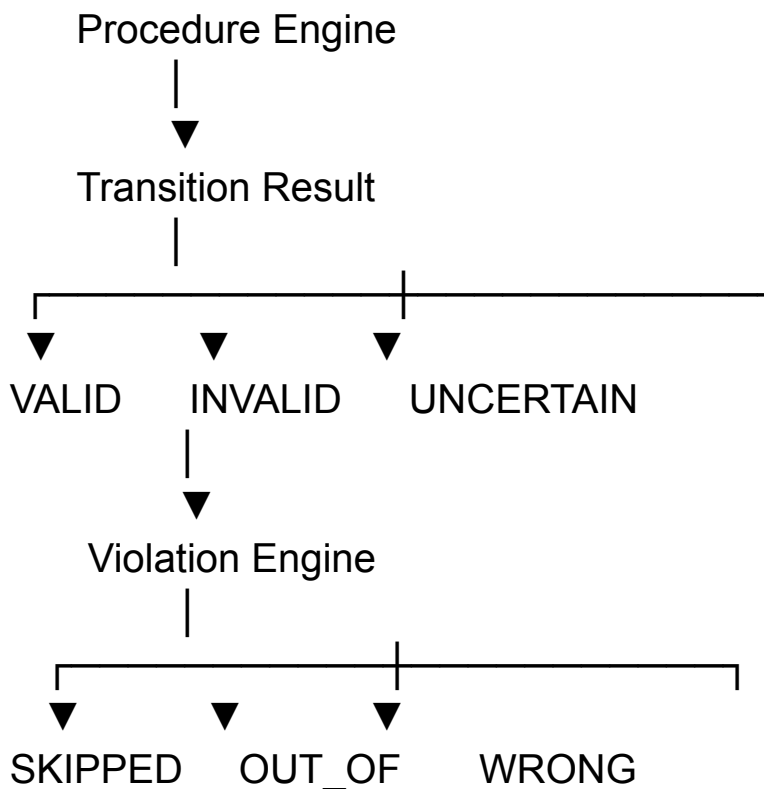
VIOLATION!!!

This is especially important in computer vision because occlusion is inevitable.

---

## 14. Violation Engine

Rather than embedding violation logic inside the procedure engine, I'd make it its own component.



ORDER      OBJECT

This makes testing much easier.

---

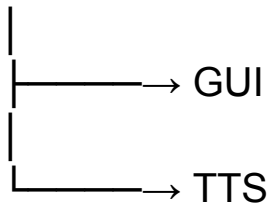
## 15. Assistance Engine

The Assistance Engine converts machine decisions into human-readable assistance.

Procedure Event



Message Generator



Example:

Event:

SKIPPED\_STEP



Message:

"Please complete the yellow component placement before closing the container."

---

## 16. Don't put an LLM in the critical loop

This deserves to be explicit.

✗ Not:

Video



LLM



Is astronaut doing experiment correctly?



Video



Computer Vision



Temporal AI



Procedure Graph

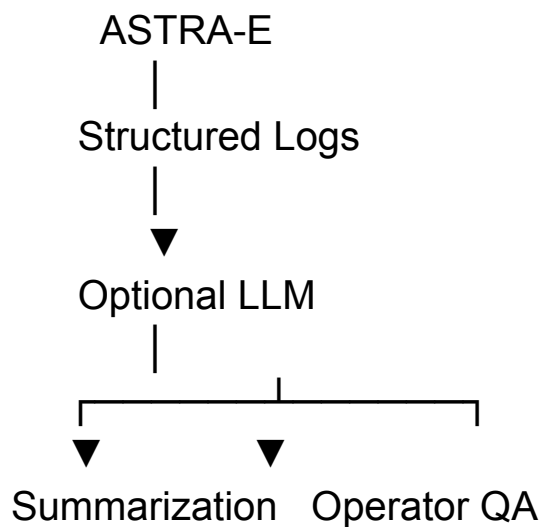


Deterministic decision



Guidance

If we eventually add an LLM:



The LLM remains **outside the safety/validation loop**.

---

# 17. Data Architecture

We should use two different data paths.

## Hot path

For real-time operation:

Camera



AI



Procedure



Alert

## Cold path

For persistence:

AI Events

Procedure Events

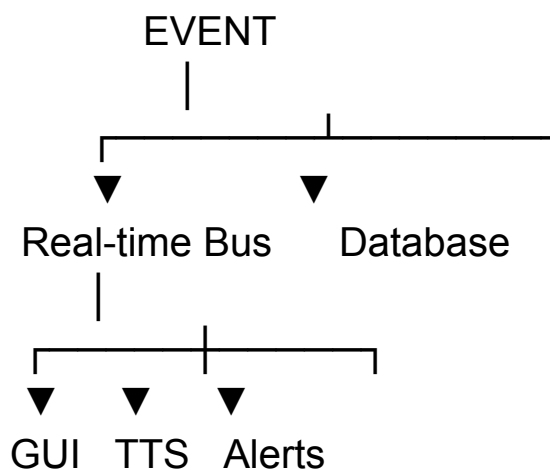
Video Metadata

System Metrics



Storage

So:



---

## 18. Database

For our prototype:

**SQLite**

is sufficient.

We don't need PostgreSQL for a single onboard system.

Potential schema:

experiments  
procedures  
procedure\_steps  
experiment\_runs  
observations  
actions  
events  
violations  
system\_metrics

Video itself should **not** be stored inside SQLite.

Store:

/videos/  
  EXP001/  
    camera\_01/  
    camera\_02/

and keep metadata in SQLite.

---

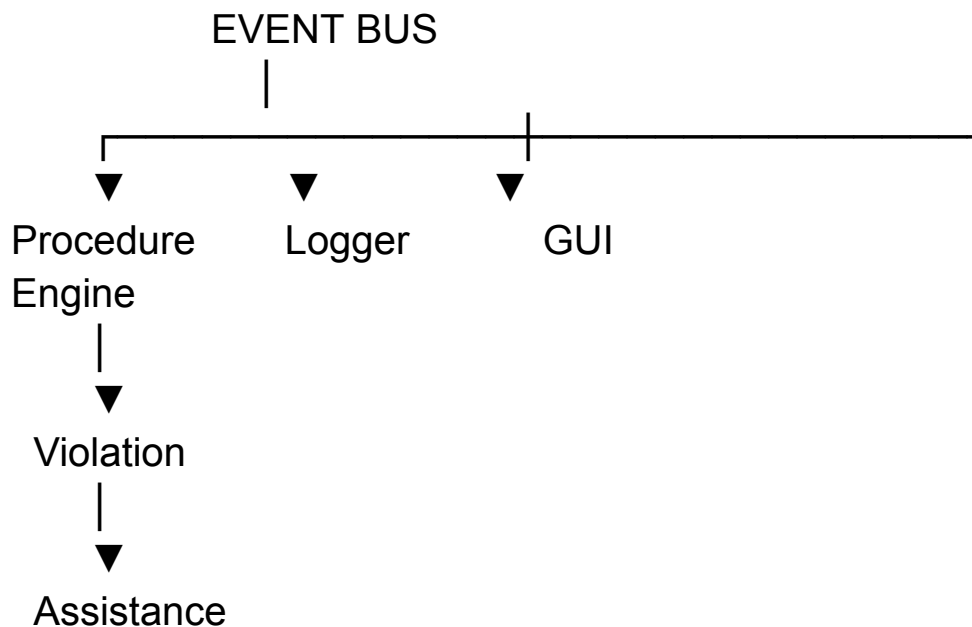
## 19. Event Bus

Internally, everything should communicate through structured events.

Example:

```
{  
  "event_type": "ACTION_CONFIRMED",  
  "timestamp": 172.52,  
  "action": "PICK_YELLOW",  
  "confidence": 0.94  
}
```

Then:



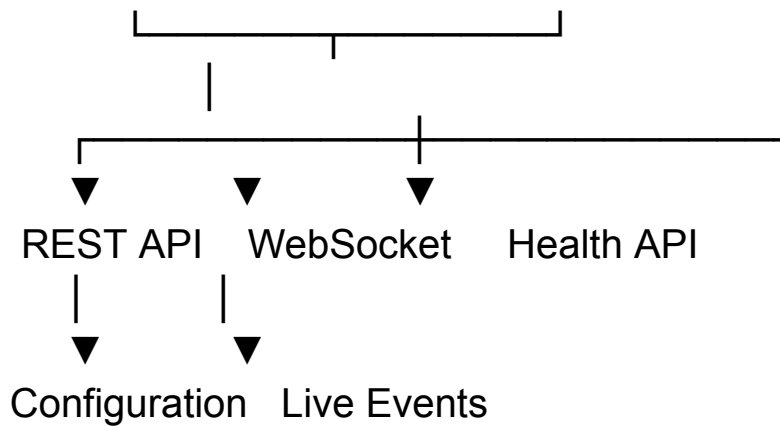
This means our modules don't become tightly coupled.

---

## 20. Backend Architecture

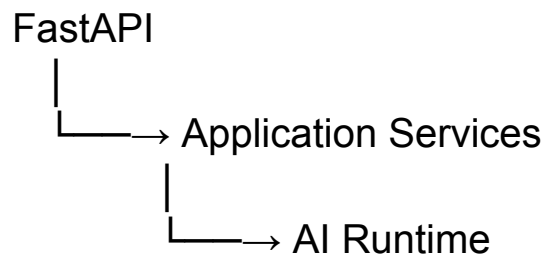
For the prototype:





FastAPI should **not** run the actual heavy ML inference in the HTTP request handler.

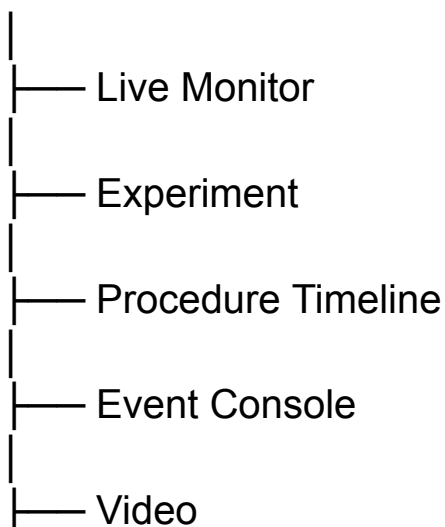
Instead:



## 21. Frontend Architecture

React + TypeScript:

ASTRA-E Dashboard



- Analytics
- System Health

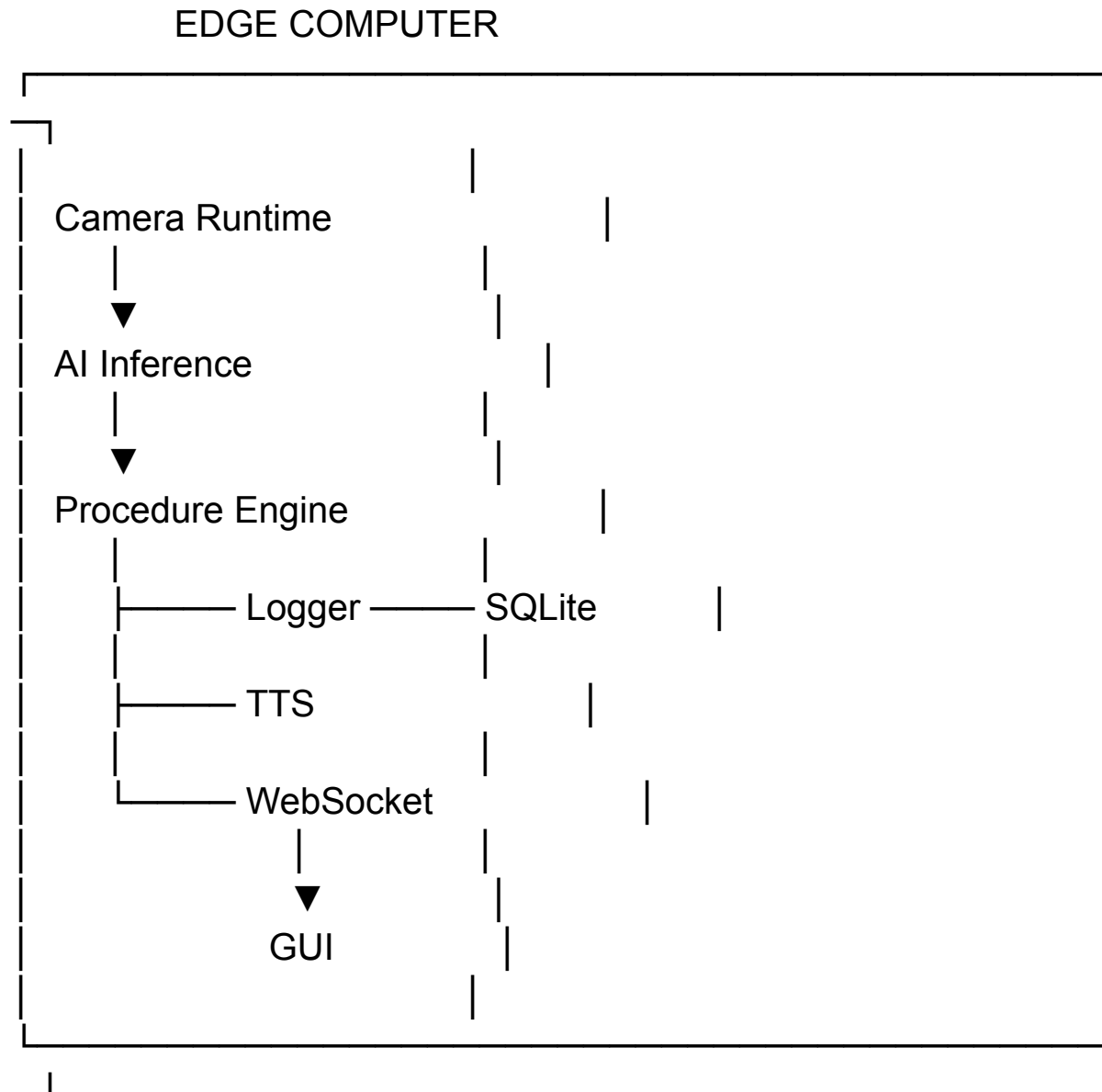
Live monitor

The most important screen.

|                                                                                     |     |  |  |                   |  |  |  |
|-------------------------------------------------------------------------------------|-----|--|--|-------------------|--|--|--|
| ASTRA-E                                                                             |     |  |  | ● OPERATIONAL     |  |  |  |
| LIVE VIDEO                                                                          |     |  |  | EXPERIMENT        |  |  |  |
|  |     |  |  | Step 4 / 7        |  |  |  |
|                                                                                     |     |  |  | ✓ Open Container  |  |  |  |
|                                                                                     |     |  |  | ✓ Pick Red        |  |  |  |
|                                                                                     |     |  |  | ✓ Place Red       |  |  |  |
|                                                                                     |     |  |  | → Pick Yellow     |  |  |  |
|                                                                                     |     |  |  | ○ Place Yellow    |  |  |  |
|                                                                                     |     |  |  | ○ Close Container |  |  |  |
| PERCEPTION                                                                          |     |  |  | EVENT             |  |  |  |
| Human                                                                               | 98% |  |  | NORMAL            |  |  |  |
| Red                                                                                 | 99% |  |  | Next: Pick Yellow |  |  |  |
| Yellow                                                                              | 97% |  |  | Confidence: 94%   |  |  |  |

## 22. Deployment Architecture

For SIH, I'd package it as a **single-node edge system**.



No cloud dependency.

---

## 23. Suggested Technology Stack

AI

Python  
PyTorch  
OpenCV  
ONNX Runtime  
CUDA/TensorRT (if available)

## **Vision**

Initially benchmark:

YOLO-family detector  
MMPose / MediaPipe  
OpenCV tracking

## **Temporal AI**

Start with:

LSTM/GRU

Then benchmark:

ST-GCN  
TCN  
Transformer

## **Backend**

FastAPI  
Pydantic  
WebSocket

## **Database**

SQLite  
SQLAlchemy

## **Frontend**

React  
TypeScript  
Tailwind

WebSocket client

**Voice**

Offline TTS

**Deployment**

Docker

ONNX

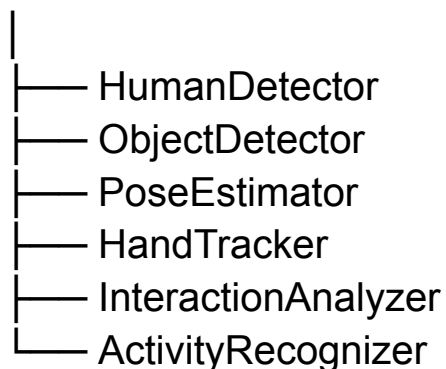
TensorRT

---

## 24. Model Runtime Architecture

I recommend we isolate AI inference behind a clean interface.

AI Runtime



Every component should expose something conceptually like:

```
result = model.infer(input)
```

rather than allowing the rest of the application to know model-specific details.

Then we can swap:

YOLO → RT-DETR

MediaPipe → MMPose

LSTM → ST-GCN

PyTorch → ONNX

without rewriting ASTRA-E.

---

## 25. Configuration-Driven Experiments

This is **very important**.

The experiment itself should be configuration, not application code.

Something like:

experiment:

id: EXP-001

name: BAS Sample Experiment

workspace:

cameras:

- CAM-01

objects:

- RED\_COMPONENT

- YELLOW\_COMPONENT

- CONTAINER

procedure:

- id: S01

action: OPEN\_CONTAINER

- id: S02

action: PICK

object: RED\_COMPONENT

- id: S03

action: PLACE

object: RED\_COMPONENT

target: TARGET\_A

- id: S04  
action: PICK  
object: YELLOW\_COMPONENT
- id: S05  
action: PLACE  
object: YELLOW\_COMPONENT  
target: TARGET\_B
- id: S06  
action: CLOSE\_CONTAINER

Then loading a new experiment becomes:

New Experiment



New YAML



Procedure Engine

rather than:

New Experiment



Modify source code



Recompile

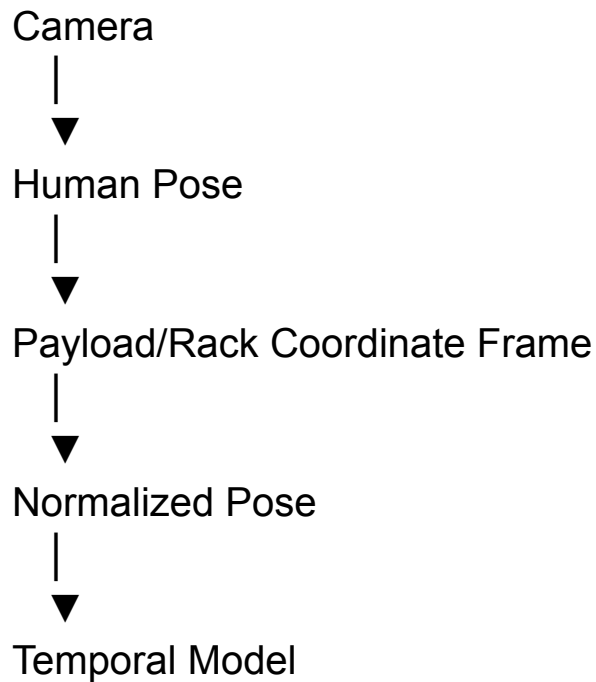
That is a **major architectural win**.

---

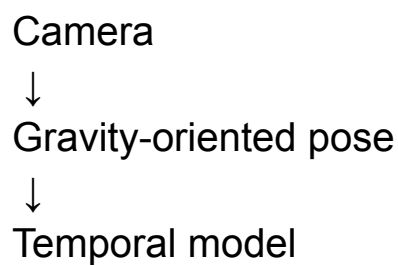
## 26. Microgravity Architecture

We should design for it without making it a blocker.

Our spatial pipeline can eventually become:



Instead of:



The key abstraction:

$$P_{\text{normalized}} = T_{\text{rack}}^{-1} P_{\text{camera}}$$

Conceptually, we're asking:

**Where is the astronaut relative to the experiment?**

rather than:

Where is the astronaut relative to Earth gravity?

---

# 27. Development Architecture: Don't Build Everything at Once

This is where we need discipline.

## Phase 0 — Architecture

SRS



HLD



Interfaces



Data schemas

**No fancy AI yet.**

---

## Phase 1 — Perception MVP

Camera



Human detection



Object detection



Visualization

Goal:

Can we reliably see the experiment?

---

## Phase 2 — Action Recognition

Video

↓  
Pose + Objects + Hands  
↓  
Temporal Model  
↓  
Action

Goal:

Can we recognize the experiment actions?

---

### **Phase 3 — Procedure Engine**

Action  
↓  
Procedure Graph  
↓  
VALID / INVALID

Goal:

Can we detect skipped/out-of-order actions?

This is our **first true ASTRA-E milestone**.

---

### **Phase 4 — Assistance**

Procedure  
↓  
Guidance  
↓  
GUI + TTS

---

### **Phase 5 — Logging**

Events



SQLite



Experiment Report

---

## Phase 6 — Edge Optimization

PyTorch



ONNX



TensorRT

Measure:

FPS

Latency

Memory

GPU

CPU

---

## Phase 7 — Microgravity Extension

Orientation augmentation



Rack-relative pose



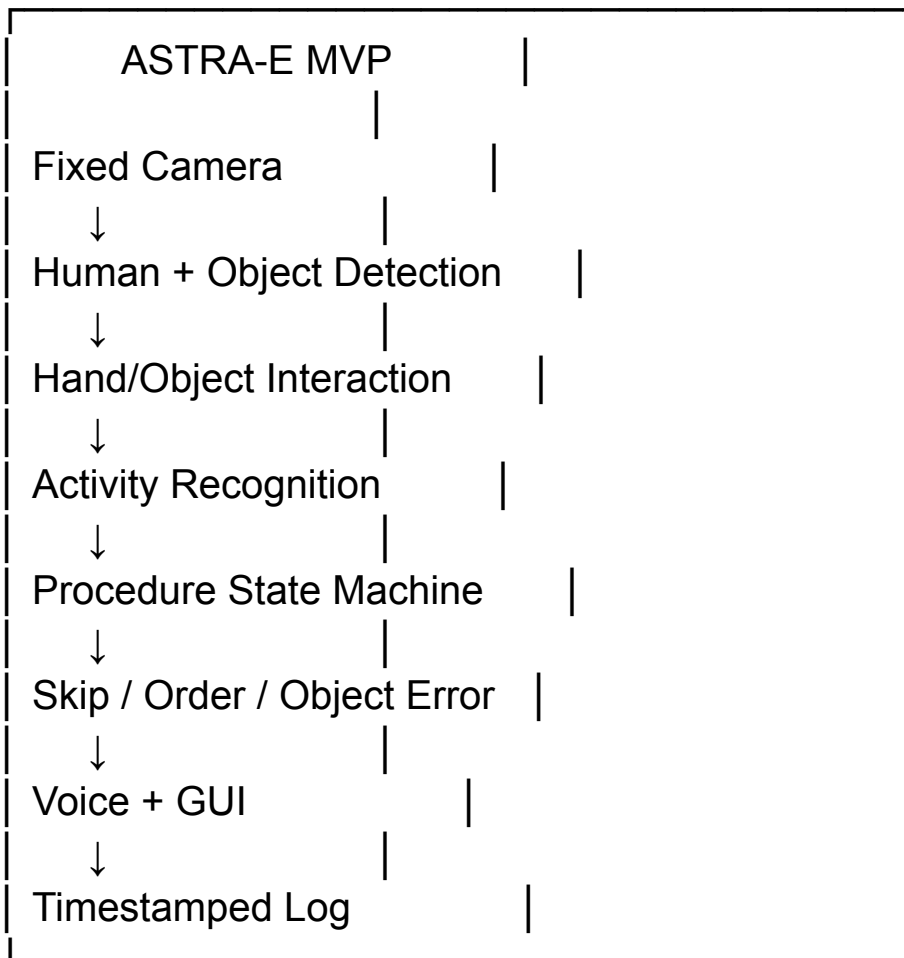
Orientation-aware recognition

This becomes our **advanced/research component**.

---

## 28. The MVP

For SIH, our minimum impressive system should be:

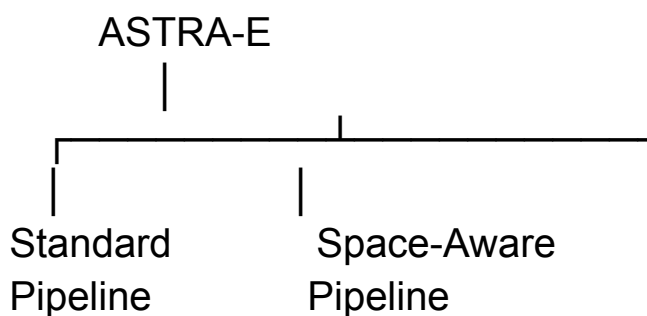


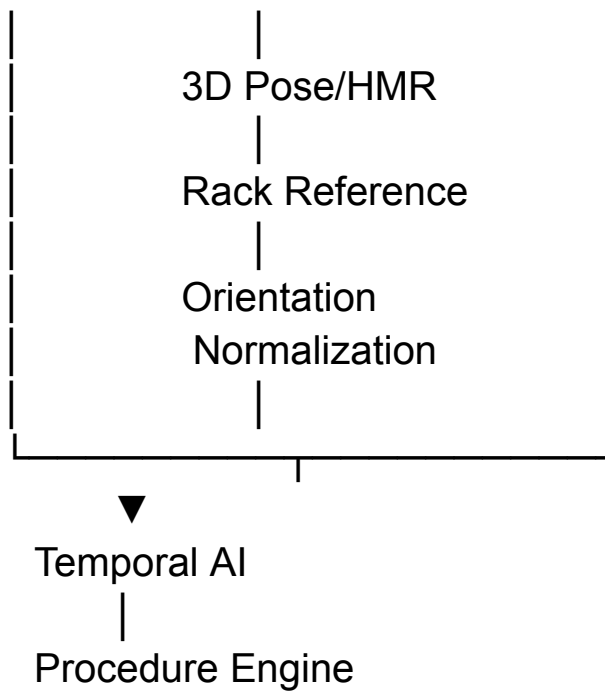
If this works **reliably**, we already have a very strong SIH prototype.

---

## 29. Advanced ASTRA-E

Then we layer research capability on top:





This gives us two stories:

#### Engineering story

**We built a working autonomous experiment assistant.**

#### Research story

**We investigated orientation-invariant activity recognition for microgravity environments.**

That's a very strong combination.

---

## 30. What We Should Freeze Now

I'd freeze these architectural principles:

### Decision                      ASTRA-E

Architect    Modular pipeline  
ure

|             |                               |
|-------------|-------------------------------|
| Input       | Fixed-camera video            |
| Perception  | Human + object + pose + hands |
| Interaction | Explicit HOI layer            |
| Activity    | Temporal recognition          |
| Reasoning   | Deterministic procedure graph |
| Confidence  | Required at every AI decision |
| Violations  | Dedicated violation engine    |
| Guidance    | GUI + offline TTS             |
| Logging     | Event-based + timestamped     |
| Databases   | SQLite initially              |
| Backend     | FastAPI                       |
| Frontend    | React + TypeScript            |
| AI runtime  | Model-agnostic                |
| Deployment  | Local edge                    |
| Cloud       | Not required                  |

|              |                                 |
|--------------|---------------------------------|
| LLM          | Optional, outside critical loop |
| Microgravity | Advanced extension              |

---

## 31. The Architecture We Should Design Next

I would now go one level deeper rather than immediately writing code.

Our next engineering artifact should be:

### **ASTRA-E HLD — High-Level Design**

Specifically, we should define **six contracts**:

1. Video → Perception
2. Perception → Interaction
3. Interaction → Activity
4. Activity → Procedure
5. Procedure → Assistance
6. Everything → Event/Logging

For each one we'll define the exact:

- input schema
- output schema
- interfaces
- data structures
- failure modes
- latency expectations
- ownership of each responsibility



# ASTRA-E — High-Level Design

HLD v1.0

## Autonomous Space Task Recognition & Assistance for Experiments

---

### 1. HLD Objective

The purpose of this HLD is to define the architecture of ASTRA-E at the subsystem level.

The HLD answers:

- What components exist?
- What responsibility does each component have?
- How do components communicate?
- What data flows through the system?
- Where does AI inference happen?
- How are experiment procedures represented?
- How are violations detected?
- How does the astronaut receive feedback?
- How does the system operate offline?
- How can another experiment be plugged into the system?

It intentionally does **not** define:

- Python classes
- individual neural-network architectures
- database SQL
- exact APIs
- exact model hyperparameters
- UI component implementation

Those belong in the LLD and implementation design.

---

## 2. Architectural Goals

ASTRA-E's architecture should satisfy seven primary goals.

### **G1 — Real-Time**

The system should process camera observations continuously and produce procedural decisions with low latency.

### **G2 — Offline First**

The core functionality must operate without cloud connectivity.

### **G3 — Modular AI**

Individual AI models must be replaceable without rewriting the system.

### **G4 — Deterministic Procedure Validation**

AI should recognize observations; deterministic logic should decide whether they conform to the experiment.

### **G5 — Explainability**

Every procedural decision should be traceable to an observed action and confidence.

### **G6 — Experiment Agnosticism**

The same software architecture should support different experiment definitions.

### **G7 — Future Space Readiness**

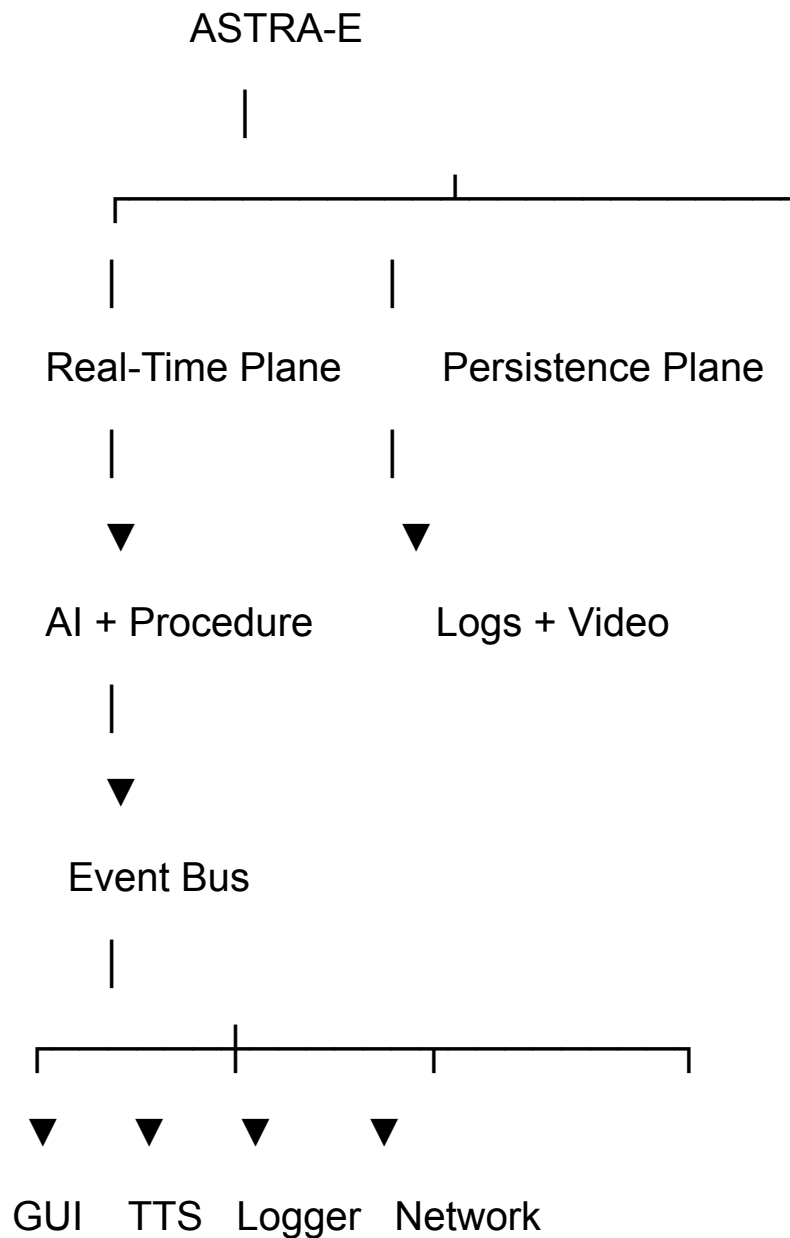
The architecture should allow orientation-aware and rack-relative perception to be added later.

---

### 3. Architectural Style

I recommend a **modular event-driven edge architecture**.

Not a traditional monolithic application.

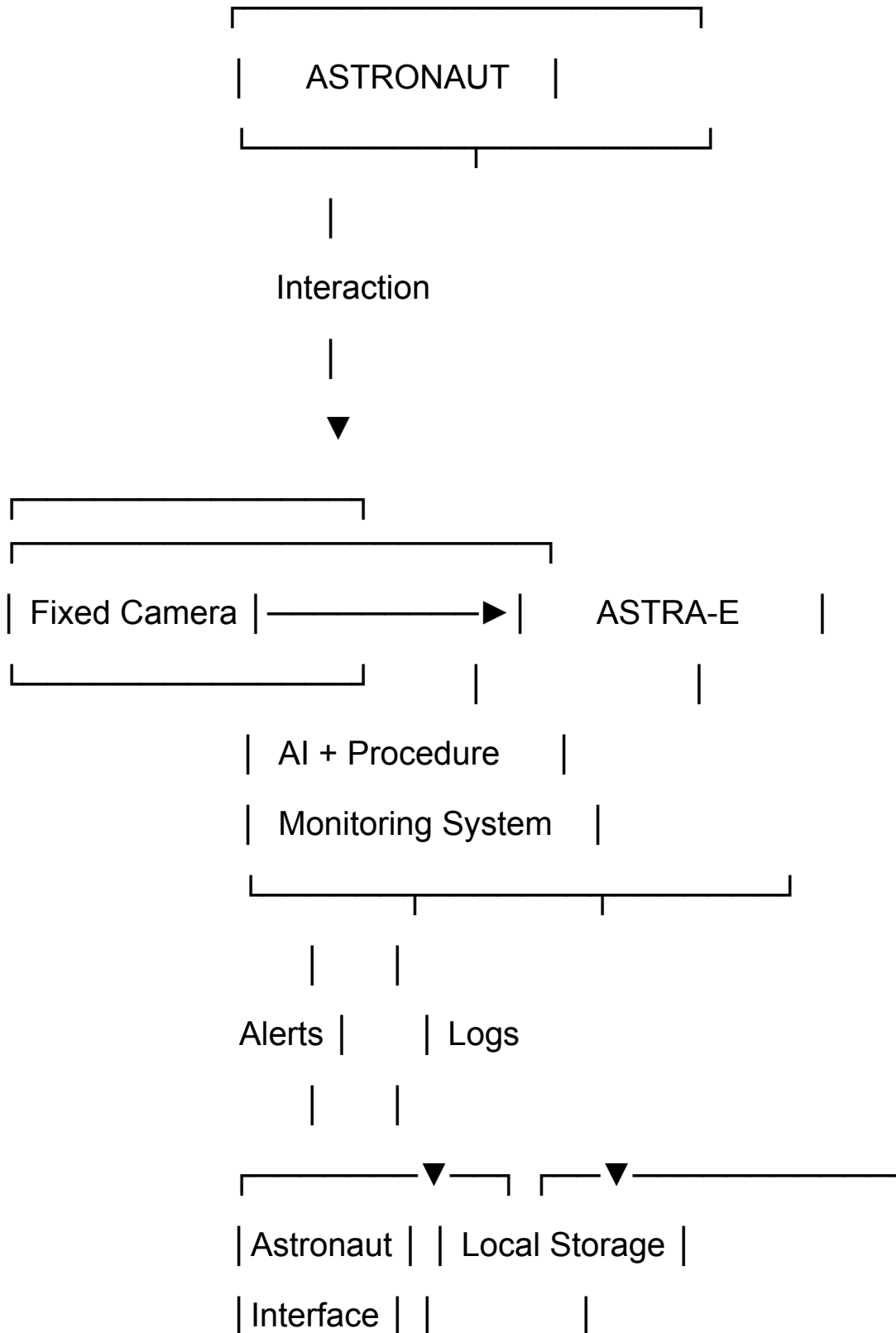


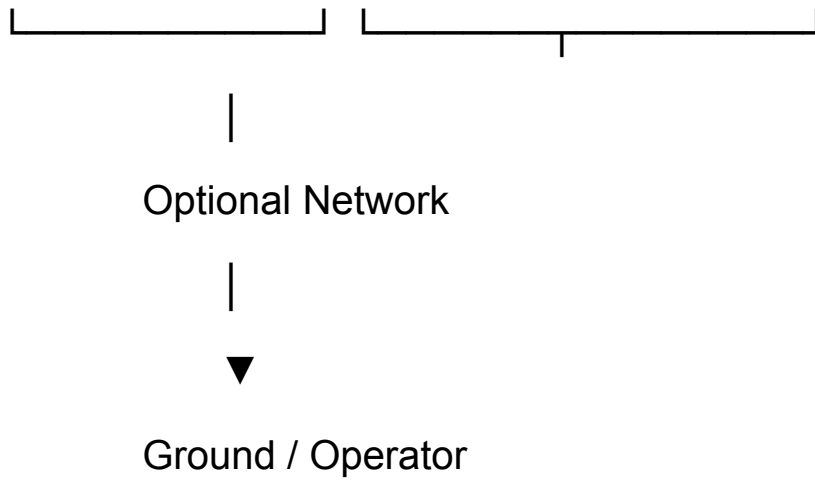
The system is still deployable as **one edge application**, but internally the components have clear boundaries.

---

## 4. System Context

At the highest level:





---

## 5. Logical Architecture

ASTRA-E will consist of **nine logical subsystems**.

ASTRA-E

- |
- | — 01. Camera & Video Subsystem
- |
- | — 02. Perception Subsystem
- |
- | — 03. Interaction Subsystem
- |
- | — 04. Temporal Activity Subsystem
- |
- | — 05. Procedure Reasoning Subsystem
- |

└─ 06. Assistance Subsystem

|

└─ 07. Event & Data Subsystem

|

└─ 08. Monitoring & Interface Subsystem

|

└─ 09. System Management Subsystem

---

## 6. Component 01 — Camera & Video Subsystem

### Responsibility

Responsible for all video ingestion.

Camera

|



Camera Manager

|

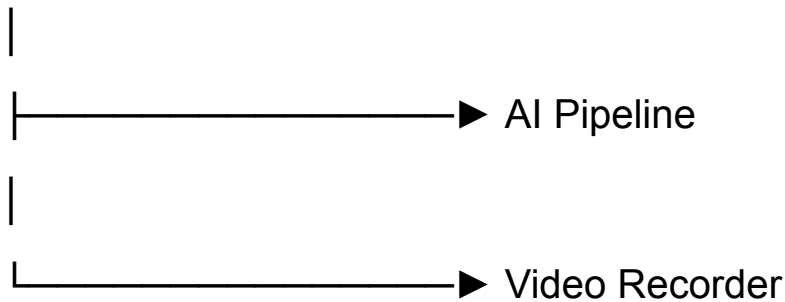


Frame Synchronizer

|



## Frame Buffer



## Responsibilities

- camera initialization;
- camera health monitoring;
- frame capture;
- timestamp generation;
- frame buffering;
- camera synchronization;
- recording;
- stream management.

## Does NOT do

- object detection;
- activity recognition;
- procedure reasoning.

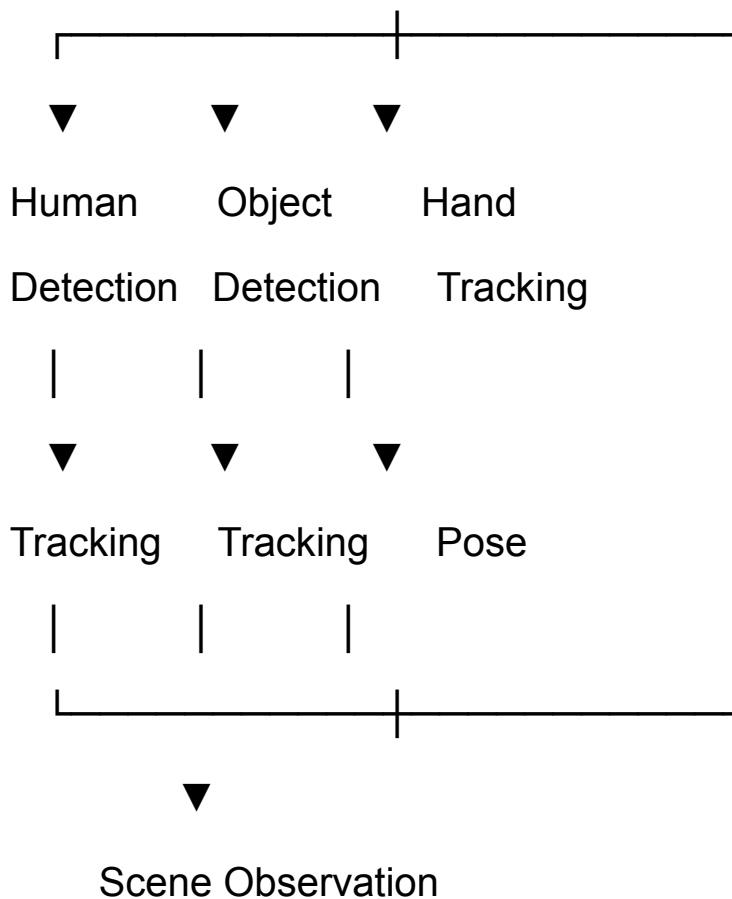
---

# 7. Component 02 — Perception Subsystem

The perception layer converts raw pixels into structured observations.

Video

|



## Output

The subsystem produces a standardized:

## SceneObservation

Conceptually:

SceneObservation

|

├── timestamp

├── humans[]

├── objects[]

├── hands[]

└── pose

|— camera\_id  
└— confidence metadata

The downstream system doesn't care whether the detector was YOLO, RT-DETR, etc.

---

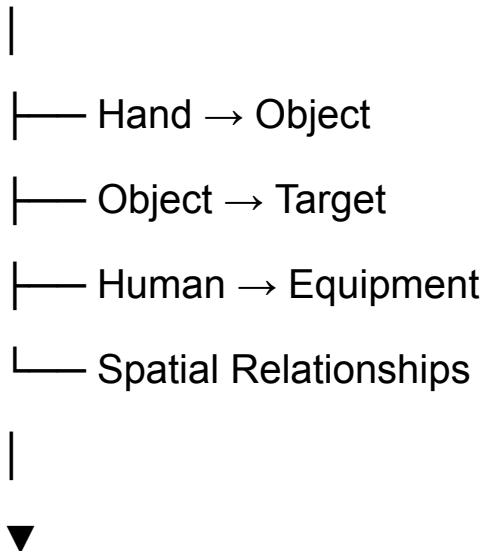
## 8. Component 03 — Interaction Subsystem

This converts raw perception into meaningful interactions.

SceneObservation



Interaction Analyzer



Interaction Events

Examples:

HAND\_APPROACH

GRASP

PICK

MOVE

PLACE

RELEASE

TOUCH

MANIPULATE

Output:

InteractionEvent

|

|— actor

|— object

|— interaction\_type

|— timestamp

|— spatial context

|— confidence

---

## 9. Component 04 — Temporal Activity Subsystem

This is our **AI activity-recognition layer**.

It receives a stream:

SceneObservation

+

InteractionEvent

+

Temporal History

and produces:

RecognizedAction

|

|— action\_type

|— confidence

|— start\_timestamp

|— end\_timestamp

|— evidence

Example:

{

"action": "PLACE\_YELLOW",

"confidence": 0.94,

"start": 152.2,

"end": 155.7

}

---

# 10. Critical Boundary

We should explicitly establish this interface:

PERCEPTION

|

| SceneObservation



INTERACTION

|

| InteractionEvent



TEMPORAL AI

|

| RecognizedAction



PROCEDURE ENGINE

Each stage has a clear responsibility.

This prevents our codebase from eventually becoming:

model.py



if prediction == ...



database



UI

which is exactly the architecture we want to avoid.

---

## 11. Component 05 — Procedure Reasoning Subsystem

This is the central **deterministic reasoning layer**.

RecognizedAction



Procedure Engine



├— Current State

├— Expected Action

├— Valid Transitions

├— Completion Conditions

└— Constraints



ProcedureDecision

Example:

Current:

S04 — PLACE\_RED

Observed:

PICK\_YELLOW

Expected:

PICK\_YELLOW



VALID TRANSITION



S05

---

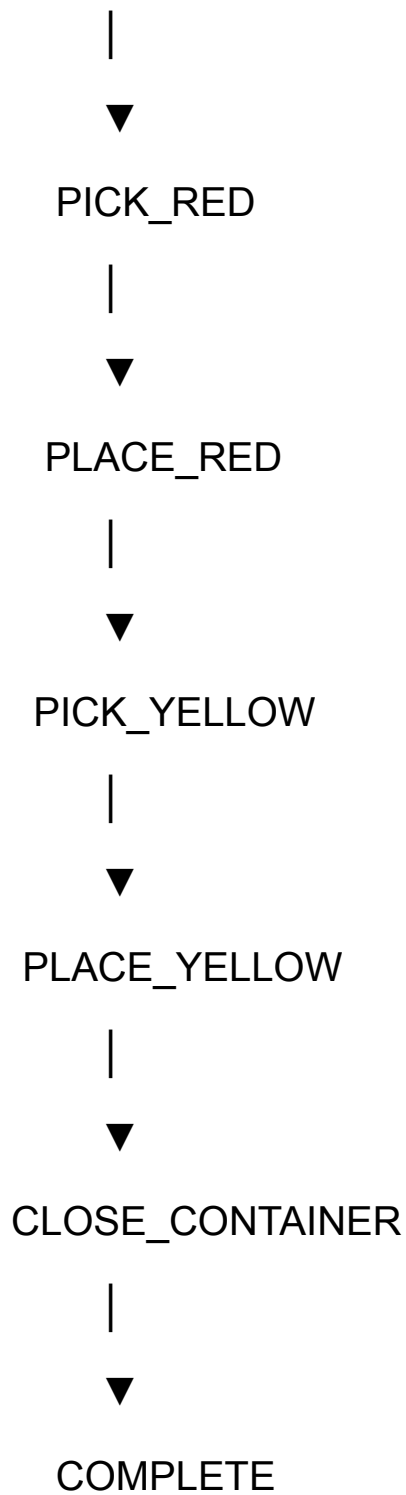
## 12. Procedure Graph

The procedure is represented as a graph:

START



OPEN\_CONTAINER



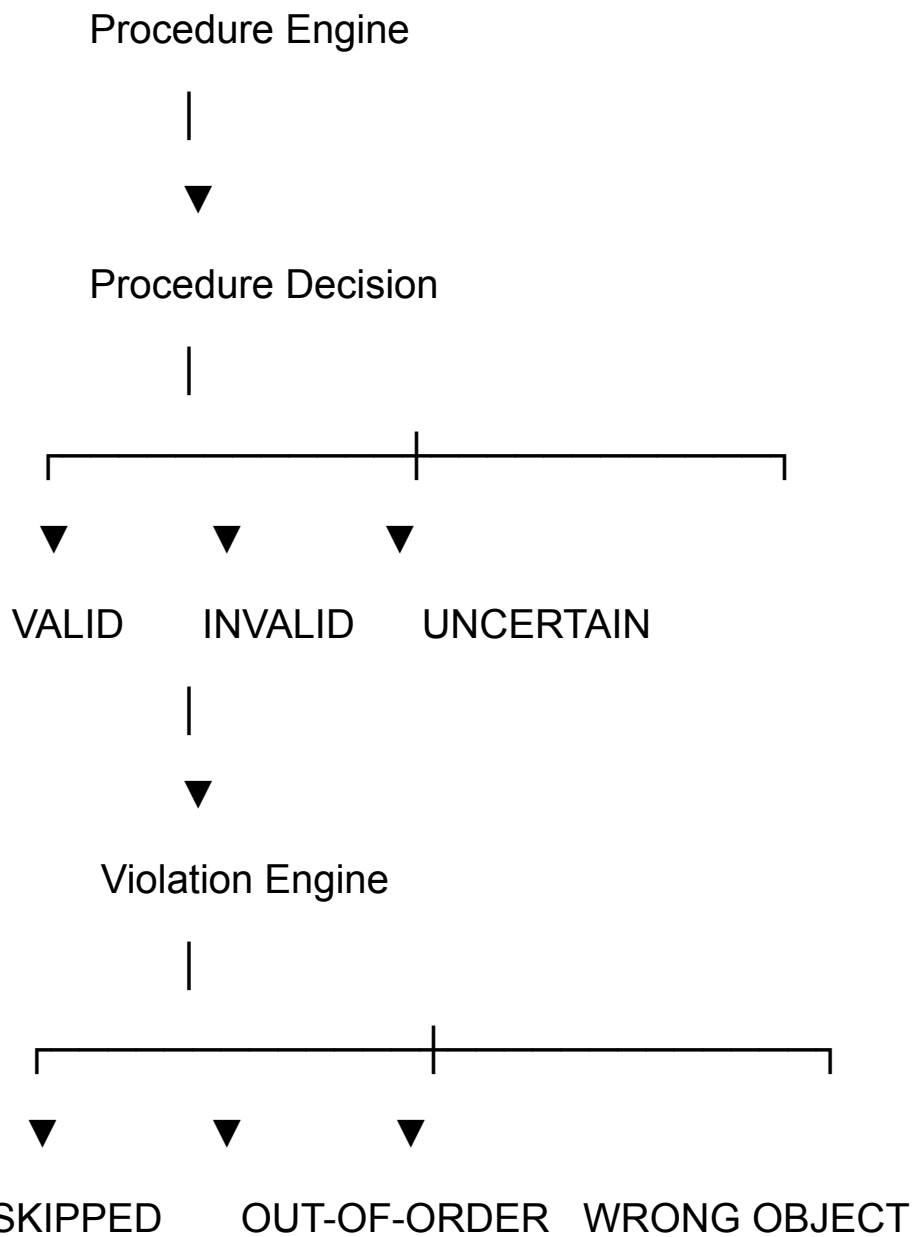
The engine should support:

- sequential transitions;
- optional steps;
- conditional branches;
- repetition where explicitly allowed;

- completion conditions;
  - failure conditions.
- 

## 13. Component 06 — Violation Engine

I recommend keeping this logically separate.



Potential violation types:

SKIPPED\_STEP

OUT\_OF\_ORDER

REPEATED\_STEP

INCORRECT\_OBJECT

INVALID\_ACTION

INCOMPLETE\_ACTION

---

## 14. Confidence Management

Confidence should be treated as a **first-class architectural concept**.

AI Prediction

|



Confidence Evaluation

|



Temporal Stabilization

|



Action Confirmation

|



Procedure Engine

The procedure engine should receive:

RecognizedAction

+

confidence

+

temporal evidence

not simply:

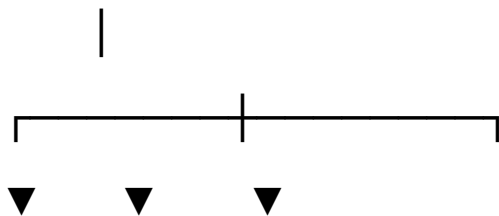
"PICK\_RED"

---

## 15. Uncertainty Handling

We need three distinct situations:

Observation



Valid Invalid Uncertain



Advance Alert Observe

**Uncertain  $\neq$  Incorrect.**

That distinction will significantly reduce false alarms.

---

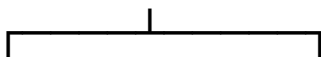
## 16. Component 07 — Assistance Subsystem

The Assistance subsystem consumes structured procedure events.

Procedure Event



Assistance Engine



GUI

TTS

Examples:

### Normal

“Next step: pick the yellow component.”

### Violation

“Warning. The yellow component placement has not been completed.”

### Complete

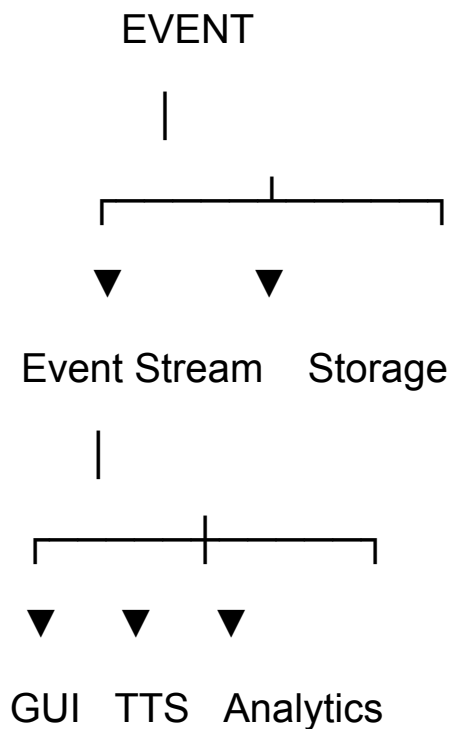
“Experiment completed successfully.”

The message generation layer should be deterministic for the core system.

---

## 17. Component 08 — Event & Data Subsystem

Everything important generates an event.



Events include:

CAMERA\_CONNECTED

ACTION\_DETECTED

ACTION\_CONFIRMED

PROCEDURE\_ADVANCED

VIOLATION\_DETECTED

ALERT\_ISSUED

EXPERIMENT\_STARTED

EXPERIMENT\_COMPLETED

SYSTEM\_ERROR

---

## 18. Persistence Architecture

We'll use two storage categories.

### Structured data

SQLite

|

|— Experiments

|— Procedures

|— Runs

|— Steps

|— Actions

|— Events

|— Violations

|— Metrics

### Binary data

File Storage

|

|— Videos

|— Camera 01

|— Camera 02

|— Event Clips

Do **not** put raw video inside SQLite.

---

## 19. Component 09 — Monitoring & Interface

The GUI is a consumer of the event stream rather than something directly coupled to AI models.

Event Stream

|



Monitoring Backend

|



WebSocket / API

|



React Dashboard

The dashboard receives:

Current State

Next Step

AI Confidence

Procedure Progress

Alerts

Camera Status

System Health

Event Timeline

---

## 20. System Management

A separate management component handles:

Configuration

|

├— Camera configuration

├— Experiment configuration

├— Model configuration

├— Storage configuration

└— Runtime configuration

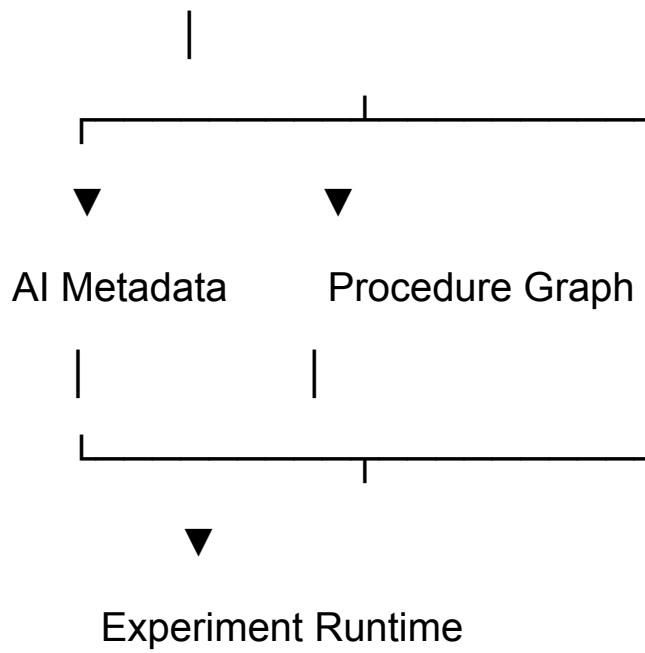
This allows us to change configuration without modifying core code.

---

## 21. Experiment Configuration

This should be a first-class architectural artifact.

Experiment Definition



Example:

experiment:

id: EXP-001

version: 1.0

objects:

- RED\_COMPONENT
- YELLOW\_COMPONENT
- CONTAINER

procedure:

- S01: OPEN\_CONTAINER
- S02: PICK\_RED
- S03: PLACE\_RED

- S04: PICK\_YELLOW
- S05: PLACE\_YELLOW
- S06: CLOSE\_CONTAINER

This makes ASTRA-E **experiment-configurable** rather than **experiment-hardcoded**.

---

## 22. End-to-End Data Flow

This is the most important HLD flow.

CAMERA

|

| VideoFrame



VIDEO INGESTION

|

| TimestampedFrame



PERCEPTION

|

| SceneObservation



INTERACTION

|

| InteractionEvent



TEMPORAL AI

|

| RecognizedAction



CONFIDENCE MANAGER

|

| ConfirmedAction



PROCEDURE ENGINE

|

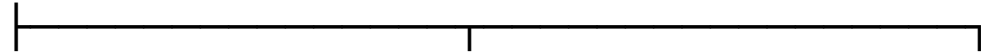
| ProcedureDecision



VIOLATION ENGINE

|

| ProcedureEvent



GUIDANCE

LOGGER

GUI

|

|



## 23. Real-Time Path vs Persistence Path

This distinction should be explicitly documented.

### Real-Time Path

Camera



Perception



Interaction



Temporal AI



Procedure



Alert



Astronaut

This path is latency-sensitive.

---

### Persistence Path

Observations

Actions

Events

Metrics

Video



Storage

This path is durability-focused.

The two should not block each other.

For example:

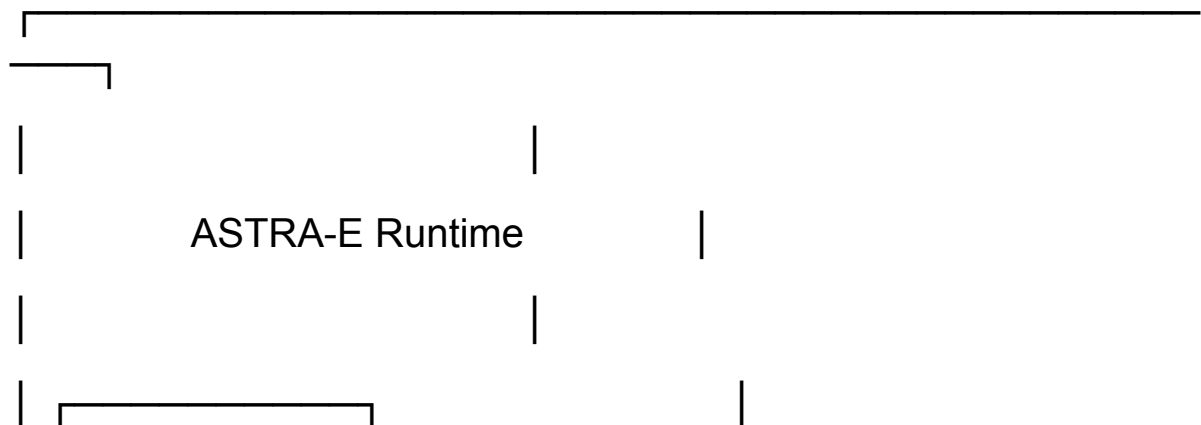
SQLite temporarily slows down → astronaut warning  
should **not** stop.

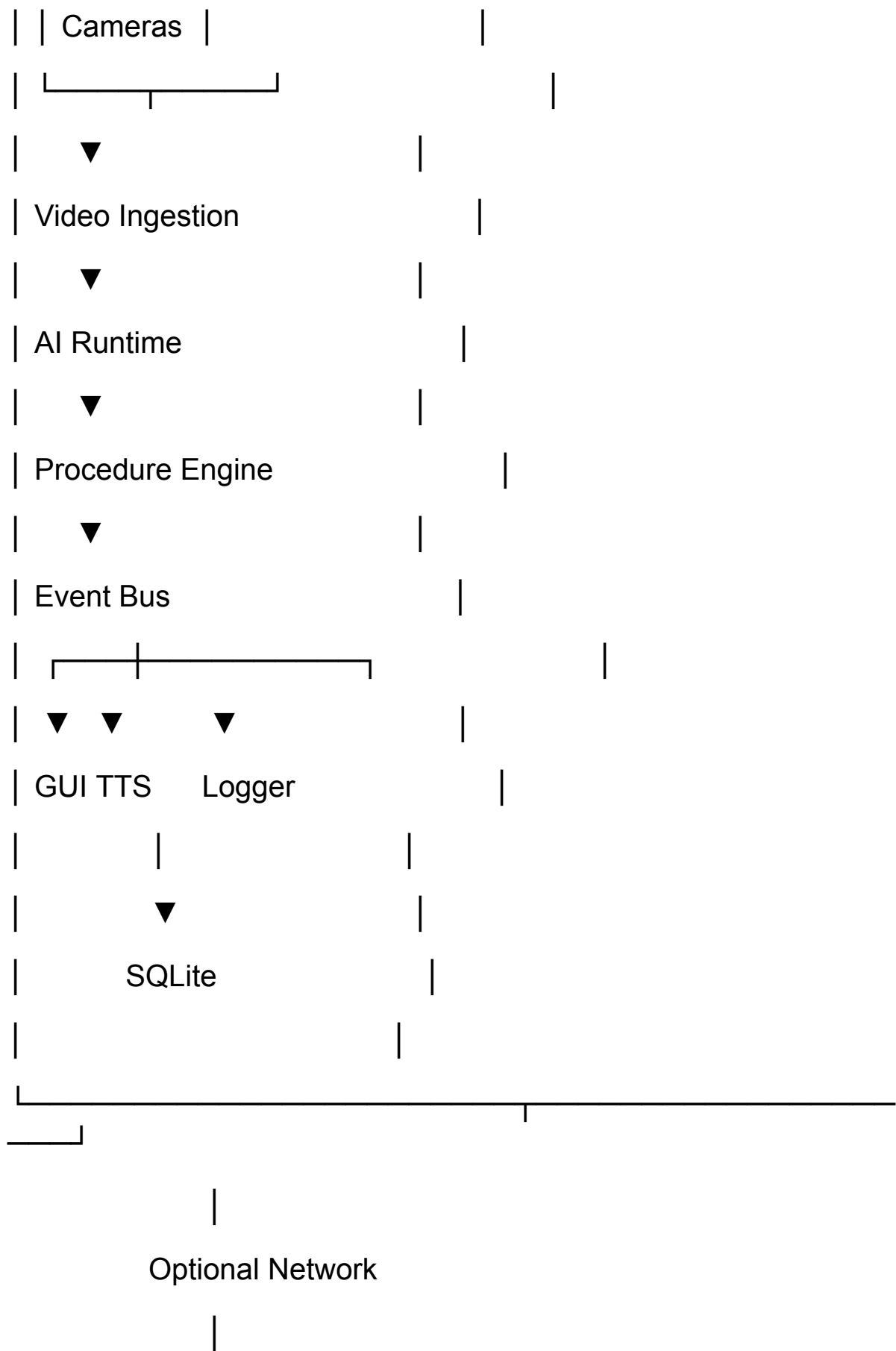
---

## 24. Deployment Architecture

For the SIH prototype:

EDGE COMPUTER







Ground / Operator

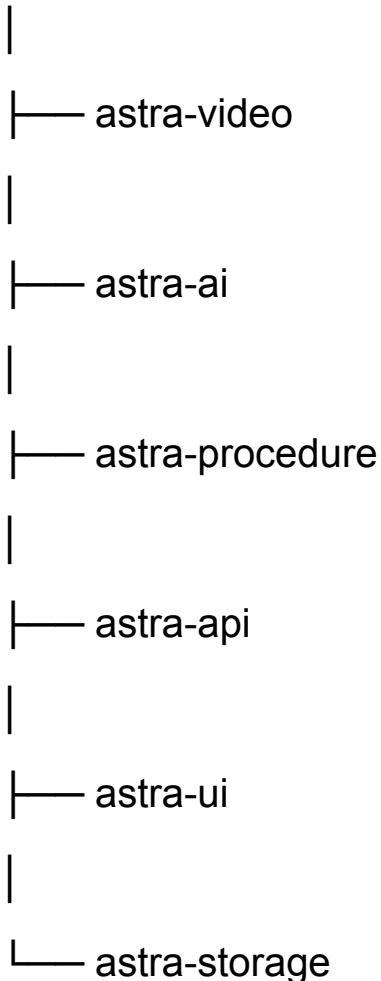
The **entire core system remains on the edge machine.**

---

## 25. Deployment Isolation

We can package the system as several processes/containers:

ASTRA-E



However, for the first prototype, I'd **not over-containerize it.**

Start with:

Python Runtime

└— Video

└— AI

└— Procedure

└— API

└— Storage

Then isolate components when performance/stability requires it.

---

## 26. Communication Model

Internally we have two communication styles.

### **Synchronous**

Used when a component needs an immediate result:

AI → Procedure

### **Event-driven**

Used for notifications:

Procedure

↓

Event

└— GUI

└— TTS

└— Logger

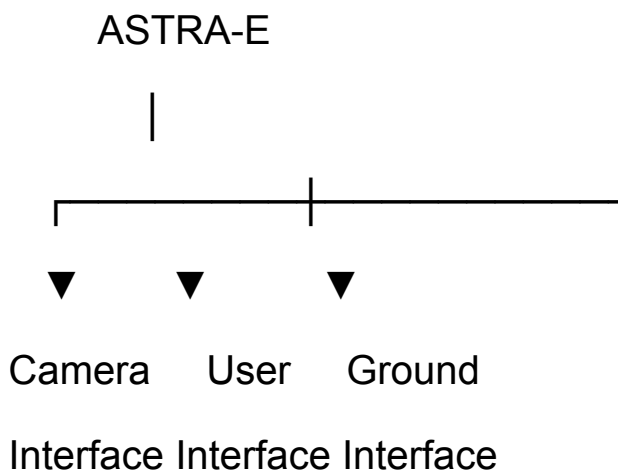
└─ Network

This avoids coupling the procedure engine to every consumer.

---

## 27. External Interfaces

At HLD level:



### Camera Interface

Input:

Video frames

### User Interface

Input:

Touch/mouse/operator commands

Output:

status

guidance

alerts

## Ground Interface

Optional:

experiment status

logs

events

video stream

health information

---

## 28. Fault Handling

We should design failure handling into the architecture.

### Camera failure

Camera Failure



Camera Manager



System Event



GUI + Logger

### AI failure

AI unavailable



Procedure state = PAUSED



Astronaut notification

**Network failure**

Network LOST



Core system continues



Local storage



Network restored



Synchronize queued data

This last one is particularly important.

---

## 29. Graceful Degradation

ASTRA-E should not have only:

WORKING

BROKEN

Instead:

FULL\_OPERATION

- |
- |— Reduced camera coverage
- |— AI degraded
- |— Network unavailable
- |— Storage degraded
- └— Procedure paused

Example:

If the network goes down:

AI      ✓

Procedure   ✓

TTS      ✓

GUI      ✓

Logging   ✓

Ground    ✗

The experiment assistant still functions.

---

## 30. Security Boundary

Although this is a prototype, the architecture should separate:

Control / Configuration

≠

Runtime AI

An operator shouldn't be able to accidentally alter the active procedure halfway through an experiment.

Therefore:

CONFIGURATION



Validated



VERSIONED



EXPERIMENT START



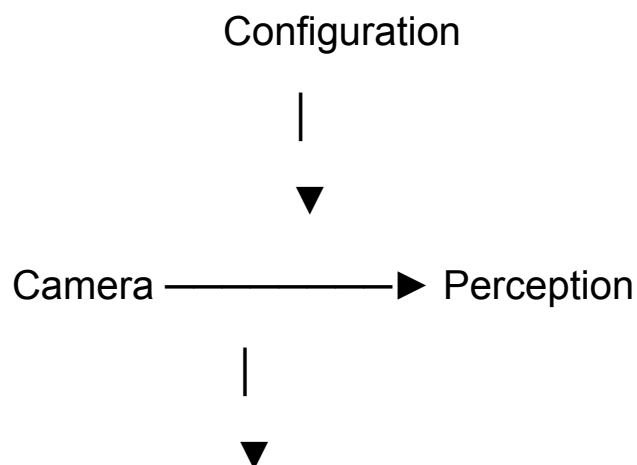
IMMUTABLE RUNTIME PROCEDURE

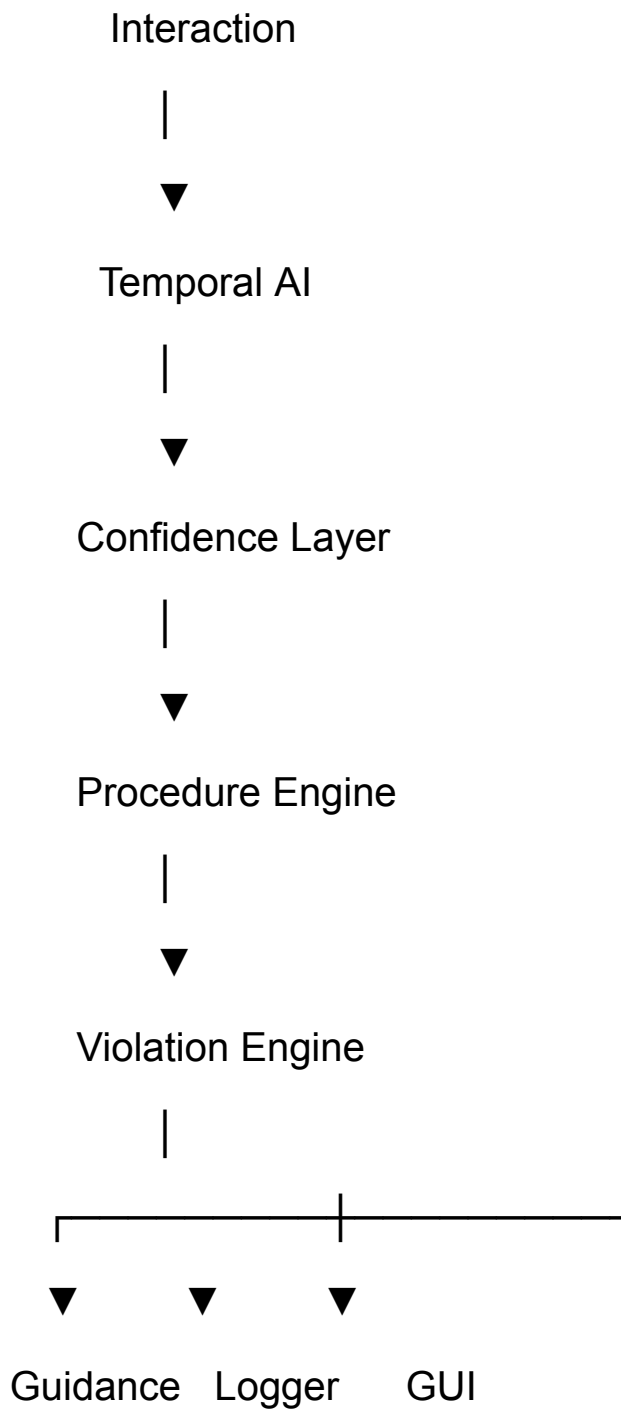
This is a subtle but important aerospace-style design principle.

---

## 31. HLD Module Dependency

The dependency direction should be:





Notice:

**GUI never calls the AI model directly.**

And:

**AI never modifies the database directly.**

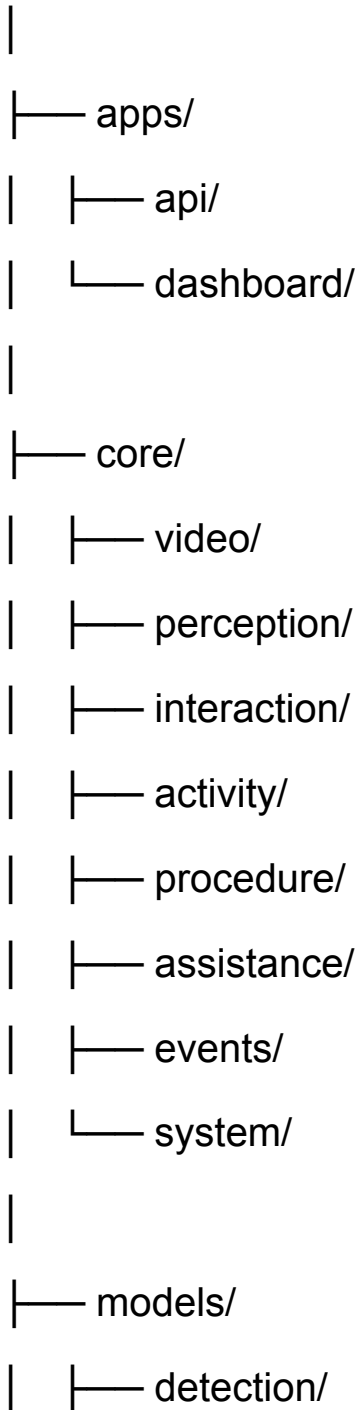
That's the kind of separation we want.

---

## 32. HLD Repository Mapping

Eventually our repository can mirror the architecture:

astra-e/



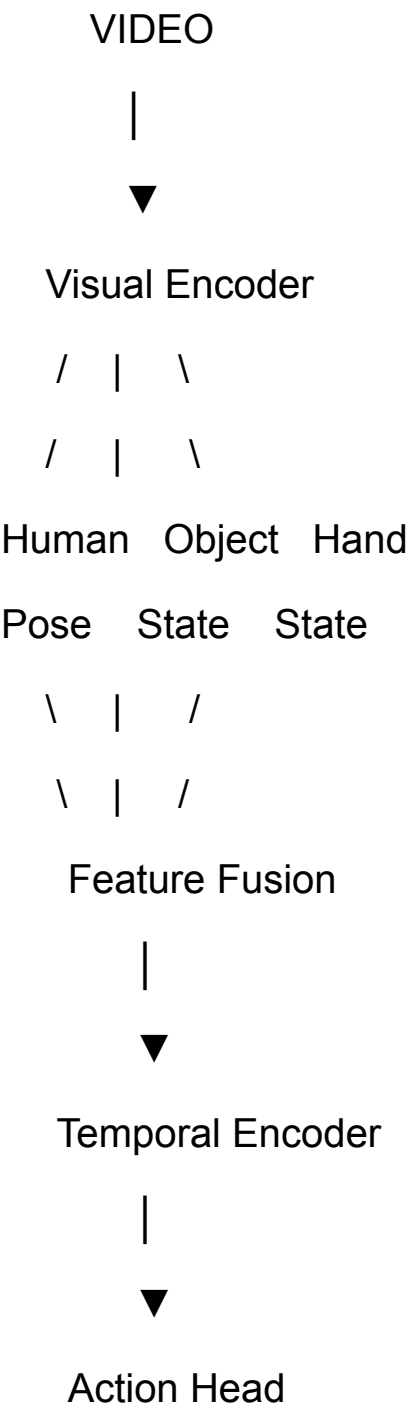
```
|   |— pose/
|   |— interaction/
|   └— activity/
|
|— experiments/
|   └— EXP001/
|
|— data/
|   |— raw/
|   |— processed/
|   └— annotations/
|
|— storage/
|
|— tests/
|
|— configs/
|
└— docs/
    |— SRS.md
    |— HLD.md
    └— LLD.md
```

This isn't the LLD yet—the purpose is simply to show how the logical architecture maps to a software boundary.

---

### 33. The AI Architecture

One thing we should explicitly document separately:



|



Action + Confidence

The actual models remain **TBD and benchmark-driven**.

This prevents us from prematurely locking ourselves into:

“We’re using LSTM because we decided that in September.”

We should let experiments determine that.

---

## 34. Microgravity Extension

The advanced architecture fits naturally:

Camera

|



3D Pose

|



Rack Coordinate Frame

|



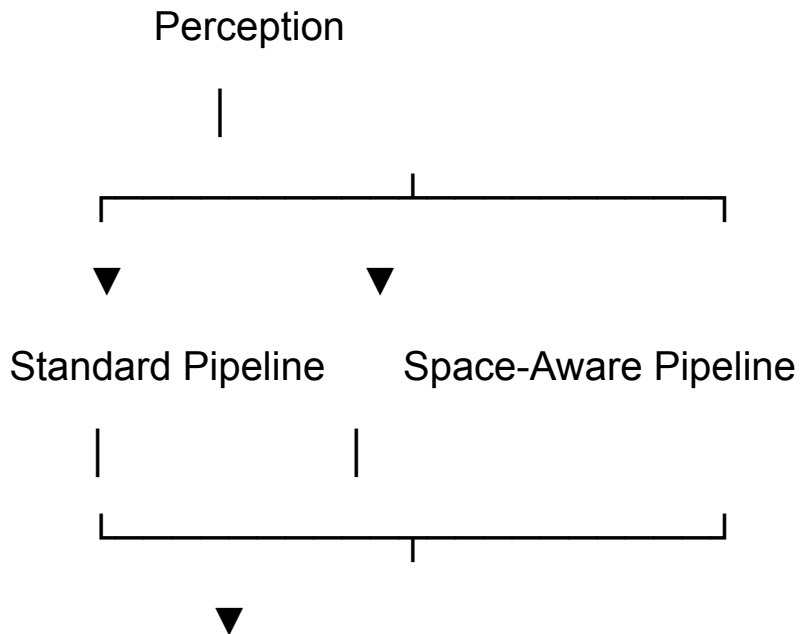
Orientation Normalization

|



## Temporal Activity Model

This can coexist with the normal pipeline:



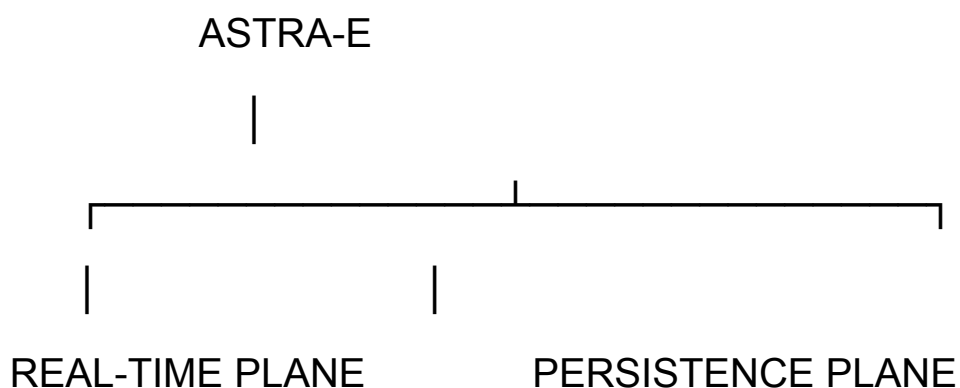
## Activity Engine

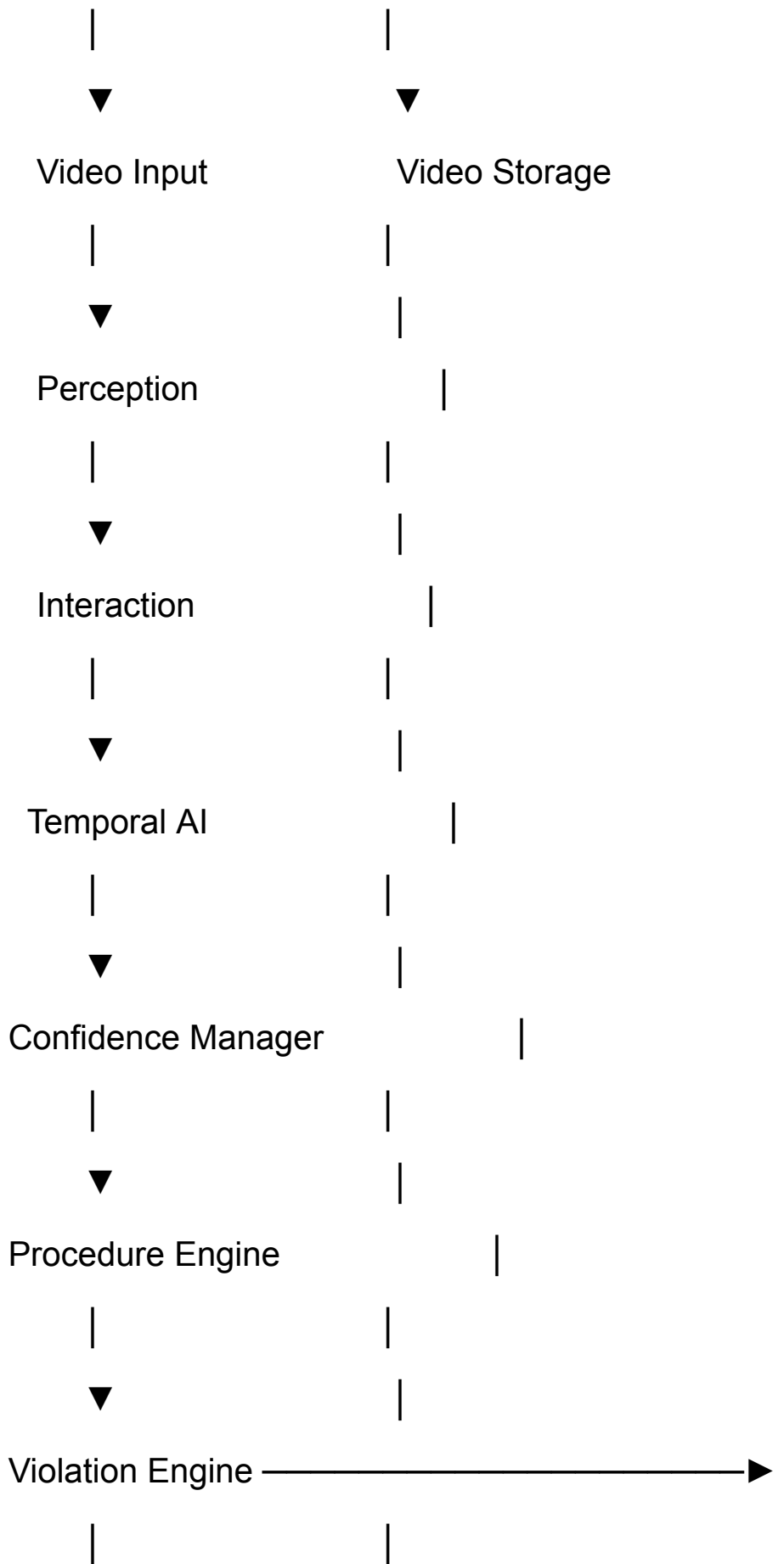
Therefore we aren't building the entire architecture around an optional feature.

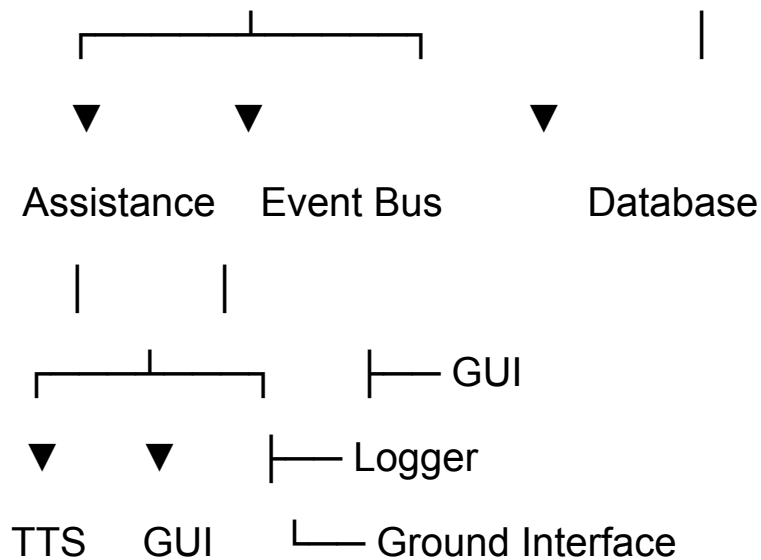
---

## 35. Final HLD

The architecture can now be summarized as:







## 36. Architecture Decisions We Can Now Freeze

| Area         | Decision             |
|--------------|----------------------|
| Architecture | Modular event-driven |
| Deployment   | Edge-first           |
| Runtime      | Offline capable      |
| Video        | Fixed cameras        |

|               |                                     |
|---------------|-------------------------------------|
| Perception    | Separate subsystem                  |
| Interaction   | Dedicated HOI subsystem             |
| HAR           | Dedicated temporal subsystem        |
| Reasoning     | Deterministic procedure engine      |
| Violations    | Separate violation engine           |
| Confidence    | First-class data                    |
| Communication | Events + synchronous internal calls |
| Storage       | SQLite + filesystem                 |
| GUI           | API/WebSocket consumer              |
| Voice         | Assistance subsystem                |

Experiment Configuration-driven  
s

AI models Pluggable

LLM Outside critical path

Microgravity Optional extension  
y

Ground link Optional, non-critical

---

 **HLD is now structurally complete.**

The **next step should be LLD**, but I'd make one intermediate artifact first:

### **ASTRA-E Architecture Contract**

We define the exact schemas flowing through the system:

VideoFrame



SceneObservation



InteractionEvent



ActionObservation



ConfirmedAction



ProcedureDecision



ViolationEvent



AssistanceEvent



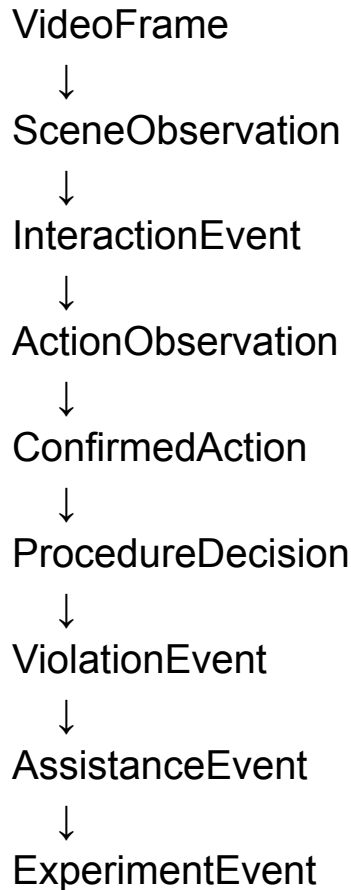
ExperimentLog

# ASTRA-E — Data Contracts

DCD v1.0

## 1. Contract Philosophy

Our fundamental pipeline is:



There are three rules:

### **Rule 1 — Every object is timestamped**

Anything related to the experiment must be traceable to a point or interval in time.

### **Rule 2 — AI outputs are probabilistic**

AI-derived objects must contain confidence/evidence.

### **Rule 3 — Decisions are deterministic**

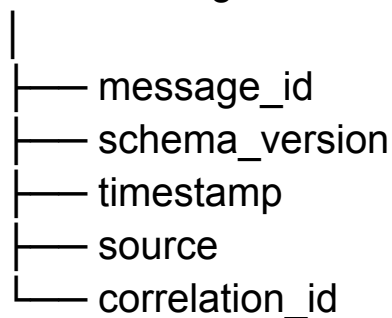
The Procedure Engine consumes AI observations but owns the decision about whether the experiment is procedurally correct.

---

## 2. Global Contract

Every message in the runtime should carry common metadata.

BaseMessage



Conceptually:

```
{
  "message_id": "01JXYZ...",
  "schema_version": "1.0",
  "timestamp": 1757060000.421,
  "source": "activity-engine",
  "correlation_id": "RUN-2026-001"
}
```

Why **correlation\_id**?

One experiment run might generate thousands of events.

We need to be able to trace:

Camera frame



Detection



Action

↓  
Procedure decision  
↓  
Violation  
↓  
Alert

back to the same experiment execution.

---

### 3. Contract 01 — VideoFrame

**Producer:** Video Acquisition

**Consumer:** Perception

VideoFrame

**Schema**

```
{  
  "message_id": "frame-000001",  
  "schema_version": "1.0",  
  
  "timestamp": 172.421,  
  
  "source": "camera-01",  
  
  "correlation_id": "RUN-001",  
  
  "frame_id": 18421,  
  
  "camera_id": "CAM-01",  
  
  "width": 1920,  
  "height": 1080,
```

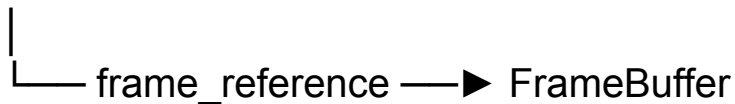
```
"format": "BGR",  
  
"frame_reference": "memory://frame/18421"  
}
```

### Important decision

The frame itself should **not normally travel through JSON**.

The contract carries a reference to the actual frame in memory/shared buffer.

Contract



This prevents massive serialization overhead.

---

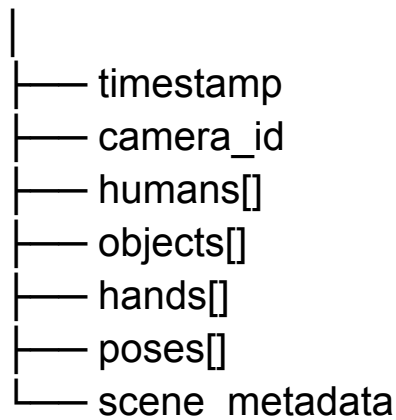
## 4. Contract 02 — SceneObservation

**Producer:** Perception

**Consumer:** Interaction + Activity

This is one of the most important contracts.

SceneObservation



Example:

```
{
  "message_id": "obs-001",
  "schema_version": "1.0",

  "timestamp": 172.45,

  "source": "perception-engine",
  "correlation_id": "RUN-001",

  "camera_id": "CAM-01",

  "humans": [
    {
      "id": "human-01",
      "bbox": [300, 100, 900, 900],
      "confidence": 0.98
    }
  ],

  "objects": [
    {
      "id": "obj-red",
      "type": "RED_COMPONENT",
      "bbox": [950, 500, 1020, 570],
      "confidence": 0.96
    }
  ],

  "hands": [
    {
      "id": "hand-right",
      "owner_id": "human-01",
      "position": [920, 510],
      "confidence": 0.91
    }
  ]
}
```

```
]
}
```

---

## 5. Entity IDs

A critical design rule:

**Objects must have stable tracking IDs.**

Bad:

```
frame 100 → RED_OBJECT
frame 101 → RED_OBJECT
frame 102 → RED_OBJECT
```

Good:

```
frame 100 → obj-red-17
frame 101 → obj-red-17
frame 102 → obj-red-17
```

This lets us reason about:

- object trajectory
- object ownership
- object manipulation
- object placement

across time.

---

## 6. Contract 03 — InteractionEvent

**Producer:** Interaction Engine

**Consumer:** Activity Engine + Event Bus

## InteractionEvent

Possible interaction types:

APPROACH

TOUCH

GRASP

PICK

MOVE

PLACE

RELEASE

MANIPULATE

Example:

```
{  
  "message_id": "interaction-42",  
  "schema_version": "1.0",  
  
  "timestamp": 175.82,  
  
  "source": "interaction-engine",  
  "correlation_id": "RUN-001",  
  
  "interaction_type": "GRASP",  
  
  "actor_id": "human-01",  
  "object_id": "obj-red",  
  
  "confidence": 0.93,  
  
  "evidence": {  
    "hand_distance": 0.03,  
    "relative_motion": 0.81  
  }  
}
```

---

## 7. Contract 04 — ActionObservation

This is the **temporal AI output**.

We distinguish it from a confirmed action.

That's important.

ActionObservation

means:

"The AI currently believes this action is occurring."

It does **not** mean:

"The astronaut definitely performed this valid procedure step."

Example:

```
{  
  "message_id": "action-127",  
  
  "timestamp": 180.24,  
  
  "source": "activity-engine",  
  
  "correlation_id": "RUN-001",  
  
  "action": "PLACE",  
  
  "object_id": "obj-red",  
  
  "target_id": "target-A",
```

```
"confidence": 0.91,  
  
"temporal_window": {  
  "start": 178.4,  
  "end": 180.2  
}  
}
```

---

## 8. Contract 05 — ConfirmedAction

This is generated after confidence/temporal validation.

ActionObservation



Confidence Manager



ConfirmedAction

Example:

```
{  
  "message_id": "confirmed-127",  
  
  "timestamp": 180.31,  
  
  "source": "confidence-manager",  
  
  "correlation_id": "RUN-001",  
  
  "action": "PLACE",  
  
  "object_id": "obj-red",
```

```
"target_id": "target-A",

"confidence": 0.94,

"confirmation": {
  "stable_frames": 12,
  "minimum_confidence": 0.85,
  "temporal_consistency": 0.92
}
}
```

This gives us a very useful distinction:

AI prediction

≠

confirmed observation

---

## 9. Contract 06 — ProcedureDefinition

Unlike the previous contracts, this is **configuration data**, not an event.

It defines what the experiment is supposed to do.

ProcedureDefinition

```
|
|— experiment_id
|— version
|— objects
|— targets
|— states
|— transitions
```

Example:

```
{
```

```
"experiment_id": "EXP-001",
"version": "1.0",

"states": [
  {
    "id": "S01",
    "action": "OPEN_CONTAINER"
  },
  {
    "id": "S02",
    "action": "PICK",
    "object": "RED_COMPONENT"
  },
  {
    "id": "S03",
    "action": "PLACE",
    "object": "RED_COMPONENT",
    "target": "TARGET_A"
  }
]
}
```

---

## 10. Procedure State

Runtime state is different from the definition.

ProcedureRuntimeState

Example:

```
{
  "experiment_id": "EXP-001",
  "procedure_version": "1.0",

  "run_id": "RUN-001",
```

```
"current_state": "S03",

"completed_states": [
  "S01",
  "S02"
],

"status": "RUNNING",

"last_action": "PICK_RED"
}
```

---

## 11. Contract 07 — ProcedureDecision

**Producer:** Procedure Engine

**Consumer:** Violation + Assistance + Logger

ProcedureDecision

Example:

```
{
  "message_id": "decision-87",

  "timestamp": 180.42,

  "source": "procedure-engine",

  "correlation_id": "RUN-001",

  "current_state": "S03",

  "observed_action": {
    "action": "PLACE",
```

```
"object": "RED_COMPONENT",  
"target": "TARGET_A"  
},  
  
"decision": "VALID",  
  
"next_state": "S04"  
}
```

Allowed decisions:

VALID  
INVALID  
UNCERTAIN  
IGNORED

---

## 12. Contract 08 — ViolationEvent

Only the procedural reasoning layer should be able to generate a **procedural violation**.

ViolationEvent

Example:

```
{  
  "message_id": "violation-13",  
  
  "timestamp": 210.52,  
  
  "source": "violation-engine",  
  
  "correlation_id": "RUN-001",  
  
  "violation_type": "OUT_OF_ORDER",  
}
```

```
"expected": {  
  "action": "PICK",  
  "object": "YELLOW_COMPONENT"  
},  
  
"observed": {  
  "action": "CLOSE_CONTAINER"  
},  
  
"severity": "WARNING"  
}
```

---

## 13. Violation Types

We should standardize these early.

```
SKIPPED_STEP  
OUT_OF_ORDER  
WRONG_OBJECT  
WRONG_TARGET  
UNAUTHORIZED_ACTION  
INCOMPLETE_ACTION  
REPEATED_ACTION
```

And severity:

```
INFO  
WARNING  
CRITICAL
```

We should **not** automatically make every violation **CRITICAL**.

---

## 14. Contract 09 — AssistanceEvent

This translates system state into astronaut-facing assistance.

```
{  
  "message_id": "assist-77",  
  
  "timestamp": 211.01,  
  
  "source": "assistance-engine",  
  
  "correlation_id": "RUN-001",  
  
  "type": "PROCEDURE_WARNING",  
  
  "priority": "HIGH",  
  
  "message": "Please complete the yellow component placement.",  
  
  "channels": [  
    "GUI",  
    "TTS"  
  ]  
}
```

Channels:

GUI  
TTS  
VISUAL\_ALERT  
GROUND\_INTERFACE

---

## 15. Contract 10 — ExperimentEvent

This is our generic event envelope for the event bus.

ExperimentEvent

It can represent:

EXPERIMENT\_STARTED  
EXPERIMENT\_PAUSED  
EXPERIMENT\_COMPLETED  
ACTION\_CONFIRMED  
VIOLATION\_DETECTED  
ALERT\_ISSUED  
SYSTEM\_ERROR

Example:

```
{  
  "event_id": "event-9001",  
  
  "event_type": "ACTION_CONFIRMED",  
  
  "timestamp": 211.12,  
  
  "run_id": "RUN-001",  
  
  "source": "activity-engine",  
  
  "payload": {  
    "action": "PLACE_YELLOW",  
    "confidence": 0.94  
  }  
}
```

---

## 16. Contract 11 — SystemHealth

This is separate from experiment data.

SystemHealth

Example:

```
{
  "timestamp": 215.2,

  "system": "ASTRA-E",

  "status": "OPERATIONAL",

  "components": {
    "camera": "HEALTHY",
    "perception": "HEALTHY",
    "activity": "HEALTHY",
    "procedure": "HEALTHY",
    "storage": "HEALTHY",
    "network": "DEGRADED"
  },

  "metrics": {
    "fps": 29.8,
    "latency_ms": 142,
    "cpu_percent": 54,
    "memory_percent": 61
  }
}
```

---

## 17. Contract 12 — ExperimentRun

This identifies one complete execution.

ExperimentRun

Example:

```
{  
  "run_id": "RUN-2026-0001",  
  
  "experiment_id": "EXP-001",  
  
  "procedure_version": "1.0",  
  
  "started_at": 1757060000,  
  
  "status": "RUNNING"  
}
```

Lifecycle:

CREATED



INITIALIZING



RUNNING



PAUSED



RUNNING



COMPLETED

Or:

RUNNING

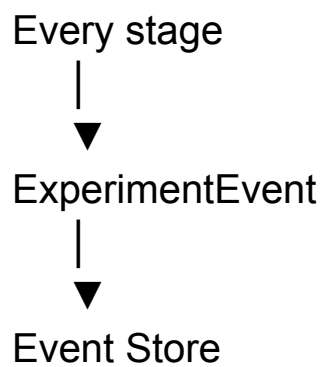
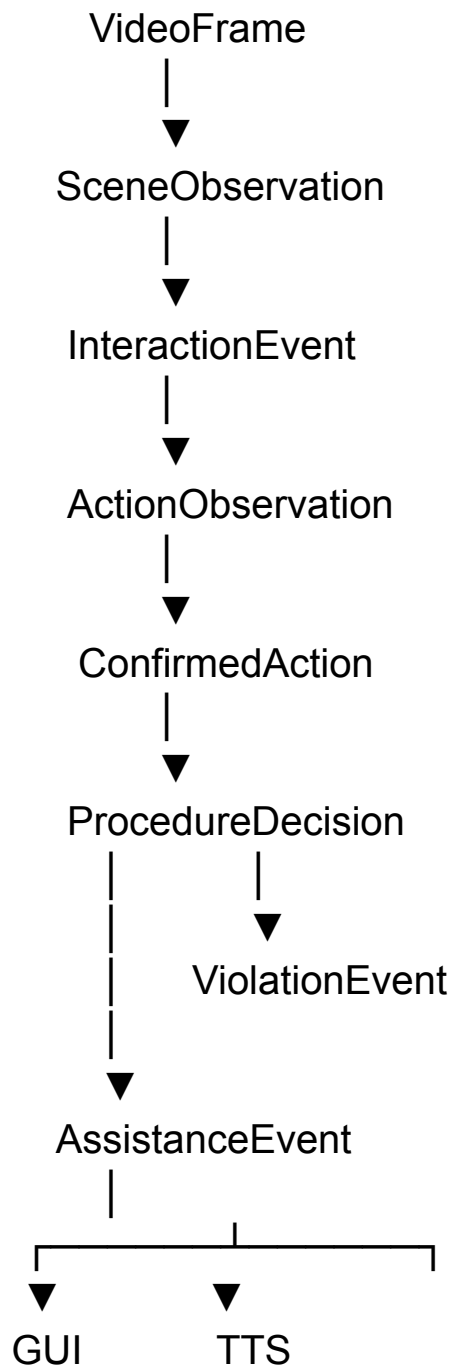


ABORTED

---

## 18. Contract Relationship

Now the architecture becomes very clean:

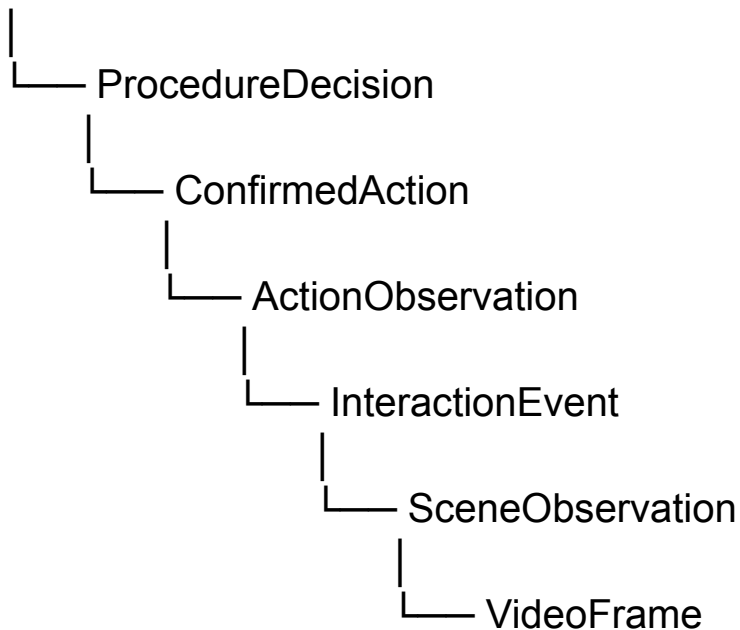


## 19. Traceability Chain

This is one of the most important properties of the system.

Every final decision should be traceable:

Violation



So if someone asks:

**"Why did ASTRA-E say the astronaut performed the wrong action?"**

we can trace the entire chain.

That's excellent for debugging, evaluation, and eventually certification-oriented thinking.

---

## 20. Confidence Semantics

We should standardize confidence.

Every AI-derived confidence:

$c \in [0, 1]$

But importantly:

**Confidence is not probability unless the model is calibrated.**

Therefore the contract should call it:

confidence

not:

probability

Later we can calibrate it using:

- temperature scaling;
  - isotonic regression;
  - reliability diagrams.
- 

## 21. Time Semantics

We need two timestamps.

**timestamp**

When the event was produced.

**event\_time**

When the actual observed action occurred.

Because AI inference has latency.

Example:

Actual event:

12.40 s

AI finishes inference:  
12.57 s

So:

```
{  
  "event_time": 12.40,  
  "timestamp": 12.57  
}
```

This distinction will be extremely useful.

---

## 22. IDs

We'll use different IDs for different entities.

run\_id  
experiment\_id  
procedure\_id  
camera\_id  
frame\_id  
human\_id  
object\_id  
interaction\_id  
action\_id  
event\_id

Never use one generic integer ID for everything.

---

## 23. Versioning

Every persistent contract should contain:

schema\_version

Example:

"schema\_version": "1.0"

Later:

1.0

1.1

2.0

### **Rule**

Minor version:

Backward compatible.

Major version:

Breaking change.

This will save us pain once multiple modules depend on the contracts.

---

## **24. Error Contract**

Every subsystem needs a standard error representation.

```
{  
  "error_id": "ERR-1001",  
  
  "timestamp": 219.3,  
  
  "source": "perception-engine",  
  
  "error_code": "MODEL_INFERENCE_TIMEOUT",
```

```
"severity": "ERROR",

"recoverable": true,

"message": "Inference exceeded configured latency threshold."
}
```

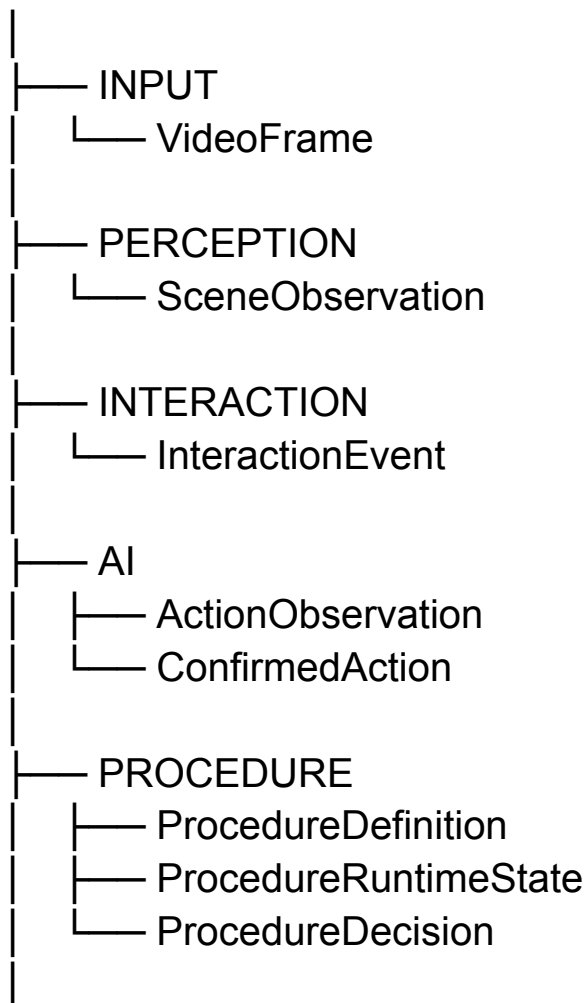
Error codes should eventually become an enum.

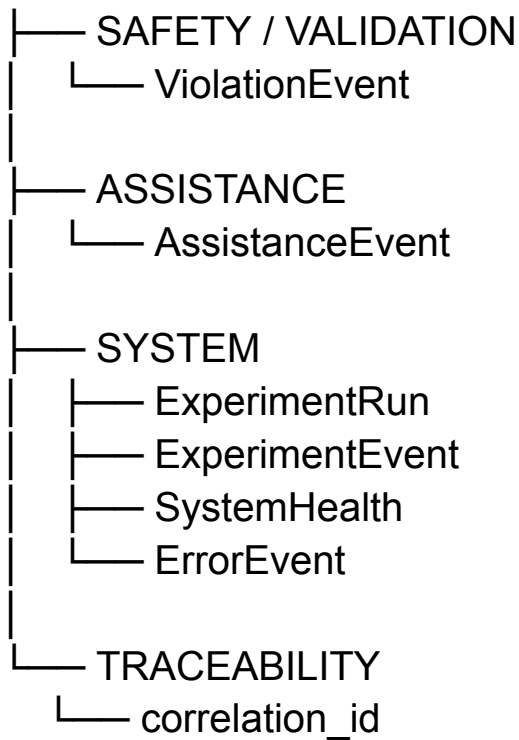
---

## 25. Contract Categories

So our data architecture now looks like:

### ASTRA-E DATA CONTRACTS





## 26. One Important Architectural Refinement

I'd make one change to our earlier HLD.

Instead of:

Temporal AI



Procedure

we should formally have:

Temporal AI



ActionObservation



Action Confirmation / Evidence Aggregation



ConfirmedAction

↓  
Procedure Engine

So the final architecture becomes:

Camera

↓  
VideoFrame

↓  
Perception

↓  
SceneObservation

↓  
Interaction

↓  
InteractionEvent

↓  
Temporal AI

↓  
ActionObservation

↓  
Confidence + Temporal Aggregation

↓  
ConfirmedAction

↓  
Procedure Engine

↓  
ProcedureDecision

↓  
Violation Engine

↓  
ViolationEvent

↓  
Assistance Engine

↓  
AssistanceEvent

↓  
GUI / TTS

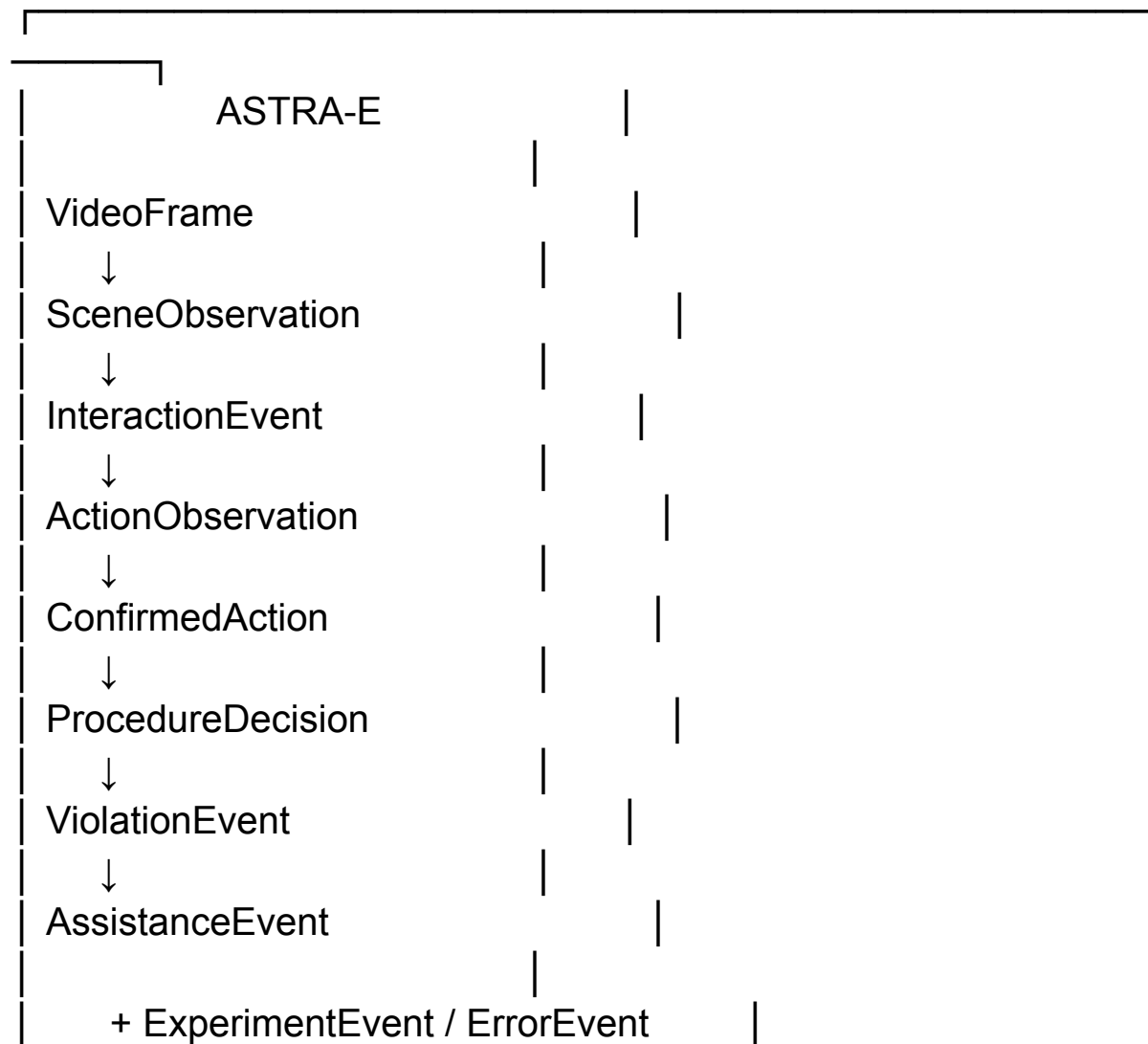
And **every significant object** is simultaneously published to the event/logging system.

---

## 27. The Contract Boundary We Should Freeze

This is the boundary I'd consider **v1.0 frozen**:

---



---

This gives us a very powerful property:

**We can replace the AI without changing the procedure system.**

For example:

LSTM



ActionObservation

can later become:

Transformer



ActionObservation

The Procedure Engine doesn't care.

Likewise:

YOLO



SceneObservation

can become:

RT-DETR



SceneObservation

without affecting downstream modules.

## **ASTRA-E — Dataset & ML Architecture**

**ML Architecture v1.0**

The core principle should be:

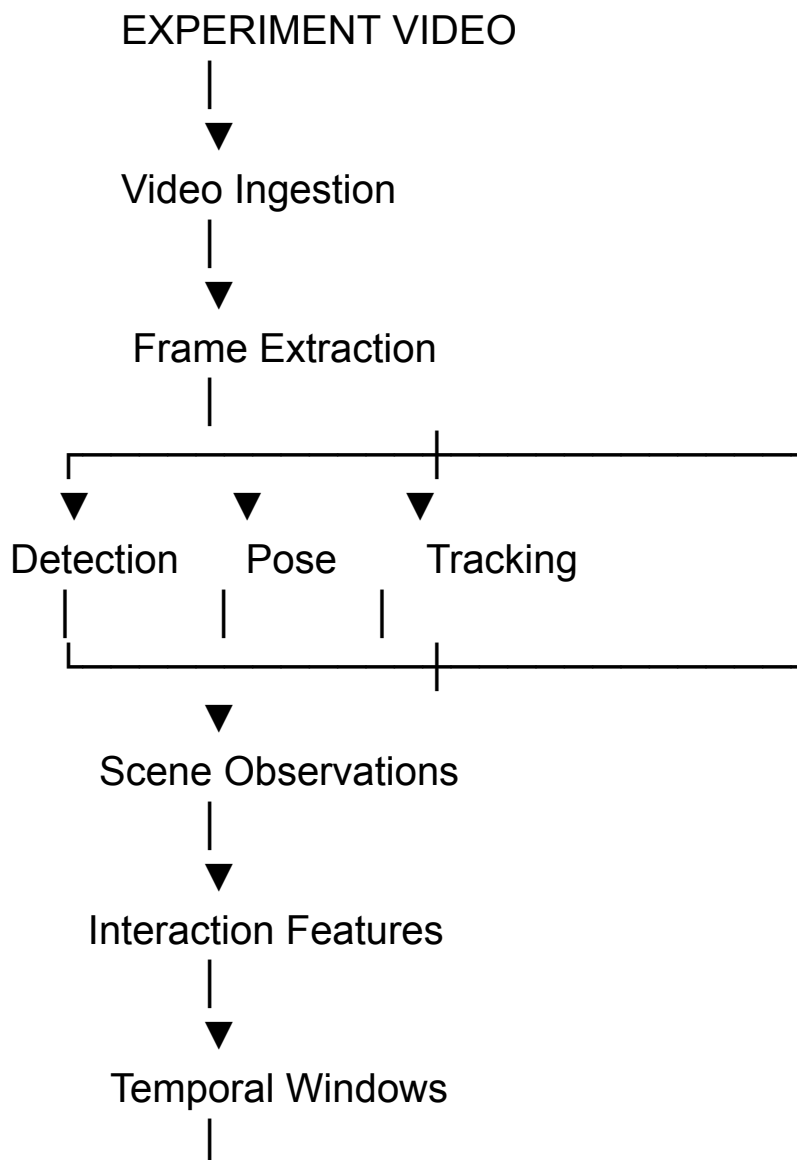
**We don't train a model to "understand the experiment." We train perception/temporal models to produce structured observations; the deterministic procedure engine remains responsible for procedural correctness.**

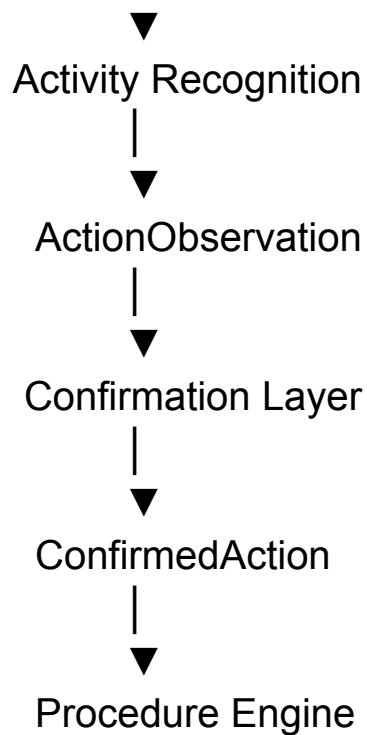
That distinction is important for both reliability and explainability.

---

## 1. ML Pipeline

Our complete ML pipeline should look like:





Notice that the ML portion ends at:

ConfirmedAction

The ML system doesn't directly produce:

"VIOLATION"

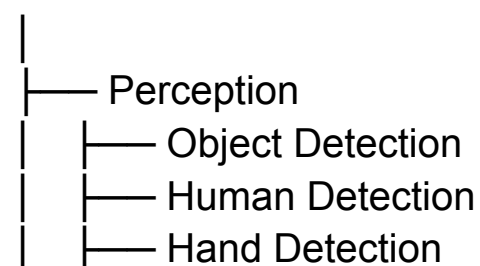
The procedure system does.

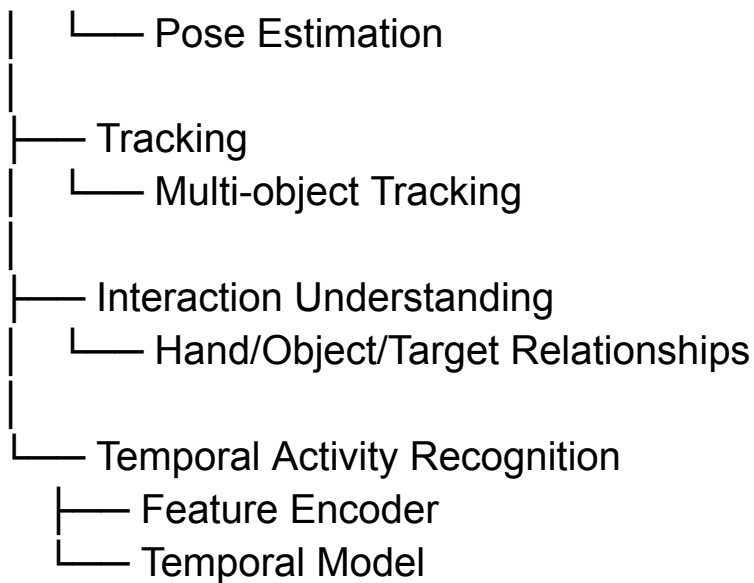
---

## 2. ML Subsystems

I'd divide the ML stack into four major systems.

ASTRA-E ML





Not every one of these needs a custom neural network.

In fact, **we shouldn't train everything ourselves.**

---

### 3. Model Responsibility Matrix

| Problem              | Initial approach           | ML?            |
|----------------------|----------------------------|----------------|
| Human detection      | pretrained detector        | ✓              |
| Object detection     | fine-tuned detector        | ✓              |
| Hand detection       | pretrained detector/pose   | ✓              |
| Pose                 | pretrained pose model      | ✓              |
| Object tracking      | tracker                    | ⚙️             |
| Hand-object relation | geometric + temporal rules | ✗<br>initially |

|                      |                        |   |
|----------------------|------------------------|---|
| Pick/place detection | temporal model + rules | ✓ |
| Procedure validation | state machine          | ✗ |
| Violation detection  | deterministic rules    | ✗ |
| Assistance           | deterministic          | ✗ |

This keeps the ML scope manageable.

---

## 4. Dataset Architecture

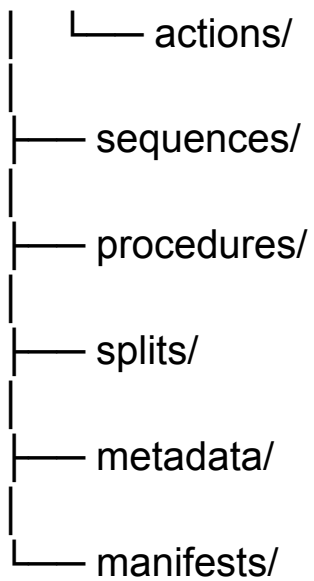
We should **not** make the dataset simply:

images/  
labels/

ASTRA-E is fundamentally a **video + temporal event** problem.

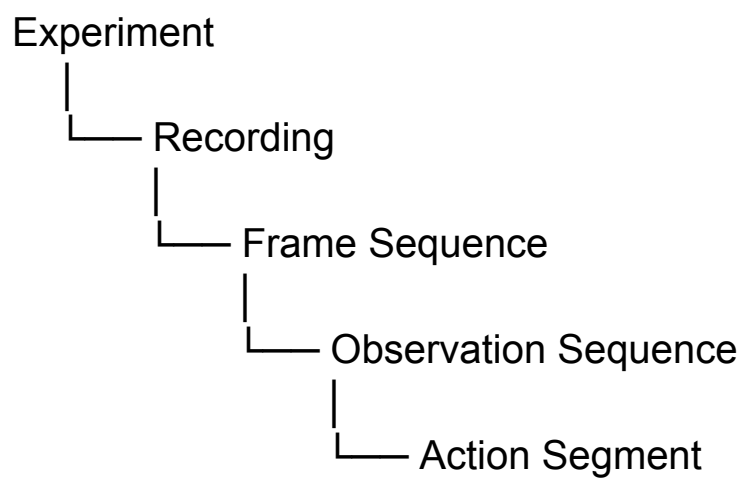
Our dataset should therefore have:

```
astra_dataset/  
|  
|— videos/  
|  
|— frames/  
|  
|— annotations/  
|  
|   |— objects/  
|   |— hands/  
|   |— poses/  
|   |— interactions/
```

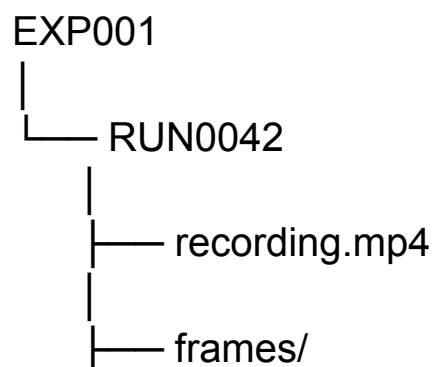


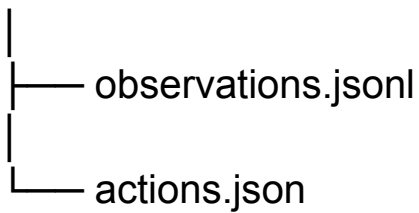
## 5. Dataset Unit Hierarchy

The fundamental hierarchy:



For example:





---

## 6. Recording Metadata

Every video needs metadata.

```
{
  "recording_id": "REC-00042",
  "experiment_id": "EXP001",
  "run_id": "RUN0042",

  "camera": {
    "id": "CAM01",
    "fps": 30,
    "width": 1920,
    "height": 1080
  },

  "environment": {
    "lighting": "normal",
    "background": "lab-A"
  },

  "procedure_version": "1.0",

  "operator": "simulated",

  "duration_seconds": 184.2
}
```

The metadata is just as important as the labels.

---

## 7. Dataset Annotation Layers

We should annotate progressively.

### Level 0 — Video

Raw recordings.

video.mp4

---

### Level 1 — Objects

frame

↓

bounding boxes

Example:

```
{
  "frame_id": 1024,
  "objects": [
    {
      "track_id": "obj-17",
      "class": "RED_COMPONENT",
      "bbox": [820, 430, 910, 520]
    }
  ]
}
```

---

## 8. Level 2 — Human / Hand / Pose

We need:

Human  
Hand  
Keypoints

Example:

human-01  
├── left\_hand  
├── right\_hand  
└── body\_pose

This gives the interaction system much richer information than raw bounding boxes.

---

## 9. Level 3 — Tracking

The dataset should preserve identity across frames.

frame 100 → obj-17  
frame 101 → obj-17  
frame 102 → obj-17  
frame 103 → obj-17

Rather than treating every detection independently.

This is essential for temporal learning.

---

## 10. Level 4 — Interaction Labels

Now we label relationships.

Example:

GRASP

between:

human-01  
hand-right  
obj-17

Contract:

```
{  
  "interaction": "GRASP",  
  "actor_id": "human-01",  
  "hand_id": "hand-right",  
  "object_id": "obj-17",  
  
  "start_frame": 1200,  
  "end_frame": 1230  
}
```

---

## 11. Level 5 — Action Segments

This is our most important temporal annotation.

Instead of:

frame 1200 = PICK

we label:

PICK  
start = 1200  
end = 1260

Example:

```
{  
  "action": "PICK",
```

```
"actor_id": "human-01",  
"object_id": "obj-17",  
  
"start_time": 40.0,  
"end_time": 42.0  
}
```

This lets us train temporal models properly.

---

## 12. Action Taxonomy

We should keep the first taxonomy small.

### **Primitive actions**

APPROACH  
TOUCH  
GRASP  
PICK  
MOVE  
PLACE  
RELEASE  
OPEN  
CLOSE  
MANIPULATE

Then higher-level actions can be composed from primitives.

For example:

PICK  
=  
APPROACH  
+  
GRASP  
+

LIFT

This gives us a hierarchical representation.

---

## 13. Don't Create 100 Classes

Initially we might be tempted to create:

```
PICK_RED
PICK_BLUE
PICK_YELLOW
PLACE_RED_TARGET_A
PLACE_RED_TARGET_B
...
```

Don't.

Instead:

```
ACTION = PICK
OBJECT = RED_COMPONENT
TARGET = TARGET_A
```

So:

Action=(verb,actor,object,target)Action = (verb, actor, object, target)

This dramatically reduces the number of classes.

---

## 14. Structured Action Representation

Our ML output should therefore look like:

```
{
```

```
"action": "PLACE",  
"actor_id": "human-01",  
"object_id": "obj-red",  
"target_id": "target-A",  
"confidence": 0.93  
}
```

Rather than:

class 27

The classifier predicts the semantic action.

The scene understanding system supplies the entities.

---

## 15. Dataset Schema

A sequence annotation might become:

```
{  
  "sequence_id": "SEQ-00192",  
  
  "recording_id": "REC-00042",  
  
  "start_frame": 1200,  
  "end_frame": 1260,  
  
  "actor": "human-01",  
  
  "action": {  
    "verb": "PICK",  
    "object": "RED_COMPONENT",  
    "target": null  
  },  
}
```

```
"label_source": "human",  
"quality": "verified"  
}
```

---

## 16. Label Sources

We should track how every label was produced.

```
label_source  
|  
├── HUMAN  
├── MODEL  
├── AUTO  
└── SYNTHETIC
```

Why?

Because:

human verified label

should not be treated the same as:

pseudo-label generated by model

---

## 17. Label Quality

Add:

quality

with:

UNVERIFIED

REVIEWED

VERIFIED

DISPUTED

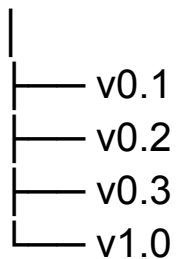
This gives us a proper annotation pipeline.

---

## 18. Dataset Versioning

Dataset versions should be immutable.

ASTRA-DATA



Each model must record the dataset version it trained on.

Example:

```
{
  "model": "activity-transformer",
  "version": "0.4",

  "dataset": "astra-actions",
  "dataset_version": "0.3"
}
```

---

## 19. Train / Validation / Test Split

**Do not split randomly by frame.**

That would create severe leakage.

Bad:

video A:

frames 1-100 → train

frames 101-150 → test

The model essentially sees the same scene/environment in both sets.

Instead:

Recording-level split

or preferably:

Experiment-level split

Example:

TRAIN

EXP001

EXP002

EXP003

EXP004

VALIDATION

EXP005

TEST

EXP006

EXP007

---

## 20. Generalization Test Set

I'd actually maintain **two test sets**.

**IID test**

Similar environment to training.

## OOD test

Different:

- lighting;
- camera position;
- object arrangement;
- operator;
- background;
- motion speed.

Then we can answer:

Does ASTRA-E actually generalize?

rather than simply:

Did the model memorize our lab?

---

## 21. Data Collection Strategy

For the SIH prototype, don't immediately try to collect thousands of hours.

Start with controlled data.

### Stage 1

5–10 experiments

with:

10–20 repetitions each

Then deliberately vary:

camera

lighting

speed

object position  
operator

---

## 22. Positive + Negative Examples

This is extremely important.

We need examples of:

correct PICK

but also:

false PICK

partial PICK

wrong object

wrong target

hesitation

repeated action

interrupted action

Otherwise the model will become overly optimistic.

---

## 23. Hard Negative Dataset

Create a dedicated dataset:

hard\_negatives/

Examples:

hand approaches object

but doesn't grasp

hand touches object

but doesn't pick

object moves  
without valid pick

person reaches toward wrong component

person performs similar-looking action

These examples are particularly valuable for reducing false positives.

---

## 24. Synthetic Data

This is where ASTRA-E can become interesting.

Because the environment is procedural, we can potentially generate synthetic training data.

Procedure



Simulation



Virtual astronaut



Virtual objects



Virtual camera



Synthetic video



Perfect labels

The labels are effectively free.

This is particularly attractive for:

- object detection;
  - pose;
  - interaction;
  - action segmentation.
- 

## 25. Synthetic → Real Pipeline

Don't train purely on synthetic data.

Use:

Synthetic



Pretraining



Real Dataset



Fine-tuning



Real-world Evaluation

This can dramatically reduce manual annotation requirements.

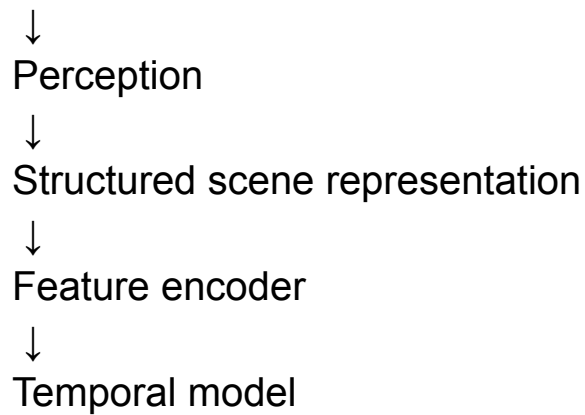
---

## 26. Feature Architecture

For temporal activity recognition, I would **not initially feed raw pixels directly into a huge video transformer.**

Instead:

Video



Example feature vector:

Human pose  
Object positions  
Hand positions  
Hand-object distance  
Object velocity  
Target distance  
Interaction state  
Previous action

This is more data-efficient for a structured procedural task.

---

## 27. Feature Vector

Conceptually:

$x_t = [P_t, H_t, O_t, D_{hand,obj}, D_{obj,target}, V_{obj}, I_t]$   
 $x_t = [P_t, H_t, O_t, D_{\{hand,obj\}}, D_{\{obj,target\}}, V_{\{obj\}}, I_t]$

where:

- $P_t$  = pose features;
- $H_t$  = hand features;
- $O_t$  = object state;
- $DD$  = spatial distances;

- $V_t$  = velocity;
- $I_{t|t}$  = interaction features.

Then:

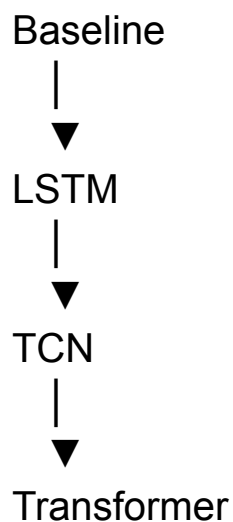
$X = [x_{t-k}, \dots, x_t]$   $X = [x_{t-k}, \dots, x_t]$

is fed into the temporal model.

---

## 28. Temporal Model Roadmap

We should benchmark progressively.



**Baseline**

Even:

MLP + temporal pooling

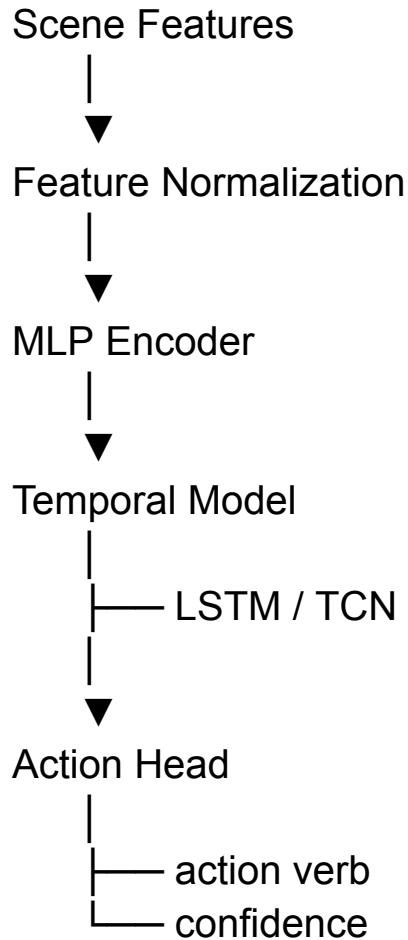
is useful.

If a Transformer only improves F1 by 1% while being 4× slower, we shouldn't use it.

---

## 29. Model Architecture

Initial architecture:



Entity information comes separately:

Object Head  
Target Head

Eventually we can investigate multimodal architectures.

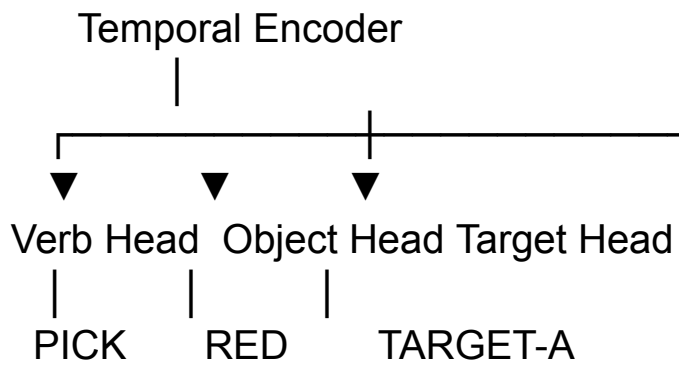
---

## 30. Multi-Head Prediction

Rather than:

PICK\_RED

use:



Mathematically:

$$L = \lambda_v L_v + \lambda_o L_o + \lambda_t L_t = \lambda_v L_v + \lambda_o L_o + \lambda_t L_t$$

where:

- $L_v$  = verb classification loss;
- $L_o$  = object classification loss;
- $L_t$  = target classification loss.

---

## 31. Class Imbalance

Some actions will naturally be much more common.

For example:

|       |     |
|-------|-----|
| MOVE  | 40% |
| PICK  | 15% |
| PLACE | 15% |
| WAIT  | 20% |
| ERROR | 2%  |

We should evaluate:

- class-weighted loss;

- focal loss;
- balanced sampling.

But don't blindly use oversampling—temporal correlations can make naive oversampling misleading.

---

## 32. Evaluation Metrics

Accuracy alone is insufficient.

For action recognition:

### **Classification**

Precision

Recall

F1

Macro-F1

Confusion Matrix

### **Temporal localization**

Segment IoU

Temporal mAP

Boundary error

### **System-level**

False violation rate

Missed violation rate

End-to-end latency

---

## 33. The Most Important Metric

For ASTRA-E, I'd prioritize:

False Violation Rate  $\boxed{\text{False Violation Rate}}$

because a false procedural alarm can be more damaging than an uncertain observation.

But we also need:

Missed Violation Rate  $\text{Missed Violation Rate}$

So evaluation becomes a tradeoff:

False Positive



False Negative

---

## 34. Confidence Calibration

Our model outputs:

$c \in [0, 1]$   $c \in [0, 1]$

But we shouldn't automatically interpret:

0.95

as:

95% chance of being correct.

We'll evaluate calibration using:

Reliability diagram

ECE

Brier score

Then tune thresholds for the actual operational requirement.

---

## 35. Uncertainty

Our system should explicitly support:

CONFIDENT  
UNCERTAIN

Example:

Action:  
PLACE

confidence:  
0.61

→ UNCERTAIN

Instead of:

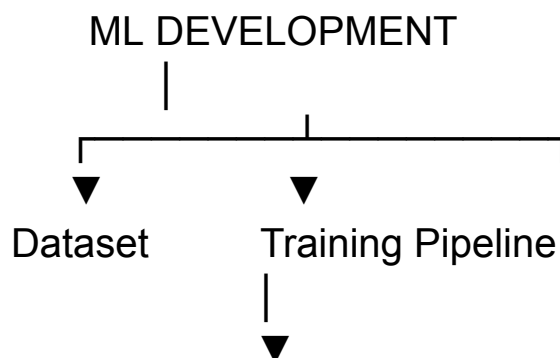
$0.61 > 0.5$   
→ ACCEPT

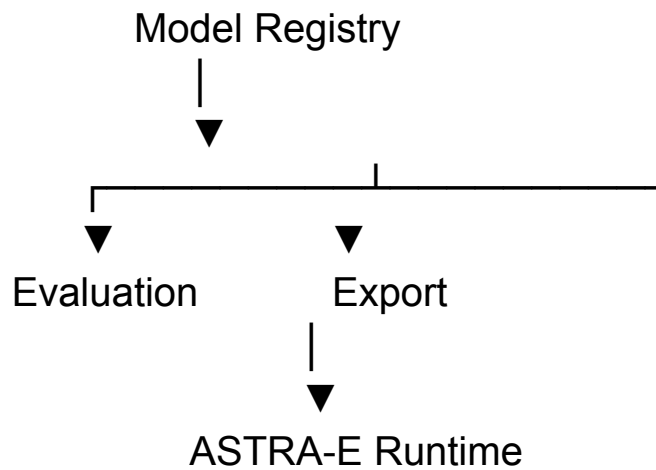
We can use hysteresis / temporal confirmation.

---

## 36. ML Runtime Architecture

Training and inference should be completely separated.





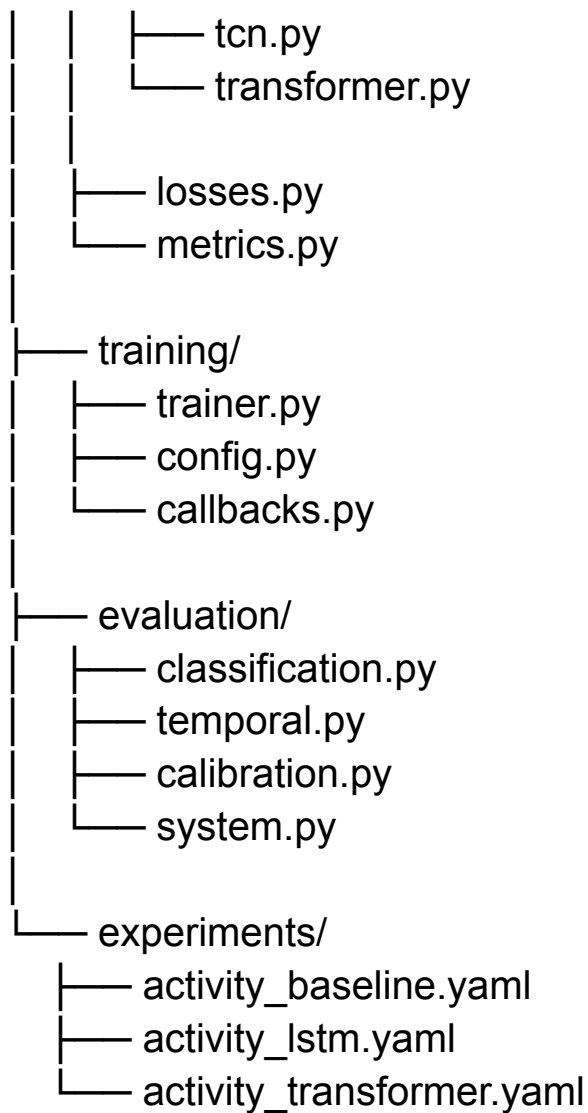
The runtime **never trains models**.

---

## 37. Training Repository Structure

I'd add this to the repo:

```
ml/
|
|-- datasets/
|   |-- schemas.py
|   |-- loaders.py
|   |-- transforms.py
|   |-- splits.py
|
|-- perception/
|   |-- detection/
|   |-- pose/
|
|-- activity/
|   |-- features.py
|   |-- datasets.py
|   |-- models/
|       |-- baseline.py
|       |-- lstm.py
```



This is separate from:

astra/

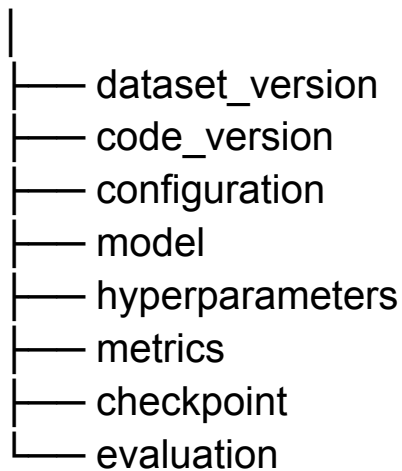
because training code should not contaminate the production runtime.

---

## 38. ML Experiment Tracking

Every training run should produce:

Experiment



At minimum:

```
{
  "run_id": "MLRUN-042",

  "dataset": "astra-actions-v0.3",

  "model": "lstm-v2",

  "epochs": 50,

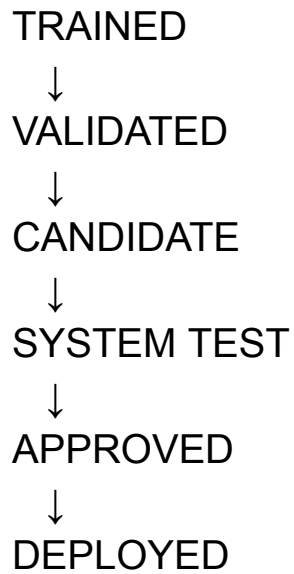
  "metrics": {
    "macro_f1": 0.91,
    "temporal_iou": 0.87,
    "ece": 0.04
  }
}
```

---

## 39. Model Promotion

Never automatically replace the production model just because a new model has higher validation accuracy.

Use:



The system test should include real experiment replay.

---

## 40. Model Card

Every deployed model should have:

model\_id  
version  
dataset  
training date  
architecture  
metrics  
known limitations  
hardware requirements  
latency

Example:

ASTRA-ACTIVITY-LSTM  
v0.3

Macro F1: 0.91

Latency: 18 ms

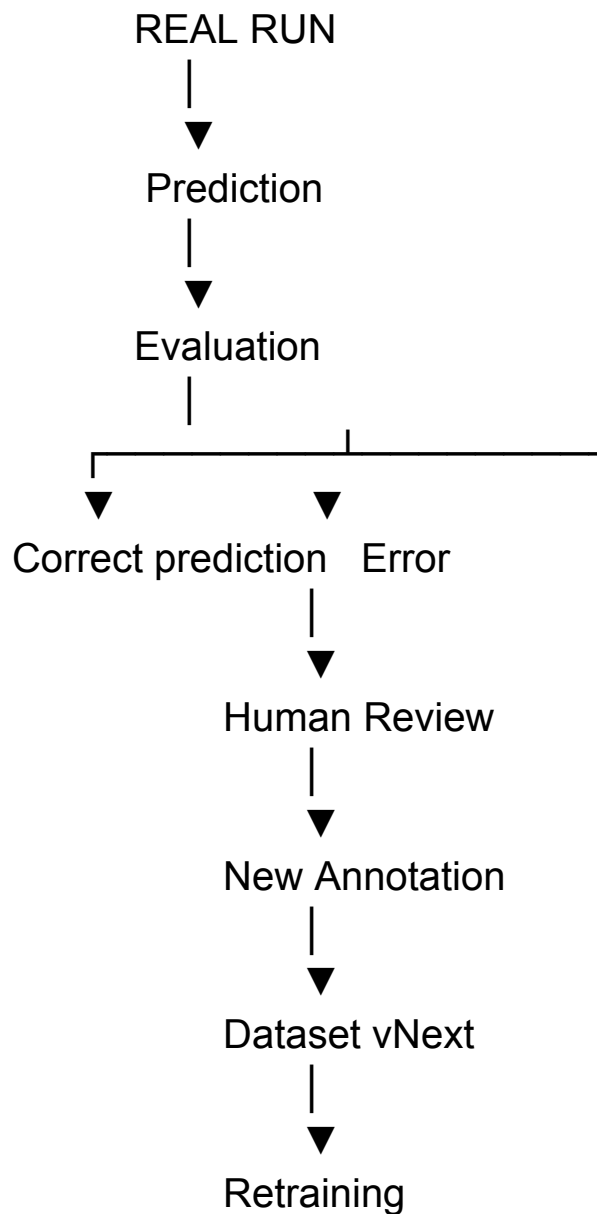
Dataset: v0.3

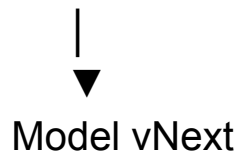
Known limitations:

- poor performance under severe occlusion
  - reduced accuracy under low lighting
- 

## 41. ML/Data Feedback Loop

This is where ASTRA-E can become genuinely strong.





This creates a continuous improvement loop.

---

## 42. Active Learning

We shouldn't manually label everything.

Select samples where:

confidence is low

or:

models disagree

or:

procedure engine rejects prediction

For example:

Model A → PICK 0.52

Model B → PLACE 0.44

This is a high-value sample for annotation.

---

## 43. Data Quality Gates

Before a recording enters training:

Video valid?



Metadata valid?  
↓  
Frames valid?  
↓  
Labels valid?  
↓  
Temporal boundaries valid?  
↓  
No train/test leakage?  
↓  
APPROVED

We should automate these checks.

---

## 44. Dataset Leakage Protection

We should generate a unique:

recording\_id  
experiment\_id  
environment\_id  
operator\_id

and make splitting happen **before preprocessing**.

Otherwise near-duplicate frames can accidentally cross the split.

---

## 45. Initial Dataset Target

For the SIH prototype, I'd target something like:

10–20 experiment recordings

↓

100–300 action sequences



10–20 action categories/primitive combinations

Then expand based on failure analysis.

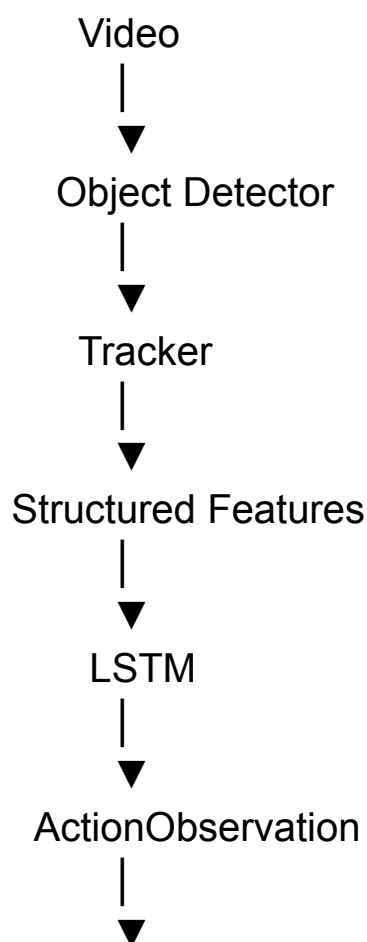
**Quality and diversity matter much more than raw video hours at this stage.**

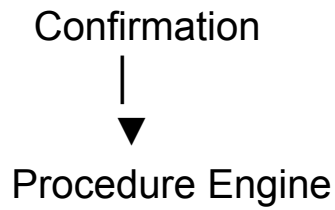
---

## 46. The ML Architecture We Should Actually Build First

Don't build the giant system immediately.

Our first ML MVP:





That gives us an end-to-end AI demonstration while keeping the research problem tractable.

---

## 47. Evolution Path

Then:

### **V0**

Rules

+

Pretrained perception

### **V1**

Detection

+

Tracking

+

LSTM

### **V2**

Better feature encoder

+

TCN/Transformer

+

uncertainty calibration

### **V3**

Multimodal temporal model

+

active learning

+

synthetic data

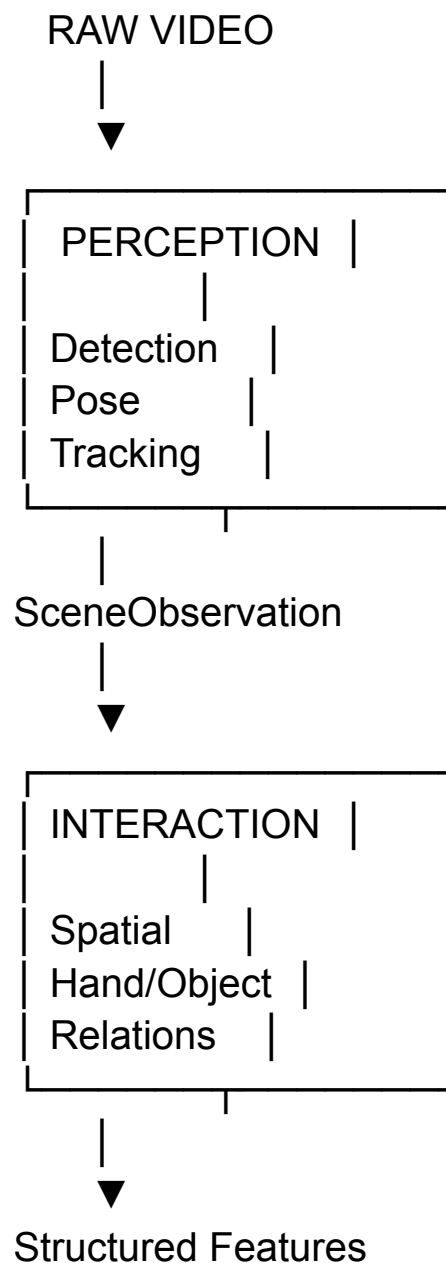
+

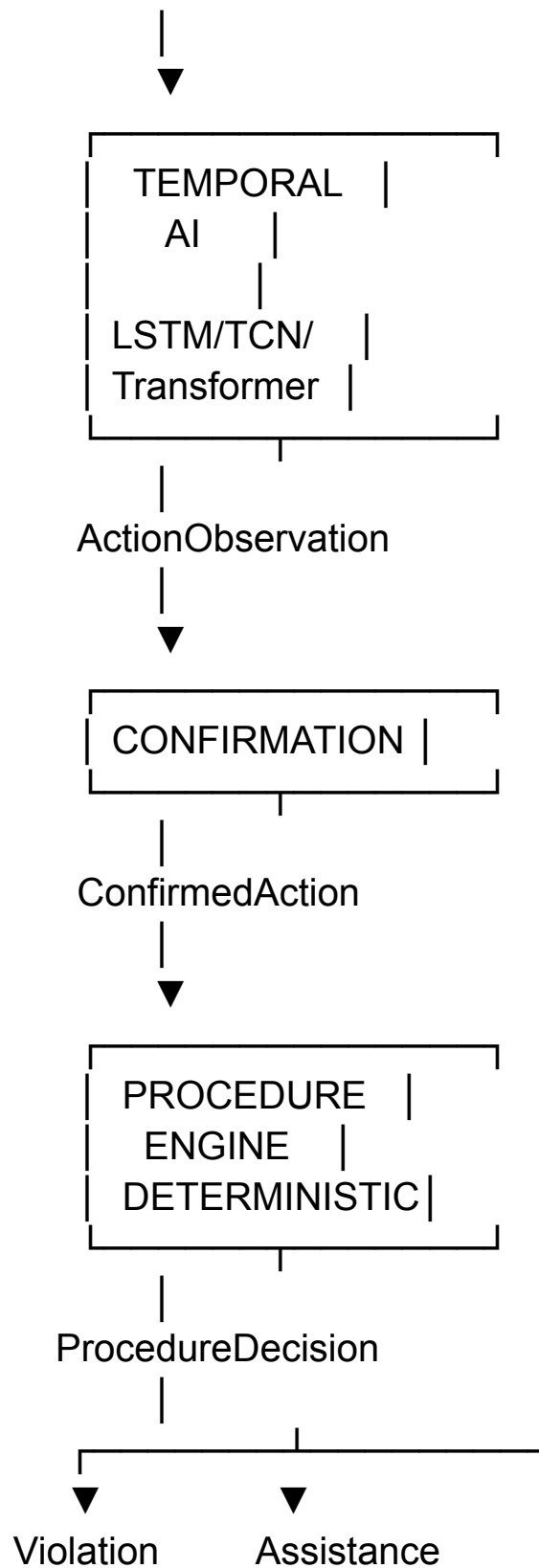
OOD evaluation

---

## 48. Final Architecture

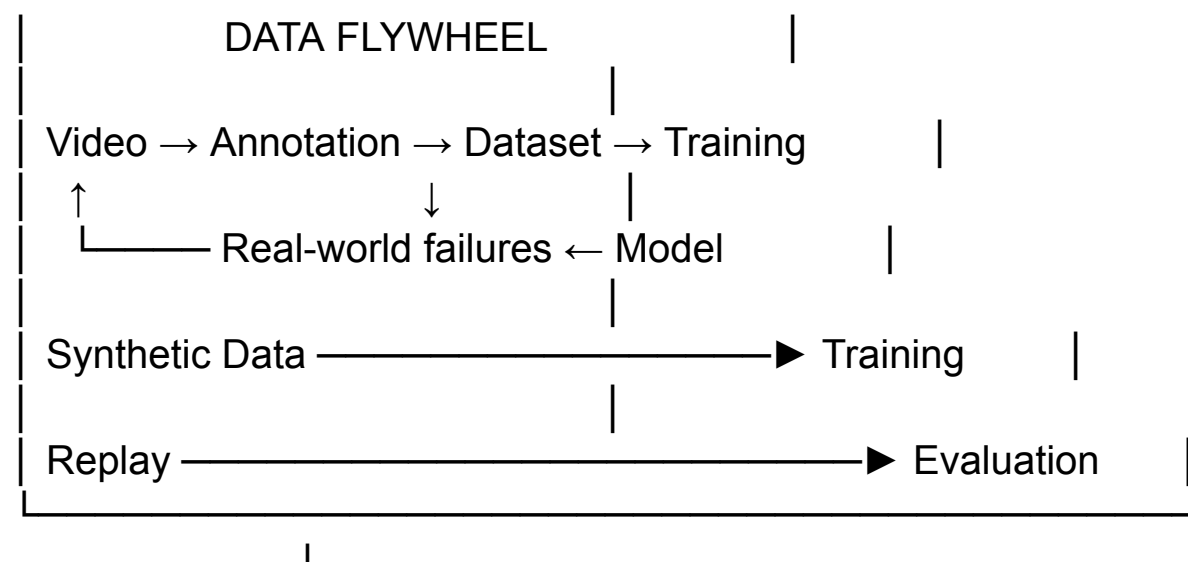
The complete ASTRA-E learning architecture becomes:





And underneath it:





# ASTRA-E Implementation Plan — v0.1

Our first objective is:

**Get one complete experiment running end-to-end with deterministic mock perception.**

Mock Video/Observations



Interaction



Action Observation



Confirmation



Procedure Engine



Violation Detection



Assistance



Event Log

Once that works, we replace the mocks with actual CV/ML.

---

## 1. Implementation Phases

| Phase | Component         | Goal                |
|-------|-------------------|---------------------|
| P0    | Project bootstrap | Runnable repository |

|            |                  |                              |
|------------|------------------|------------------------------|
| <b>P1</b>  | Contracts        | Freeze system interfaces     |
| <b>P2</b>  | Event Bus        | Component communication      |
| <b>P3</b>  | Procedure Engine | Core deterministic reasoning |
| <b>P4</b>  | Violation Engine | Detect procedural deviations |
| <b>P5</b>  | Mock Runtime     | Full end-to-end simulation   |
| <b>P6</b>  | Video Layer      | Real camera/video input      |
| <b>P7</b>  | Perception       | Detection + tracking         |
| <b>P8</b>  | Activity ML      | LSTM/TCN baseline            |
| <b>P9</b>  | Confirmation     | Temporal confidence          |
| <b>P10</b> | API              | Runtime observability        |
| <b>P11</b> | Dashboard        | Human interface              |
| <b>P12</b> | Evaluation       | Replay + benchmarks          |

The **P0–P5** system should work before we seriously touch ML.

---

## 2. P0 — Bootstrap

Start with:

```
astrae/  
├── apps/  
├── astra/  
├── configs/  
├── docs/  
├── experiments/  
├── ml/  
├── models/  
├── scripts/  
├── tests/  
├── data/  
├──  
├── .env.example  
├── .gitignore  
├── Dockerfile  
├── docker-compose.yml  
├── Makefile  
├── pyproject.toml  
└── README.md
```

Create the Python package:

```
astra/
```

and make every architectural module importable.

---

### 3. Dependency Policy

This is important.

We establish a dependency direction:

```
apps  
↓  
runtime
```

↓  
domain modules

↓  
contracts

Infrastructure:

storage

events

configuration

models

can support the modules, but the **procedure engine must remain independent of ML frameworks.**

Therefore:

procedure/

must **not** import:

torch

cv2

tensorflow

ultralytics

This gives us deterministic testing.

---

## 4. P1 — Contracts First

Create:

astra/contracts/

|— base.py

|— video.py

|— perception.py

- |— interaction.py
- |— activity.py
- |— procedure.py
- |— violation.py
- |— assistance.py
- |— experiment.py
- |— system.py
- |— errors.py

Use **Pydantic** for external/runtime data contracts.

For example:

```
class ActionObservation(BaseModel):
    observation_id: str
    timestamp: float

    actor_id: str
    action: str

    object_id: str | None = None
    target_id: str | None = None

    confidence: float

    source: str
```

This becomes a fundamental system object.

---

## 5. Don't Couple Contracts to Implementations

Bad:

```
class ActionObservation:
    model: YOLOModel
```

Good:

```
class ActionObservation:
    action: ActionType
    confidence: float
```

The contract says **what happened**.

The implementation decides **how we detected it**.

---

## 6. P2 — Event Bus

Implement:

```
astra/events/
├── bus.py
├── handlers.py
└── topics.py
```

Initial event topics:

```
class EventTopic(str, Enum):
    ACTION_OBSERVED = "action.observed"
    ACTION_CONFIRMED = "action.confirmed"

    PROCEDURE_STARTED = "procedure.started"
    PROCEDURE_TRANSITIONED = "procedure.transitioned"

    VIOLATION_DETECTED = "violation.detected"

    ASSISTANCE_ISSUED = "assistance.issued"

    EXPERIMENT_STARTED = "experiment.started"
```

```
EXPERIMENT_COMPLETED = "experiment.completed"
```

```
SYSTEM_ERROR = "system.error"
```

Start with an **in-process event bus**.

Don't introduce Kafka/RabbitMQ/etc. yet.

---

## 7. P3 — Procedure Engine

This is our first genuinely important implementation.

```
astra/procedure/  
├── engine.py  
├── state.py  
├── graph.py  
├── transition.py  
└── validator.py
```

The engine receives:

ConfirmedAction

and evaluates:

CurrentState

+

Action

↓

Transition

↓

NextState

---

## 8. State Machine

Example:

START

|  
| PICK RED



PICK\_RED

|  
| PLACE RED → TARGET\_A



PLACE\_RED

|  
| PICK BLUE



PICK\_BLUE



COMPLETE

But if:

PICK\_RED



PLACE\_BLUE

occurs:

Current State = PICK\_RED

Expected:

PLACE RED → TARGET\_A

Received:

PLACE BLUE

Result:

VIOLATION

This is deterministic.

---

## 9. Procedure Engine Interface

Something like:

```
class ProcedureEngine:
```

```
    def start(self, procedure: ProcedureDefinition) -> None:
```

```
        ...
```

```
    def process(
```

```
        self,
```

```
        action: ConfirmedAction
```

```
    ) -> ProcedureDecision:
```

```
        ...
```

```
    @property
```

```
    def state(self) -> ProcedureState:
```

```
        ...
```

And:

```
class ProcedureDecision(BaseModel):
```

```
    accepted: bool
```

```
    previous_state: str
```

```
    next_state: str | None
```

```
    violation: Violation | None = None
```

---

## 10. P4 — Violation Engine

```
astra/violation/  
├── detector.py  
└── classifier.py
```

Initially:

```
class ViolationDetector:
```

```
    def evaluate(  
        self,  
        decision: ProcedureDecision  
    ) -> Violation | None:  
        ...
```

Don't make the violation engine another AI system.

At this stage:

Procedure says expected X

+

Received Y

↓

VIOLATION

---

## 11. P5 — Mock Runtime

This is where things become exciting.

We'll create:

```
apps/runtime/main.py
```

with a synthetic experiment.

Input:

```
actions = [
```

```
Pick("RED"),  
Place("RED", "TARGET_A"),  
Pick("BLUE"),  
Place("BLUE", "TARGET_B"),  
]
```

Then intentionally inject:

```
Place("BLUE", "TARGET_A")
```

and verify the system catches it.

---

## 12. Expected Runtime

Something like:

|                 |  |
|-----------------|--|
| ASTRA-E RUNTIME |  |
|-----------------|--|

Experiment: EXP001

Run: RUN0001

[START] Procedure initialized

[OBS] PICK RED

[CONF] 0.98

[PROC] START → PICK\_RED ✓

[OBS] PLACE RED → TARGET\_A

[CONF] 0.96

[PROC] PICK\_RED → PLACE\_RED ✓

[OBS] PICK BLUE

[CONF] 0.94  
[PROC] PLACE\_RED → PICK\_BLUE ✓

[OBS] PLACE BLUE → TARGET\_A  
[CONF] 0.97

[VIOLATION]  
Expected target: TARGET\_B  
Received target: TARGET\_A

[ASSIST]  
Incorrect placement. Please use TARGET\_B.

[OBS] PLACE BLUE → TARGET\_B  
[CONF] 0.99

[PROC] → COMPLETE ✓

[RUN] EXP001 completed

If we can get this working, **we have the skeleton of the actual system.**

---

## 13. P6 — Real Video Layer

Only after the state machine works:

```
astra/video/  
├── camera.py  
├── capture.py  
├── buffer.py  
└── recorder.py
```

Implement:

MockCamera  
FileCamera  
OpenCVCamera

All implement:

```
class Camera(Protocol):  
  
    def start(self) -> None:  
        ...  
  
    def read(self) -> VideoFrame:  
        ...  
  
    def stop(self) -> None:  
        ...
```

Then runtime doesn't care whether the source is:

MP4  
webcam  
RTSP  
simulation

---

## 14. P7 — Perception

Now introduce actual CV.

```
video  
↓  
detector  
↓  
tracker  
↓  
scene observation
```

Start with:

### **Object detection**

Detect:

PERSON

HAND

COMPONENTS

TOOLS

TARGETS

Then tracking:

object\_id

track\_id

bbox

velocity

---

## **15. Important: Don't Train Yet**

For the first CV implementation:

Pretrained model



Fine-tune only if necessary

The purpose is to establish the pipeline.

We don't want to spend two weeks training a model only to discover our data contract is wrong.

---

## **16. P8 — Activity Model**

Once perception outputs structured observations:

SceneObservation[t]  
SceneObservation[t-1]  
SceneObservation[t-2]  
...

we build:

```
ml/activity/  
├── datasets.py  
├── features.py  
├── models/  
│   ├── baseline.py  
│   └── lstm.py  
├── losses.py  
└── metrics.py
```

First model:

MLP  
↓  
LSTM  
↓  
Action Head

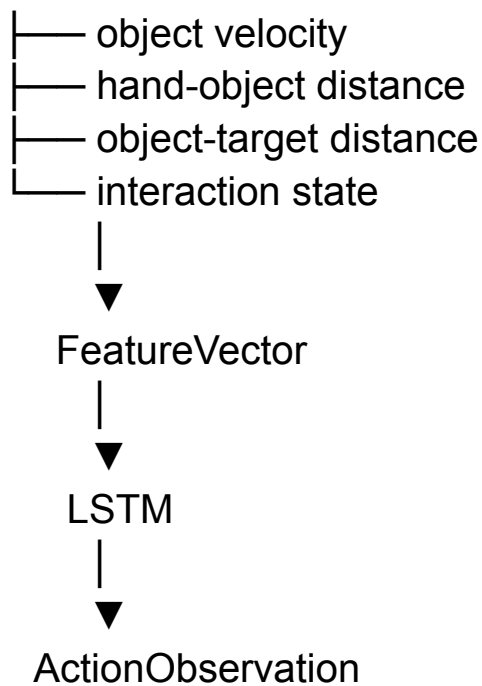
Don't start with a Transformer.

---

## 17. Feature Pipeline

Example:

```
Frame  
├──  
├── hand position  
├── object position  
└── target position
```



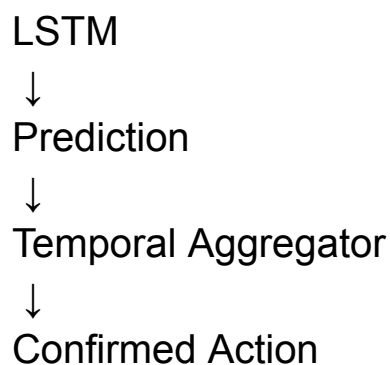
This gives us a strong baseline with limited training data.

---

## 18. P9 — Confirmation

The model shouldn't immediately trigger the procedure engine.

Instead:



For example:

t1 PICK 0.61  
t2 PICK 0.72  
t3 PICK 0.83

t4 PICK 0.91

becomes:

CONFIRMED PICK

This greatly reduces noisy predictions.

---

## 19. P10 — API

Then expose runtime state:

apps/api/

Initial endpoints:

GET /health

GET /system/status

GET /experiments

GET /experiments/{id}

GET /runs

GET /runs/{id}

GET /runs/{id}/events

GET /runs/{id}/violations

And WebSocket:

/ws/runs/{run\_id}

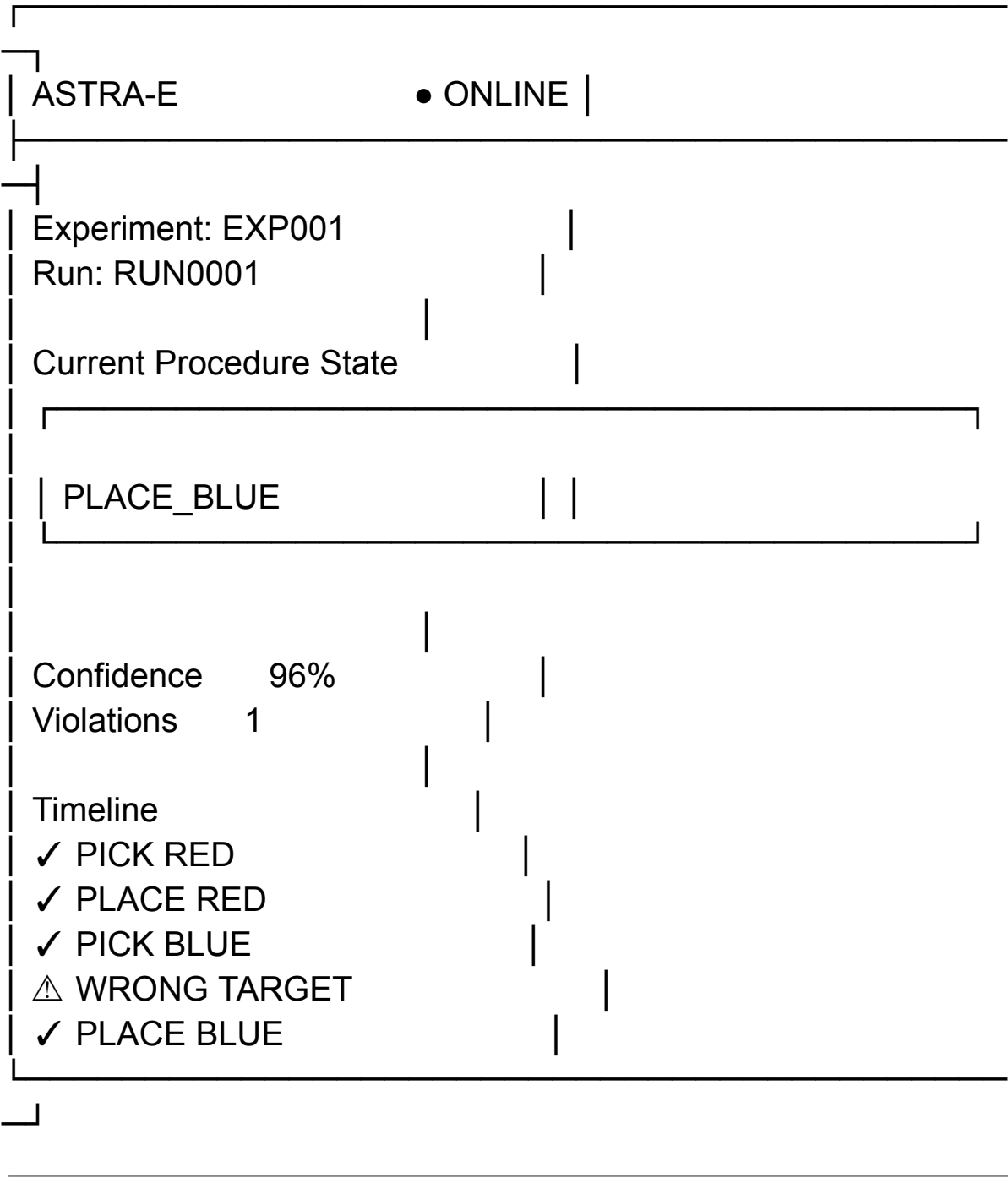
for live events.

---

# 20. P11 — Dashboard

Only after the backend is stable.

Dashboard:



# 21. P12 — Replay System

This should become one of our strongest engineering features.

Recorded Run



Replay Engine



Perception



Activity Model



Procedure Engine

Then:

```
python scripts/replay_run.py RUN0001
```

We can reproduce failures deterministically.

This is **extremely valuable** for debugging ML systems.

---

## 22. Implementation Milestones

I'd define actual Git milestones:

### **M0 — Skeleton**

Repository builds

Imports work

Tests run

### **M1 — Deterministic Core**

Contracts

Event bus

Procedure engine

Violation engine

### **M2 — End-to-End Mock**

Synthetic experiment



procedure



violation



assistance

### **M3 — Video**

MP4



frames



observations

### **M4 — Computer Vision**

Detection

Tracking

### **M5 — ML**

Features



LSTM



Action recognition

### **M6 — Real-Time**

Camera



CV



ML



Procedure

### **M7 — Interface**

# API

+

## WebSocket

+

# Dashboard

## M8 — Evaluation

## Replay

+

## metrics

+

# benchmark

+

## failure analysis

## 23. What We Should Code Right Now

I recommend we don't jump around.

Our first coding sprint should be:

## SPRINT 01 — ASTRA-E CORE

1. Create repository
2. Configure pyproject.toml
3. Create package structure
4. Implement contracts
5. Implement EventBus
6. Implement ProcedureGraph
7. Implement ProcedureEngine

